

TRƯỜNG ĐẠI HỌC GIAO THÔNG VẬN TẢI
KHOA CÔNG NGHỆ THÔNG TIN



ĐỒ ÁN TỐT NGHIỆP

ĐỀ TÀI

KIỂM THỬ TỰ ĐỘNG WEBSITE BOOKING HUB VỚI SELENIUM

Sinh viên thực hiện	: Kiều Thị Yến Vy
Mã sinh viên	: 5240063
Lớp	: CNTT1
Khóa	: 28.1
Ngành đào tạo	: Công nghệ thông tin
Giảng viên hướng dẫn	: Hoàng Văn Thông

Hà Nội – 2026

TRƯỜNG ĐẠI HỌC GIAO THÔNG VẬN TẢI
KHOA CÔNG NGHỆ THÔNG TIN



ĐỒ ÁN TỐT NGHIỆP

ĐỀ TÀI

KIỂM THỬ TỰ ĐỘNG WEBSITE BOOKING HUB VỚI SELENIUM

Sinh viên thực hiện	: Kiều Thị Yến Vy
Mã sinh viên	: 5240063
Lớp	: CNTT1
Khóa	: 28.1
Ngành đào tạo	: Công nghệ thông tin
Giảng viên hướng dẫn	: Hoàng Văn Thông

Hà Nội – 2026

LỜI CẢM ƠN

Lời đầu tiên em muốn được gửi lời cảm ơn sâu sắc đến thầy TS.Hoàng Văn Thông người đã trực tiếp hướng dẫn, chỉ bảo và giúp đỡ em rất nhiều trong suốt khoảng thời gian em thực hiện đồ án, đã luôn sát sao và đưa ra những lời khuyên để em có thể hoàn thiện đồ án một cách tốt nhất.

Em cũng xin chân thành cảm ơn tất cả các thầy cô giáo trong trường Đại học Giao thông vận tải nói chung và đặc biệt là các thầy cô trong Khoa Công nghệ thông tin nói riêng đã luôn là chỗ dựa vững chắc cho em. Các thầy cô không chỉ dạy dỗ, cho em kiến thức từ các môn đại cương đến các môn chuyên ngành mà còn luôn động viên, cổ vũ và định hướng giúp em phát triển trong tương lai.

Với điều kiện thời gian cũng như kinh nghiệm còn hạn chế, em không thể tránh được những thiếu sót. Em rất mong nhận được sự thông cảm và chỉ bảo, đóng góp ý kiến của thầy cô để em có thể thực hiện đồ án tốt hơn.

Cuối cùng, em xin kính chúc các thầy cô luôn mạnh khỏe, công tác tốt, tiếp tục chèo lái và đào tạo ra nhiều thế hệ học sinh xuất sắc hơn nữa.

Em xin chân thành cảm ơn!

Hà Nội, ngày tháng năm 2026

Sinh viên thực hiện

Kiều Thị Yến Vy

MỤC LỤC

LỜI CẢM ƠN.....	3
MỤC LỤC	4
DANH MỤC HÌNH VẼ	6
DANH MỤC BẢNG BIỂU.....	8
MỞ ĐẦU	10
CHƯƠNG 1 : CƠ SỞ LÝ THUYẾT	11
1.1. Tổng quan về khái niệm kiểm thử phần mềm.....	11
1.1.1. Khái niệm kiểm thử phần mềm.....	11
1.1.2. Mục tiêu của kiểm thử phần mềm.....	11
1.1.3. Vai trò của kiểm thử phần mềm.....	11
1.1.4. Các mức kiểm thử phần mềm	12
1.1.5. Các kĩ thuật kiểm thử	12
1.2. Kiểm thử tự động.....	12
1.2.1. Khái niệm kiểm thử tự động	12
1.2.2. Lợi ích của kiểm thử tự động	13
1.2.3. Hạn chế của kiểm thử tự động	13
1.2.4. Khi nào nên dùng kiểm thử tự động	14
1.2.5. Các công nghệ và phương pháp kiểm thử tự động.....	14
1.3. Kiểm thử phần mềm bằng Selenium.....	14
1.3.1. Tổng quan về Selenium.....	14
1.3.2. Selenium IDE	16
1.3.3. Selenium WebDriver.....	17
1.3.4. Lý do lựa chọn Selenium cho website Booking Hub	19
CHƯƠNG 2 : PHÂN TÍCH HỆ THỐNG VÀ XÂY DỰNG KỊCH BẢN KIỂM THỬ	20
2.1. Giới thiệu hệ thống Booking Hub.....	20
2.2. Xác định các chức năng cần kiểm thử	20
2.2.1. Nhóm chức năng phía khách hàng:.....	20
2.2.2. Nhóm chức năng phía quản trị viên (Admin).....	25
2.3. Kịch bản kiểm thử	34
2.3.1. Kịch bản kiểm thử phía khách hàng.....	34
2.3.2. Kịch bản kiểm thử phía quản trị viên.....	36

CHƯƠNG 3 : THỰC HIỆN KIỂM THỬ TỰ ĐỘNG BẰNG SELENIUM	41
3.1. Kiểm thử sử dụng Selenium IDE	41
3.1.1. Kiểm thử phía khách hàng	41
3.1.2. Kiểm thử phía quản trị viên.....	58
3.2. Kiểm thử sử dụng Selenium WebDriver	74
3.2.1. Kiểm thử phía khách hàng	74
3.2.2. Kiểm thử phía quản trị viên.....	86
KẾT LUẬN.....	101
DANH MỤC TÀI LIỆU THAM KHẢO	102

DANH MỤC HÌNH VẼ

Hình vẽ 2.1. Use Case Tìm chuyến.....	21
Hình vẽ 2.2. Use Case Đăng nhập.....	21
Hình vẽ 2.3. Use Case Đăng ký.....	22
Hình vẽ 2.4. Use Case Đặt vé.....	23
Hình vẽ 2.5. Use Case Vé của tôi.....	24
Hình vẽ 2.6. Use Case Quản lý nhà xe.....	25
Hình vẽ 2.7. Use Case Quản lý phương tiện.....	26
Hình vẽ 2.8. Use Case Quản lý chuyến xe.....	27
Hình vẽ 2.9. Use Case Quản lý địa điểm	29
Hình vẽ 2.10. Use Case Quản lý đặt vé.....	30
Hình vẽ 2.11. Use Case Quản lý dịch vụ	31
Hình vẽ 2.12. Use Case Quản lý sơ đồ ghế.....	32
Hình vẽ 2.13. Use Case Quản lý người dùng.....	33
Hình vẽ 2.14. Use Case Quản lý đánh giá	34
Hình vẽ 3.1. Test case HM-01.....	41
Hình vẽ 3.2. Test case HM-07.....	42
Hình vẽ 3.3. Test case HM-10.....	42
Hình vẽ 3.4. Test case LG-01	43
Hình vẽ 3.5. Test case LG-10.....	43
Hình vẽ 3.6. Test case LG-11	44
Hình vẽ 3.7. Test case RG-04.....	45
Hình vẽ 3.8. Test case RG-07.....	45
Hình vẽ 3.9. Test case RG-10.....	46
Hình vẽ 3.10. Test case RG-13.....	46
Hình vẽ 3.11. Test case SR-05	47
Hình vẽ 3.12. Test case SR-09	47
Hình vẽ 3.13. Test case SR-12	48
Hình vẽ 3.14. Test case BK-01.....	49
Hình vẽ 3.15. Test case BK-04.....	49
Hình vẽ 3.16. Test case BK-08.....	50
Hình vẽ 3.17. Test case BK-09.....	50
Hình vẽ 3.18. Test case PM-01	51
Hình vẽ 3.19. Test case PM-07	52
Hình vẽ 3.20. Test case PM-10	52
Hình vẽ 3.21. Test case MB-01	53
Hình vẽ 3.22. Test case MB-08.....	54
Hình vẽ 3.23. Test case MB-10.....	54
Hình vẽ 3.24. Test case MB-14.....	55
Hình vẽ 3.25. Test case PF-01	56
Hình vẽ 3.26. Test case PF-08.....	56
Hình vẽ 3.27. Test case PF-09.....	57
Hình vẽ 3.28. Test case PF-13.....	57
Hình vẽ 3.29. Test case AL-01.....	58
Hình vẽ 3.30. Test case AL-03.....	59

Hình vẽ 3.31. Test case AL-04.....	59
Hình vẽ 3.32. Test case AD-01	60
Hình vẽ 3.33. Test case AD-04	60
Hình vẽ 3.34. Test case LC-01	61
Hình vẽ 3.35. Test case LC-07	61
Hình vẽ 3.36. Test case LC-08	62
Hình vẽ 3.37. Test case LC-09	62
Hình vẽ 3.38. Test case BC-01	63
Hình vẽ 3.39. Test case BC-06.....	63
Hình vẽ 3.40. Test case BC-07.....	64
Hình vẽ 3.41. Test case BC-08.....	64
Hình vẽ 3.42. Test case BC-11	65
Hình vẽ 3.43. Test case VH-10	65
Hình vẽ 3.44. Test case VH-11	66
Hình vẽ 3.45. Test case VH-12	66

DANH MỤC BẢNG BIỂU

Bảng 2.1. Bảng mô tả Use Case Tìm chuyến	21
Bảng 2.2. Bảng mô tả Use Case Đăng nhập	22
Bảng 2.3. Bảng mô tả Use Case Đăng ký	22
Bảng 2.4. Bảng mô tả Use Case Đặt vé	23
Bảng 2.5. Bảng mô tả Use Case Vé của tôi	24
Bảng 2.6. Bảng mô tả Use Case Quản lý nhà xe	25
Bảng 2.7. Bảng mô tả Use Case Quản lý phương tiện	26
Bảng 2.8. Bảng mô tả Use Case Quản lý chuyến xe	27
Bảng 2.9. Bảng mô tả Use Case Quản lý địa điểm	29
Bảng 2.10. Bảng mô tả Use Case Quản lý đặt vé	30
Bảng 2.11. Bảng mô tả Use Case Quản lý dịch vụ	31
Bảng 2.12. Bảng mô tả Use Case Quản lý sơ đồ ghế	32
Bảng 2.13. Bảng mô tả Use Case Quản lý người dùng	33
Bảng 2.14. Bảng mô tả Use Case Quản lý đánh giá	34
Bảng 3.1. Test case HM-01	41
Bảng 3.2. Test case HM-07	42
Bảng 3.3. Test case HM-10	42
Bảng 3.4. Test case LG-01	43
Bảng 3.5. Test case LG-10	43
Bảng 3.6. Test case LG-11	44
Bảng 3.7. Test case RG-04	45
Bảng 3.8. Test case RG-07	45
Bảng 3.9. Test case RG-10	46
Bảng 3.10. Test case RG-13	47
Bảng 3.11. Test case SR-05	47
Bảng 3.12. Test case SR-09	48
Bảng 3.13. Test case SR-12	48
Bảng 3.14. Test case BK-01	49
Bảng 3.15. Test case BK-04	50
Bảng 3.16. Test case BK-08	50
Bảng 3.17. Test case BK-09	51
Bảng 3.18. Test case PM-01	51
Bảng 3.19. Test case PM-07	52
Bảng 3.20. Test case PM-10	52
Bảng 3.21. Test case MB-01	53
Bảng 3.22. Test case MB-08	54
Bảng 3.23. Test case MB-10	54
Bảng 3.24. Test case MB-14	55
Bảng 3.25. Test case PF-01	56
Bảng 3.26. Test case PF-08	56
Bảng 3.27. Test case PF-09	57
Bảng 3.28. Test case PF-13	57
Bảng 3.29. Test case AL-01	58

Bảng 3.30. Test case AL-03	59
Bảng 3.31. Test case AL-04	59
Bảng 3.32. Test case AD-01	60
Bảng 3.33. Test case AD-04.....	60
Bảng 3.34. Test case LC-01	61
Bảng 3.35. Test case LC-07	61
Bảng 3.36. Test case LC-08	62
Bảng 3.37. Test case LC-09	62
Bảng 3.38. Test case BC-01	63
Bảng 3.39. Test case BC-06	63
Bảng 3.40. Test case BC-07	64
Bảng 3.41. Test case BC-08	64
Bảng 3.42. Test case BC-11	65
Bảng 3.43. Test case VH-10.....	65
Bảng 3.44. Test case VH-11.....	66
Bảng 3.45. Test case VH-12.....	66

MỞ ĐẦU

Ngày nay, nhu cầu di chuyển bằng xe khách của mọi người ngày càng lớn, kéo theo đó là sự phát triển của rất nhiều ứng dụng và nền tảng website đặt vé trực tuyến để phục vụ nhu cầu của hành khách. Các website này ra đời giúp người dùng có thể tìm kiếm chuyến xe, so sánh giá vé và lựa chọn chỗ ngồi một cách nhanh chóng mà không cần phải ra tận bến xe như trước. Một website đặt vé xe khách chất lượng cần phải đáp ứng được các yêu cầu như: giao diện trực quan, dễ thao tác, hiển thị sơ đồ ghế chính xác, tìm kiếm thông tin điểm đi, điểm đến linh hoạt và hỗ trợ quản lý thông tin đặt vé hiệu quả.

Để xây dựng được một website chất lượng thì giai đoạn kiểm thử càng phải được thực hiện chặt chẽ. Với đề tài “Kiểm thử tự động website Booking Hub với Selenium” em sử dụng công cụ Selenium hỗ trợ quá trình kiểm thử tự động các chức năng của website một cách dễ dàng và chính xác nhất. Em sẽ giới thiệu tổng quan về Selenium, từ các thành phần cơ bản cho đến các tính năng chính của từng công cụ con như Selenium WebDriver, Selenium IDE. Bên cạnh đó, báo cáo còn đi sâu vào các bước cấu hình và triển khai các kịch bản kiểm thử tự động với Selenium, kết hợp thực tế trên website Booking Hub để người đọc có cái nhìn cụ thể và sinh động hơn về cách thức công cụ này hoạt động.

CHƯƠNG 1 : CƠ SỞ LÝ THUYẾT

1.1. Tổng quan về khái niệm kiểm thử phần mềm

1.1.1. Khái niệm kiểm thử phần mềm

Kiểm thử phần mềm là quá trình đánh giá và xác minh rằng một hệ thống phần mềm hoạt động đúng theo yêu cầu đã được xác định. Quá trình này bao gồm việc thực thi chương trình với các dữ liệu đầu vào khác nhau nhằm phát hiện lỗi, sai sót hoặc các hành vi không mong muốn.

Kiểm thử không chỉ đơn thuần là tìm lỗi mà còn nhằm đảm bảo rằng phần mềm đáp ứng được các tiêu chí về chất lượng như tính đúng đắn, độ tin cậy, hiệu năng và khả năng sử dụng.

1.1.2. Mục tiêu của kiểm thử phần mềm

Mục tiêu chính của việc kiểm thử phần mềm không chỉ đơn thuần là tìm ra lỗi mà còn là quá trình để đảm bảo website hoạt động một cách tốt nhất trước khi đến tay người dùng. Trước hết, quá trình này giúp em phát hiện kịp thời các lỗi (bug) còn tồn tại trong hệ thống, từ đó có phương án sửa chữa sớm để tránh những sai sót không đáng có khi đưa vào sử dụng thực tế. Bên cạnh đó, việc kiểm thử còn nhằm xác minh xem các chức năng của phần mềm đã đáp ứng đúng và đủ các yêu cầu nghiệp vụ đã đề ra ban đầu hay chưa.

Ngoài ra, thông qua kiểm thử, em có thể xác nhận được hệ thống vận hành ổn định trong nhiều điều kiện khác nhau, từ đó đưa ra cái nhìn khách quan để đánh giá chất lượng tổng thể của phần mềm. Việc kiểm tra chặt chẽ ngay từ giai đoạn phát triển cũng đóng vai trò quan trọng trong việc giảm thiểu tối đa các rủi ro phát sinh khi triển khai thực tế, giúp website Booking Hub trở nên tin cậy hơn và đảm bảo trải nghiệm tốt nhất cho người sử dụng.

1.1.3. Vai trò của kiểm thử phần mềm

Trong quy trình phát triển bất kỳ hệ thống nào, kiểm thử luôn đóng một vai trò hết sức quan trọng để đảm bảo chất lượng của sản phẩm đầu ra. Nếu một website như Booking Hub không được kiểm tra kỹ lưỡng, hệ thống có thể gặp phải những lỗi nghiêm trọng ngay khi vận hành, gây ảnh hưởng trực tiếp đến người dùng và uy tín của nhà xe. Việc thực hiện kiểm thử sớm không chỉ giúp em phát hiện ra các lỗi tiềm ẩn để giảm thiểu chi phí cũng như thời gian sửa chữa sau này, mà còn giúp tăng độ tin cậy và tính ổn định cho toàn bộ hệ thống.

Bên cạnh đó, kiểm thử phần mềm còn là yếu tố then chốt để nâng cao trải nghiệm của người sử dụng, giúp họ cảm thấy yên tâm hơn khi thực hiện các thao tác đặt vé hay thanh toán trực tuyến. Một quá trình kiểm thử bài bản với các kịch bản rõ ràng cũng sẽ hỗ trợ rất nhiều cho việc bảo trì và nâng cấp website sau này, giúp người phát triển dễ dàng kiểm soát các tính năng mới mà không làm ảnh hưởng đến những chức năng cũ đã hoạt động ổn định.

1.1.4. Các mức kiểm thử phần mềm

Thông thường, trong quy trình phát triển phần mềm, việc kiểm thử sẽ không thực hiện dàn trải mà được chia thành nhiều mức độ khác nhau để dễ dàng kiểm soát chất lượng. Đầu tiên là mức kiểm thử đơn vị (Unit Testing), đây là bước cơ bản nhất để kiểm tra các thành phần nhỏ nhất trong mã nguồn như các hàm hay các lớp (class). Mục tiêu là đảm bảo từng đơn vị code riêng lẻ đều chạy đúng chức năng của nó. Sau khi các đơn vị đã hoạt động ổn định, chúng ta sẽ tiến hành kiểm thử tích hợp (Integration Testing) để xem xét cách các module tương tác với nhau, giúp phát hiện sớm các lỗi phát sinh khi kết nối các thành phần lại thành một khối.

Tiếp đến là mức kiểm thử hệ thống (System Testing), ở giai đoạn này toàn bộ phần mềm sẽ được kiểm tra một cách tổng thể như một hệ thống hoàn chỉnh. Việc này giúp xác minh xem tất cả các tính năng có phối hợp nhịp nhàng và đáp ứng đúng cấu hình kỹ thuật đã đề ra hay không. Cuối cùng là mức kiểm thử chấp nhận (Acceptance Testing), đây thường là bước cuối cùng để người dùng hoặc khách hàng trực tiếp kiểm tra lại, nhằm xác nhận xem hệ thống đã thực sự thỏa mãn các yêu cầu nghiệp vụ và sẵn sàng để đưa vào sử dụng thực tế hay chưa. Việc phân chia rõ ràng các mức kiểm thử như vậy giúp quá trình phát hiện và xử lý lỗi trở nên khoa học và hiệu quả hơn rất nhiều.

1.1.5. Các kỹ thuật kiểm thử

Để quá trình kiểm thử đạt hiệu quả cao, người kiểm thử thường áp dụng các kỹ thuật khác nhau tùy vào mục đích và khả năng tiếp cận mã nguồn. Phổ biến nhất là kỹ thuật kiểm thử hộp đen (Black-box Testing), đây là phương pháp mà chúng ta chỉ tập trung vào việc kiểm tra các chức năng dựa trên đầu vào và kết quả đầu ra mà không cần quan tâm đến cấu trúc code bên trong hoạt động ra sao. Kỹ thuật này giúp đánh giá phần mềm dưới góc nhìn của người dùng cuối để xem hệ thống có đáp ứng đúng các yêu cầu nghiệp vụ hay không.

Ngược lại với hộp đen, kiểm thử hộp trắng (White-box Testing) lại đòi hỏi người kiểm thử phải nắm rõ cấu trúc bên trong và logic của chương trình. Ở kỹ thuật này, việc kiểm tra sẽ đi sâu vào các câu lệnh, vòng lặp và các nhánh điều kiện để đảm bảo các đường dẫn của code đều hoạt động chính xác. Ngoài ra, còn có sự kết hợp giữa hai phương pháp trên được gọi là kiểm thử hộp xám (Gray-box Testing). Với kỹ thuật này, người kiểm thử vừa có thể kiểm tra các chức năng bên ngoài như hộp đen, vừa có một phần hiểu biết về cấu trúc dữ liệu hoặc logic bên trong để từ đó thiết kế các kịch bản kiểm thử bao quát và hiệu quả hơn. Việc lựa chọn đúng kỹ thuật kiểm thử sẽ giúp quá trình tìm lỗi trở nên nhanh chóng và tối ưu hơn rất nhiều.

1.2. Kiểm thử tự động

1.2.1. Khái niệm kiểm thử tự động

Kiểm thử tự động hiểu đơn giản là việc mình không ngồi click tay hay nhập liệu thủ công nữa, mà thay vào đó là sử dụng các công cụ hỗ trợ hoặc viết code (script) để máy tự động thực hiện các bước kiểm tra. Với cách này, mình chỉ cần thiết lập trước các thao tác, dữ liệu đầu vào và kết quả mong đợi. Khi chạy, công cụ sẽ tự động thao tác

trên website, sau đó so sánh kết quả thực tế với cái mình dự tính ban đầu để báo xem kịch bản đó đạt (Pass) hay lỗi (Fail).

Điểm tiện lợi nhất của kiểm thử tự động chính là khả năng chạy đi chạy lại nhiều lần một cách nhanh chóng mà không lo bị nhầm lẫn hay mệt mỏi như khi làm thủ công. Điều này cực kỳ có ích cho những phần việc nhàm chán như kiểm thử hồi quy tức là phải kiểm tra lại các tính năng cũ mỗi khi mình sửa code mới để đảm bảo chúng không bị hỏng. Dù lúc đầu mình sẽ phải bỏ công sức ra để nghiên cứu và viết kịch bản, nhưng khi đã có bộ script hoàn chỉnh thì việc kiểm soát chất lượng phần mềm sẽ trở nên chuyên nghiệp và tiết kiệm thời gian hơn rất nhiều.

1.2.2. Lợi ích của kiểm thử tự động

So với việc kiểm thử bằng tay truyền thống, kiểm thử tự động mang lại rất nhiều lợi ích thiết thực, nhất là đối với những dự án cần sự ổn định cao. Đầu tiên phải kể đến khả năng tiết kiệm thời gian, vì các kịch bản sau khi đã viết xong có thể chạy đi chạy lại nhiều lần một cách nhanh chóng mà không cần tốn thêm công sức nhập liệu. Điều này cũng giúp tăng độ chính xác lên rất nhiều, vì máy móc sẽ thực hiện đúng y hệt các bước đã lập trình, giúp mình giảm thiểu tối đa những sai sót không đáng có do yếu tố con người như nhìn nhầm hay thao tác thiếu.

Một điểm cộng lớn nữa của kiểm thử tự động chính là tính tái sử dụng cao, khi mà các đoạn mã kiểm thử (test script) có thể được dùng lại cho nhiều phiên bản cập nhật khác nhau của phần mềm. Ngoài ra, kiểm thử tự động còn hỗ trợ rất tốt cho việc tích hợp vào quy trình CI/CD, giúp quá trình kiểm tra diễn ra liên tục mỗi khi có code mới được đẩy lên. Nhờ việc giải phóng được những thao tác lặp đi lặp lại, người kiểm thử có thể dành thêm thời gian để thiết kế thêm nhiều kịch bản khó, từ đó tăng độ bao phủ kiểm thử và đảm bảo mọi ngóc ngách của phần mềm đều được kiểm tra kỹ càng.

1.2.3. Hạn chế của kiểm thử tự động

Mặc dù có rất nhiều ưu điểm nhưng kiểm thử tự động không thể thay thế hoàn toàn được con người vì nó vẫn tồn tại một số hạn chế nhất định. Đầu tiên chính là chi phí đầu tư ban đầu khá cao, bởi mình sẽ tốn không ít thời gian và công sức để thiết lập môi trường cũng như viết code cho các kịch bản kiểm thử. Khác với kiểm thử thủ công là ai cũng có thể làm được ngay, kiểm thử tự động đòi hỏi người thực hiện phải có kiến thức về lập trình để có thể viết và vận hành được các đoạn script.

Bên cạnh đó, một khó khăn mà em thấy rất rõ là việc bảo trì script cực kỳ vất vả mỗi khi giao diện website có sự thay đổi. Chỉ cần một nút bấm hay một ô nhập liệu bị đổi tên hoặc đổi vị trí là mã nguồn kiểm thử có thể bị lỗi ngay lập tức, bắt buộc mình phải vào cập nhật lại code. Ngoài ra, kiểm thử tự động cũng không thể đánh giá được trải nghiệm người dùng tức là những cảm nhận về màu sắc, bố cục hay sự tiện lợi khi sử dụng website vì những thứ mang tính cảm quan này chỉ có con người mới có thể nhận xét chính xác được. Do đó, mình cần phải biết cách kết hợp linh hoạt giữa cả hai phương pháp để đạt được hiệu quả tốt nhất.

1.2.4. Khi nào nên dùng kiểm thử tự động

Trong thực tế, không phải lúc nào mình cũng đưa kiểm thử tự động vào vì nó còn tùy thuộc vào đặc điểm của dự án. Thời điểm thích hợp nhất để áp dụng là khi hệ thống cần phải kiểm tra đi kiểm tra lại nhiều lần các kịch bản giống nhau, giúp mình không phải ngồi thao tác thủ công gây nhàm chán. Đặc biệt là khi hệ thống đã có những chức năng hoạt động ổn định và ít thay đổi, việc viết script sẽ mang lại hiệu quả cao vì mình không phải tốn công sửa code liên tục.

Bên cạnh đó, kiểm thử tự động là lựa chọn hàng đầu cho việc kiểm thử hồi quy nhằm đảm bảo rằng những tính năng cũ vẫn chạy tốt sau khi mình cập nhật hoặc sửa lỗi ở các phần khác. Với những dự án có quy mô lớn, khối lượng dữ liệu đầu vào nhiều và phức tạp, việc để máy tự động xử lý sẽ giúp quá trình kiểm tra diễn ra nhanh chóng, chính xác và bao quát hơn rất nhiều so với sức người. Việc xác định đúng thời điểm áp dụng giúp mình tận dụng được tối đa sức mạnh của công cụ mà vẫn tiết kiệm được nguồn lực cho dự án.

1.2.5. Các công nghệ và phương pháp kiểm thử tự động

Hiện nay, trên thị trường có rất nhiều công nghệ hỗ trợ kiểm thử tự động mạnh mẽ, giúp người phát triển có thêm nhiều lựa chọn tùy theo đặc điểm của dự án. Bên cạnh Selenium, chúng ta có thể kể đến các công cụ phổ biến khác như Cypress, Playwright hay Katalon Studio. Các công cụ này đều có những ưu điểm riêng, ví dụ như Cypress và Playwright thường có tốc độ thực thi kịch bản rất nhanh và khả năng xử lý đợi (waiting) tốt hơn trên các trình duyệt hiện đại. Tuy nhiên, chúng lại thường yêu cầu kiến thức chuyên sâu về ngôn ngữ JavaScript hoặc bị giới hạn ở một số trình duyệt nhất định. Trong khi đó, các công cụ như Katalon Studio lại cung cấp giao diện kéo thả rất dễ dùng cho người mới, nhưng lại thường tốn chi phí bản quyền để sử dụng được đầy đủ các tính năng nâng cao.

Về phương pháp thực hiện, kiểm thử tự động thường tập trung vào hai hướng tiếp cận chính là kiểm thử theo hướng dữ liệu (Data-driven Testing) và kiểm thử theo hướng hành vi (Keyword-driven Testing). Với Data-driven, em có thể chạy một kịch bản kiểm thử với nhiều bộ dữ liệu khác nhau từ file Excel hay CSV, giúp kiểm tra được nhiều trường hợp nhập liệu mà không cần sửa code. Còn với Keyword-driven, kịch bản sẽ được xây dựng dựa trên các từ khóa hành động, giúp những người không quá rành về lập trình vẫn có thể hiểu và tham gia vào quá trình kiểm thử. Việc kết hợp linh hoạt giữa các công nghệ và phương pháp này sẽ giúp quá trình kiểm soát chất lượng website trở nên bao quát và hiệu quả hơn.

1.3. Kiểm thử phần mềm bằng Selenium

1.3.1. Tổng quan về Selenium

1.3.1.1. Khái niệm

Selenium có thể hiểu là một bộ công cụ mã nguồn mở chuyên dùng để hỗ trợ kiểm thử tự động cho các ứng dụng web. Điểm hay của công cụ này là nó cho phép mình mô phỏng lại y hệt các thao tác của một người dùng thực tế trên trình duyệt, chẳng hạn

như tự động nhấp chuột vào các nút bấm, nhập dữ liệu vào các ô trống hay tự động chuyển giữa các trang với nhau. Nhờ những thao tác giả lập này, mình có thể kiểm tra xem hệ thống hoạt động có chính xác và ổn định hay không mà không cần phải ngồi thực hiện bằng tay.

Một lý do nữa khiến Selenium được sử dụng rất phổ biến trong cộng đồng kiểm thử hiện nay là vì khả năng linh hoạt của nó. Công cụ này có thể chạy tốt trên nhiều trình duyệt khác nhau như Chrome, Firefox hay Microsoft Edge, đồng thời hỗ trợ rất nhiều ngôn ngữ lập trình quen thuộc như Java, Python, hay C#,... Chính vì sự đa dạng và mạnh mẽ như vậy, Selenium đã trở thành lựa chọn hàng đầu để em áp dụng vào việc kiểm thử cho dự án website của mình, giúp quá trình làm đồ án trở nên chuyên nghiệp và hiệu quả hơn.

1.3.1.2. Các thành phần của Selenium

Để hỗ trợ tốt nhất cho quá trình kiểm thử, Selenium được chia ra thành nhiều thành phần khác nhau, mỗi loại sẽ phục vụ cho một mục đích cụ thể. Đầu tiên là Selenium IDE, đây là một công cụ khá tiện lợi dưới dạng tiện ích mở rộng trên trình duyệt, cho phép em ghi lại (record) các thao tác mình vừa làm rồi thực hiện lại y hệt như vậy. Công cụ này rất phù hợp để tạo nhanh các kịch bản kiểm thử cơ bản mà mình không cần phải biết quá nhiều về lập trình.

Thành phần quan trọng nhất và cũng là trọng tâm trong đồ án của em chính là Selenium WebDriver. Đây là bộ thư viện mạnh mẽ cho phép em viết code để điều khiển trực tiếp trình duyệt, giúp xử lý được các kịch bản kiểm thử phức tạp và logic hơn nhiều so với việc chỉ ghi lại thao tác đơn thuần. Cuối cùng là Selenium Grid, một thành phần hỗ trợ việc chạy kiểm thử song song trên nhiều máy tính hoặc nhiều trình duyệt khác nhau cùng một lúc. Điều này cực kỳ hữu ích khi dự án lớn dần, giúp mình tối ưu hóa thời gian và kiểm tra được độ tương thích của website trên nhiều môi trường khác nhau một cách nhanh chóng.

1.3.1.3. Đặc điểm của Selenium

Lý do khiến Selenium được rất nhiều lập trình viên lựa chọn là nhờ vào những đặc điểm rất thực tế và linh hoạt. Điểm cộng lớn nhất chính là việc đây là một công cụ mã nguồn mở và hoàn toàn miễn phí, nên mình có thể dễ dàng tiếp cận, cài đặt mà không lo về vấn đề bản quyền. Bên cạnh đó, Selenium có khả năng tương thích cực kỳ tốt khi có thể chạy trên hầu hết các trình duyệt phổ biến hiện nay như Chrome, Firefox, Edge hay Safari, đồng thời hoạt động mượt mà trên nhiều hệ điều hành khác nhau từ Windows, macOS cho đến Linux.

Ngoài ra, Selenium hỗ trợ đa dạng các ngôn ngữ lập trình, giúp mình có thể thoải mái lựa chọn ngôn ngữ mà mình thạo nhất như Java, Python, C# hay JavaScript để viết script. Không chỉ dừng lại ở việc điều khiển trình duyệt, công cụ này còn có khả năng tích hợp rất tốt với các framework hỗ trợ kiểm thử khác (như TestNG, JUnit) và các công cụ trong quy trình CI/CD. Chính nhờ sự linh động này mà em có thể dễ dàng mở rộng các kịch bản kiểm thử và quản lý dự án một cách chuyên nghiệp hơn.

1.3.1.4. Vai trò của Selenium trong kiểm thử phần mềm

Trong quy trình đảm bảo chất lượng cho các ứng dụng web, Selenium đóng vai trò như một trợ thủ đắc lực giúp thay đổi cách thức kiểm thử truyền thống. Vai trò quan trọng nhất của nó là giúp tự động hóa hoàn toàn các tác vụ kiểm thử mang tính chất lặp đi lặp lại. Thay vì mình phải ngồi click chuột và nhập dữ liệu thủ công cho hàng chục kịch bản giống nhau, Selenium sẽ thay con người thực hiện các bước đó một cách tự động, giúp tiết kiệm rất nhiều thời gian và công sức.

Bên cạnh đó, việc sử dụng Selenium còn giúp nâng cao hiệu quả và độ chính xác cho toàn bộ quá trình kiểm thử. Vì máy móc thực hiện theo đúng kịch bản đã lập trình sẵn nên sẽ hạn chế được tối đa những sai sót do con người gây ra khi thiếu tập trung. Đặc biệt, Selenium đóng vai trò cực kỳ then chốt trong việc hỗ trợ kiểm thử hồi quy. Mỗi khi hệ thống có sự thay đổi hoặc cập nhật code mới, mình chỉ cần chạy lại bộ script có sẵn là có thể biết ngay các tính năng cũ có còn hoạt động ổn định hay không. Nhờ vậy, dự án có thể giảm bớt sự phụ thuộc quá nhiều vào kiểm thử thủ công, giúp người kiểm thử có thêm thời gian để tập trung vào những phần nghiệp vụ khó và quan trọng hơn.

1.3.2. Selenium IDE

1.3.2.1. Khái niệm

Selenium IDE có thể hiểu đơn giản là một công cụ kiểm thử tự động hoạt động dưới dạng một tiện ích mở rộng (extension) trên các trình duyệt phổ biến như Chrome hay Firefox. Điểm đặc biệt của công cụ này là khả năng cho phép mình ghi lại (record) toàn bộ các thao tác trực tiếp trên website, sau đó chuyển đổi chúng thành các kịch bản kiểm thử để có thể thực hiện lại nhiều lần một cách tự động. Chính nhờ cách tiếp cận rất trực quan và đơn giản này, Selenium IDE thường được nhiều người ưu tiên sử dụng trong giai đoạn đầu khi mới làm quen với khái niệm kiểm thử tự động.

Thay vì phải viết những dòng code phức tạp ngay từ đầu, công cụ này giúp mình nhanh chóng tạo ra các bộ test case cơ bản để kiểm tra các luồng đi chính của website. Mặc dù có cấu tạo gọn nhẹ, nhưng Selenium IDE vẫn hỗ trợ rất tốt trong việc quản lý các kịch bản kiểm thử, cho phép mình chỉnh sửa các bước thao tác hoặc thêm vào các câu lệnh kiểm tra để xác minh tính chính xác của dữ liệu hiển thị. Đây thực sự là một bước đệm quan trọng giúp em hiểu rõ hơn về luồng hoạt động của một kịch bản kiểm thử trước khi tiến xa hơn với những kỹ thuật lập trình kịch bản phức tạp trên WebDriver

1.3.2.2. Các chức năng chính của Selenium

Để hỗ trợ người dùng một cách tối đa, Selenium IDE được trang bị rất nhiều tính năng hữu ích, giúp việc tạo kịch bản trở nên nhanh chóng và dễ dàng hơn. Tính năng nổi bật nhất chính là ghi và phát lại thao tác, cho phép em ghi lại toàn bộ những gì mình vừa thực hiện trên trình duyệt và cho máy chạy lại y hệt một cách tự động. Trong quá trình đó, mình cũng có thể thoải mái chỉnh sửa kịch bản kiểm thử bằng cách thay đổi các bước, thêm bớt các điều kiện kiểm tra hoặc điều chỉnh dữ liệu đầu vào cho phù hợp với từng trường hợp cụ thể.

Bên cạnh đó, công cụ này còn hỗ trợ rất nhiều lệnh kiểm thử thông dụng như click, type, open hay các lệnh xác nhận như assert và verify, giúp em xây dựng được một quy trình kiểm tra đầy đủ các bước. Việc quản lý test case cũng trở nên khoa học hơn khi Selenium IDE cho phép gom nhóm nhiều kịch bản lại với nhau để dễ dàng theo dõi và thực thi. Đặc biệt, dù là công cụ đơn giản nhưng nó vẫn hỗ trợ điều khiển luồng với các cấu trúc logic như if, else hay vòng lặp while, giúp xử lý được cả những tình huống cần điều kiện phức tạp.

Trong lúc xây dựng kịch bản, em còn có thể sử dụng chức năng debug để chạy thử từng bước hoặc đặt các điểm dừng (breakpoint) nhằm tìm ra lỗi một cách chính xác nhất. Cuối cùng, một tính năng cực kỳ quan trọng là xuất kịch bản kiểm thử, cho phép em chuyển đổi những gì đã ghi lại được sang các ngôn ngữ lập trình như Java, Python hay C#,... Điều này tạo điều kiện thuận lợi để em nâng cấp kịch bản lên Selenium WebDriver sau này, giúp bài đồ án có tính kế thừa và chuyên nghiệp hơn.

1.3.2.3. Ưu và nhược điểm của Selenium IDE

Sau khi trực tiếp trải nghiệm và sử dụng Selenium IDE trong giai đoạn đầu của đồ án, em nhận thấy công cụ này có những ưu và nhược điểm rất rõ rệt. Về mặt ưu điểm, đây là công cụ cực kỳ dễ tiếp cận và sử dụng, vì nó không yêu cầu em phải có quá nhiều kiến thức chuyên sâu về lập trình. Nhờ giao diện trực quan, em có thể nhanh chóng tạo ra các kịch bản kiểm thử cơ bản chỉ bằng vài thao tác ghi lại trên trình duyệt, rất phù hợp cho những người mới bắt đầu bước chân vào lĩnh vực kiểm thử tự động như chúng em. Ngoài ra, việc hỗ trợ cài đặt trên nhiều trình duyệt phổ biến cũng giúp cho việc kiểm tra nhanh các tính năng trở nên thuận tiện hơn rất nhiều.

Tuy nhiên, bên cạnh những điểm tiện lợi thì Selenium IDE cũng có một số hạn chế nhất định khi áp dụng vào các dự án thực tế. Nhược điểm lớn nhất là công cụ này không thể đáp ứng tốt các kịch bản kiểm thử có logic phức tạp hoặc cần xử lý nhiều điều kiện ràng buộc. Thêm vào đó, các kịch bản được tạo ra bằng cách ghi thao tác thường rất khó bảo trì; chỉ cần giao diện hệ thống thay đổi nhẹ về vị trí nút bấm hay tên thẻ là script sẽ bị lỗi ngay lập tức. Công cụ này cũng khá hạn chế trong việc mở rộng quy mô kiểm thử hoặc tích hợp sâu vào các quy trình phát triển phần mềm chuyên nghiệp. Chính vì những lý do này, em chỉ sử dụng Selenium IDE như một công cụ hỗ trợ ban đầu và sau đó chuyển hướng sang các giải pháp mạnh mẽ hơn để đảm bảo chất lượng cho website Booking Hub.

1.3.3. Selenium WebDriver

1.3.3.1. Khái niệm

Khác với Selenium IDE, Selenium WebDriver là một công cụ mạnh mẽ hơn hẳn, cho phép lập trình viên trực tiếp xây dựng các kịch bản kiểm thử thông qua các mã lệnh code để điều khiển trình duyệt. Thay vì chỉ ghi lại các thao tác một cách máy móc, thông qua WebDriver, em có thể lập trình để mô phỏng lại mọi hành động của người dùng một cách chính xác, linh hoạt và thông minh hơn rất nhiều. Điều này giúp hệ thống xử lý được những tình huống thực tế phức tạp mà các công cụ ghi hình đơn thuần thường gặp khó khăn.

Chính nhờ khả năng can thiệp sâu vào cấu trúc của trang web và sự ổn định vượt trội, WebDriver đã trở thành công cụ được sử dụng phổ biến nhất trong các dự án kiểm thử tự động chuyên nghiệp hiện nay. Việc sử dụng WebDriver không chỉ giúp em nâng cao kỹ năng lập trình kịch bản mà còn giúp bài đồ án có độ tin cậy cao hơn, đáp ứng tốt yêu cầu kiểm tra kỹ lưỡng cho một hệ thống website có nhiều luồng nghiệp vụ như Booking Hub.

1.3.3.2. Các tính năng chính của Selenium WebDriver

Để trở thành công cụ quan trọng nhất trong bộ Selenium, WebDriver sở hữu nhiều tính năng mạnh mẽ giúp em có thể thực hiện những kịch bản kiểm thử từ đơn giản đến chuyên sâu. Trước hết phải kể đến khả năng hỗ trợ nhiều trình duyệt khác nhau như Chrome, Firefox, Edge,... giúp đảm bảo website luôn hoạt động tốt trên mọi nền tảng mà người dùng sử dụng. Bên cạnh đó, WebDriver còn hỗ trợ rất nhiều ngôn ngữ lập trình phổ biến như Java, Python, C# hay JavaScript. Điều này giúp em có thể tận dụng tối đa kiến thức lập trình sẵn có của mình để xây dựng bộ mã nguồn kiểm thử một cách tự nhiên và hiệu quả nhất.

Một trong những ưu điểm lớn nhất mà em thấy ở WebDriver là khả năng kiểm thử toàn bộ hệ thống. Nhờ việc tương tác trực tiếp với trình duyệt, công cụ này giúp mô phỏng chính xác từng hành vi của người dùng thực tế, từ đó cho phép em kiểm tra được những luồng nghiệp vụ hoàn chỉnh như tìm kiếm chuyến xe, chọn ghế cho đến khi đặt vé thành công. Để làm được điều đó, WebDriver cung cấp đa dạng cách xác định phần tử trên trang web thông qua ID, Name, CSS Selector hay XPath, giúp việc định vị các nút bấm hay ô nhập liệu trở nên cực kỳ linh hoạt.

Ngoài ra, trong quá trình viết script cho website Booking Hub, em còn có thể sử dụng cơ chế chờ linh hoạt (gồm implicit wait và explicit wait) để xử lý các vấn đề về tốc độ tải trang, tránh tình trạng script bị lỗi khi phần tử chưa kịp hiển thị. WebDriver cũng tỏ ra rất mạnh mẽ khi có thể xử lý các tình huống phức tạp thường gặp như các cửa sổ popup, iframe hay việc chuyển đổi giữa nhiều tab trình duyệt khác nhau. Cuối cùng, với khả năng tích hợp cao, em có thể dễ dàng kết hợp WebDriver với các framework như TestNG hay JUnit để quản lý báo cáo và đồng bộ vào quy trình phát triển chuyên nghiệp, giúp bài đồ án mang tính thực tiễn và hoàn thiện hơn.

1.3.3.3. Ưu nhược điểm của Selenium WebDriver

Trong quá trình nghiên cứu để đưa vào áp dụng cho đồ án, em nhận thấy Selenium WebDriver sở hữu những ưu điểm vượt trội nhưng cũng đi kèm với một số thách thức nhất định. Về mặt ưu điểm, đây là một công cụ mã nguồn mở hoàn toàn miễn phí, giúp em có thể tận dụng đầy đủ các tính năng hiện đại nhất mà không phải lo lắng về chi phí bản quyền. Quan trọng hơn, WebDriver có khả năng kiểm thử các hệ thống phức tạp và mô phỏng rất sát hành vi của người dùng thực tế, giúp các kịch bản kiểm thử trên website Booking Hub của em trở nên tin cậy và có giá trị thực tiễn cao hơn. Bên cạnh đó, nhờ vào cộng đồng hỗ trợ lớn và nguồn tài liệu phong phú, em có thể dễ dàng tìm kiếm giải pháp cho các vấn đề kỹ thuật phát sinh, cũng như dễ dàng mở rộng và tích hợp thêm các thư viện bổ trợ để hoàn thiện bộ script của mình.

Tuy nhiên, đi cùng đó là một số hạn chế mà người dùng cần phải lưu ý. Trước hết, WebDriver đòi hỏi người thực hiện phải có kiến thức về lập trình để có thể viết và quản lý mã nguồn kiểm thử hiệu quả. Quá trình thiết lập môi trường ban đầu cũng tương đối phức tạp, đòi hỏi em phải cấu hình kỹ lưỡng từ thư viện cho đến các driver tương ứng với từng trình duyệt. Một điểm bất tiện nữa là Selenium WebDriver không có sẵn công cụ báo cáo trực quan, vì thế em cần phải tích hợp thêm các thư viện bên ngoài để có được báo cáo chi tiết cho đề án. Cuối cùng, công cụ này chỉ tập trung vào ứng dụng web và không hỗ trợ kiểm thử các ứng dụng mobile native trực tiếp, điều này đòi hỏi phải có các công cụ bổ trợ khác nếu muốn mở rộng phạm vi kiểm thử sau này.

1.3.4. Lý do lựa chọn Selenium cho website Booking Hub

Sau khi dành thời gian tìm hiểu giữa các công cụ kiểm thử tự động hiện nay, em đã quyết định chọn Selenium để thực hiện kiểm thử cho website Booking Hub. Quyết định này xuất phát từ những đặc điểm rất thực tế, giúp một sinh viên như em có thể triển khai đề án một cách thuận lợi và chuyên nghiệp nhất.

Trước hết, tính chất mã nguồn mở và miễn phí của Selenium là một điểm cộng rất lớn. Việc tiếp cận được một bộ công cụ mạnh mẽ mà không phải lo lắng về chi phí bản quyền giúp em có thể thoải mái cài đặt, nghiên cứu và thử nghiệm các kịch bản kiểm thử trên máy cá nhân mà không gặp bất kỳ rào cản nào. Bên cạnh đó, sự linh hoạt và khả năng tương thích cao của công cụ này cũng đóng vai trò then chốt. Vì website đặt vé xe khách Booking Hub hướng tới nhiều đối tượng người dùng, việc đảm bảo hệ thống chạy tốt trên mọi trình duyệt như Chrome, Firefox hay Microsoft Edge là cực kỳ cần thiết. Selenium cho phép em kiểm tra tính ổn định của giao diện đặt vé và sơ đồ chọn ghế trên nhiều nền tảng khác nhau chỉ với một bộ kịch bản duy nhất.

Khả năng tùy biến bằng ngôn ngữ lập trình cũng là lý do quan trọng khiến em tin tưởng lựa chọn công cụ này. Selenium hỗ trợ viết kịch bản bằng Java ngôn ngữ mà em đã có nền tảng từ đó giúp em chủ động hơn trong việc xử lý các logic nghiệp vụ của web. Ngoài ra, một yếu tố thực tế không thể bỏ qua là cộng đồng hỗ trợ lớn. Trong quá trình thực hiện đề án, nếu gặp phải những kịch bản khó hay lỗi kỹ thuật phát sinh, em có thể nhanh chóng tìm thấy lời giải trên các diễn đàn công nghệ lớn. Điều này giúp em tự tin hơn trong việc xử lý vấn đề và đảm bảo tiến độ hoàn thành đề án đúng hạn.

Selenium không chỉ đơn thuần là một công cụ hỗ trợ mà còn là môi trường tốt để em rèn luyện tư duy lập trình và cách quản lý chất lượng cho một sản phẩm website thực tế. Việc áp dụng Selenium giúp quy trình kiểm thử cho dự án Booking Hub của em trở nên bài bản, tin cậy và đạt hiệu quả cao hơn

CHƯƠNG 2 : PHÂN TÍCH HỆ THỐNG VÀ XÂY DỰNG KỊCH BẢN KIỂM THỬ

2.1. Giới thiệu hệ thống Booking Hub

Để có thể triển khai các kịch bản kiểm thử tự động một cách sát thực tế, em đã lựa chọn kiểm thử hệ thống Booking Hub. Đây là một nền tảng đặt vé xe khách trực tuyến, được thiết kế nhằm giúp người dùng có thể dễ dàng tìm kiếm các chuyến xe, tự do lựa chọn vị trí ghế ngồi mong muốn, tiến hành đặt vé và thực hiện thanh toán trên môi trường web.

Mục tiêu cốt lõi của hệ thống là tối ưu hóa trải nghiệm người dùng, giúp quy trình đặt vé trở nên nhanh chóng, tiện lợi và đảm bảo mọi thông tin về giá vé, lịch trình đều minh bạch. Không chỉ dừng lại ở phía người dùng, Booking Hub còn đóng vai trò là một công cụ quản lý đắc lực cho phía nhà xe trong việc điều hành các chuyến xe, theo dõi doanh thu và quản lý thông tin khách hàng một cách khoa học.

Về mặt vận hành, hệ thống được phân chia rõ ràng thành hai nhóm đối tượng sử dụng chính với các chức năng chuyên biệt:

- Khách hàng: Đây là nhóm người dùng phổ thông, sử dụng hệ thống để tra cứu thông tin và thực hiện các bước đặt vé cho nhu cầu di chuyển của cá nhân.
- Quản trị viên (Admin): Đây là nhóm người dùng có quyền hạn cao hơn, chịu trách nhiệm quản lý toàn bộ dữ liệu hệ thống, từ việc cập nhật thông tin chuyến xe cho đến việc giám sát các hoạt động vận hành chung của nền tảng.

Chính nhờ sự phân cấp người dùng và các luồng nghiệp vụ rõ ràng như vậy, Booking Hub trở thành một môi trường lý tưởng để em áp dụng các kịch bản kiểm thử tự động với Selenium, từ các chức năng cơ bản như đăng nhập đến các quy trình phức tạp như đặt vé và quản trị dữ liệu.

2.2. Xác định các chức năng cần kiểm thử

Dựa trên các yêu cầu hệ thống đã phân tích, các chức năng chính cần kiểm thử của hệ thống Booking Hub được xác định như sau:

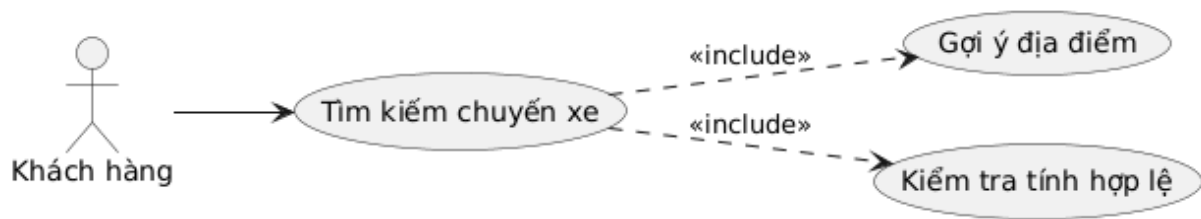
2.2.1. Nhóm chức năng phía khách hàng:

2.2.1.1. Tìm kiếm chuyến xe theo điểm đi, điểm đến, ngày khởi hành

– Mô tả chức năng

Chức năng này cho phép khách hàng tra cứu thông tin về các chuyến xe khách dựa trên nhu cầu về hành trình và thời gian. Hệ thống sẽ truy vấn dữ liệu để lọc ra các chuyến xe thỏa mãn điều kiện và hiển thị đầy đủ các thông tin cần thiết giúp khách hàng so sánh và đưa ra quyết định đặt vé

– Biểu đồ Use Case



Hình vẽ 2.1. Use Case Tìm chuyến

Bảng 2.1. Bảng mô tả Use Case Tìm chuyến

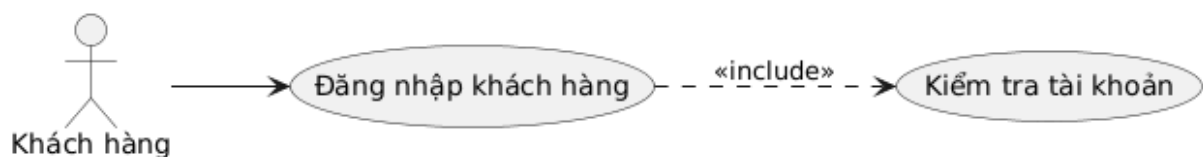
Tên Use Case	Tìm kiếm chuyến xe theo điểm đi, điểm đến, ngày khởi hành
Tác nhân	Khách hàng
Mô tả	Người dùng thực hiện cung cấp thông tin lộ trình và thời gian để hệ thống truy vấn các chuyến xe khả dụng trong cơ sở dữ liệu.
Điều kiện	Người dùng đã truy cập thành công vào website Booking Hub (Trang chủ hoặc trang Tìm kiếm).
Luồng sự kiện chính	- Người dùng lựa chọn điểm đi và điểm đến. - Người dùng chọn ngày đi. - Người dùng nhấn nút “Tìm kiếm”. - Hệ thống thực hiện kiểm tra các ràng buộc dữ liệu. - Hệ thống tìm kiếm các chuyến xe. - Hệ thống trả về danh sách các chuyến xe thỏa mãn điều kiện lên màn hình.
Luồng ngoại lệ	- Trường hợp bỏ trống: Nếu người dùng chưa nhập một trong các thông tin bắt buộc, hệ thống hiển thị cảnh báo. - Trường hợp không có kết quả: Nếu không có chuyến xe nào chạy trong ngày đã chọn, hệ thống hiển thị thông báo “Không có chuyến xe nào phù hợp”.
Kết quả	Danh sách các chuyến xe thỏa mãn điều kiện

2.2.1.2. Đăng nhập

– Mô tả chức năng

Đây là chức năng bắt buộc khách hàng phải thực hiện nếu muốn đặt vé trên hệ thống Booking Hub. Việc đăng nhập giúp hệ thống biết được ai đang đặt vé để lưu thông tin đặt vé và quản lý lịch sử chuyến đi.

– Biểu đồ Use Case



Hình vẽ 2.2. Use Case Đăng nhập

Bảng 2.2. Bảng mô tả Use Case Đăng nhập

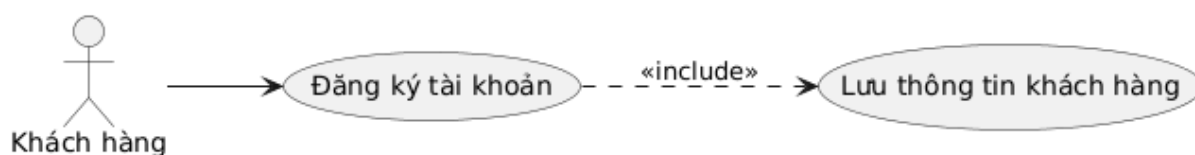
Tên Use Case	Đăng nhập dành cho Khách hàng
Tác nhân	Khách hàng
Mô tả	Khách hàng điền thông tin tài khoản để đăng nhập vào hệ thống.
Điều kiện	Khách hàng đã có tài khoản trên hệ thống.
Luồng sự kiện chính	- Khách hàng bấm vào nút "Đăng nhập" trên thanh menu hoặc khi hệ thống yêu cầu. - Hệ thống hiển thị giao diện đăng nhập. - Khách hàng nhập thông tin. - Khách hàng bấm nút “Đăng nhập”. - Hệ thống kiểm tra thông tin đăng nhập. - Hệ thống thông báo đăng nhập thành công.
Luồng ngoại lệ	- Trường hợp nhập sai: Nếu thông tin đăng nhập không đúng, hệ thống cảnh báo cho người dùng. - Trường hợp để trống: Nếu người dùng không nhập thông tin, hệ thống cảnh báo cho người dùng.
Kết quả	Khách hàng vào được trạng thái đã đăng nhập và có thể thực hiện các chức năng đặt vé, xem lịch sử đặt vé.

2.2.1.3. Đăng ký

– Mô tả chức năng

Để có thể sử dụng các dịch vụ của Booking Hub, đặc biệt là đặt vé, người dùng mới cần phải thông qua bước đăng ký tài khoản. Chức năng này giúp hệ thống thu thập các thông tin cơ bản của khách hàng.

– Biểu đồ Use Case



Hình vẽ 2.3. Use Case Đăng ký

Bảng 2.3. Bảng mô tả Use Case Đăng ký

Tên Use Case	Đăng ký tài khoản khách hàng
Tác nhân	Khách hàng
Mô tả	Người dùng cung cấp thông tin cá nhân để tạo tài khoản mới trên hệ thống nhằm mục đích đăng nhập và đặt vé xe.
Điều kiện	Người dùng chưa có tài khoản trên hệ.
Luồng sự kiện chính	- Người dùng bấm vào nút “Đăng ký” trên trang đăng nhập. - Hệ thống hiển thị giao diện đăng nhập. - Người dùng điền đầy đủ các thông tin theo yêu cầu của hệ thống.

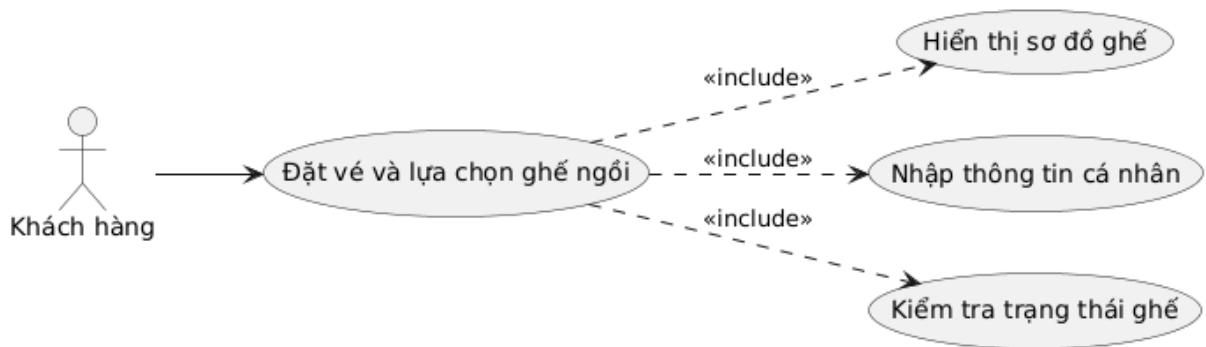
	- Người dùng bấm vào nút “Đăng ký” để gửi thông tin. - Hệ thống kiểm tra xem các thông tin có hợp lệ không. - Hệ thống lưu thông tin và thông báo người dùng đã đăng ký thành công.
Luồng ngoại lệ	- Trường hợp đăng ký không thành công, hệ thống hiển thị thông tin cảnh báo cho người dùng
Kết quả	Hệ thống tạo mới một tài khoản và khách hàng có thể dùng tài khoản đó để đăng nhập.

2.2.1.4. Đặt vé

– Mô tả chức năng

Đây là chức năng quan trọng nhất trong luồng nghiệp vụ của khách hàng trên hệ thống Booking Hub. Chức năng này cho phép người dùng tương tác với sơ đồ ghế của xe để lựa chọn vị trí chỗ ngồi mong muốn. Ngoài việc chọn ghế, người dùng còn thực hiện chọn dịch vụ cần thiết và cung cấp các thông tin cá nhân để tiến hành đặt vé.

– Biểu đồ Use Case



Hình vẽ 2.4. Use Case Đặt vé

Bảng 2.4. Bảng mô tả Use Case Đặt vé

Tên Use Case	Đặt vé
Tác nhân	Khách hàng
Mô tả	Người dùng thực hiện chọn vị trí ghế trên sơ đồ ghế, cung cấp thông tin đặt vé và xác nhận để hệ thống kiểm tra và đặt vé.
Điều kiện	- Người dùng đã chọn được một chuyến xe cụ thể từ kết quả tìm kiếm. - Người dùng đã đăng nhập.
Luồng sự kiện chính	- Hệ thống hiển thị sơ đồ ghế tương ứng với loại xe. - Người dùng nhấn chọn vào các vị trí ghế mong muốn trên sơ đồ. - Người dùng nhập các thông tin bắt buộc: Họ tên, Số điện thoại, Email. - Người dùng chọn phương thức thanh toán. - Người dùng nhấn nút “Đặt vé”.

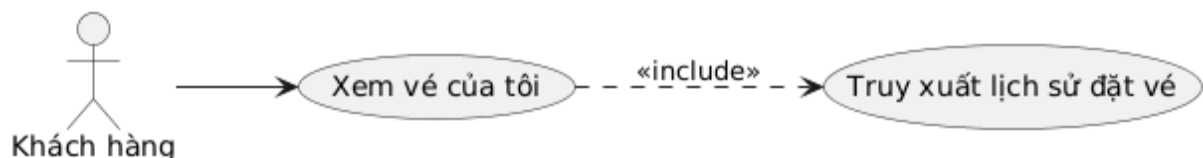
	- Hệ thống thực hiện kiểm tra trạng thái của các ghế đã chọn. - Hệ thống thông báo kết quả cho người dùng.
Luồng ngoại lệ	- Trường hợp ghế đã bị đặt: Nếu tại thời điểm nhấn nút đặt vé mà ghế đã có người khác hoàn tất giao dịch trước, hệ thống hiển thị thông báo "Ghế bạn chọn hiện đã hết, vui lòng chọn vị trí khác". - Trường hợp thiếu thông tin: Nếu người dùng bỏ trống các trường bắt buộc, hệ thống hiển thị cảnh báo yêu cầu nhập đầy đủ. - Trường hợp sai định dạng: Hệ thống báo lỗi nếu số điện thoại hoặc email không đúng định dạng quy định.
Kết quả	Thông tin đặt vé được ghi nhận trên hệ thống.

2.2.1.5. Vé của tôi

– Mô tả chức năng

Đây là chức năng dành cho khách hàng đã có tài khoản và đã đăng nhập thành công. Mục đích của chức năng này là giúp người dùng quản lý lại toàn bộ các vé xe mà mình đã thực hiện trên hệ thống Booking Hub. Tại đây, khách hàng có thể kiểm tra lại các thông tin quan trọng như mã vé, thời gian xe chạy, trạng thái vé (đã thanh toán, đã hủy hoặc đã hoàn thành chuyến đi). Điều này giúp người dùng dễ dàng theo dõi hành trình của mình và cung cấp mã vé cho nhà xe khi lên xe.

– Biểu đồ Use Case



Hình vẽ 2.5. Use Case Vé của tôi

Bảng 2.5. Bảng mô tả Use Case Vé của tôi

Tên Use Case	Xem vé của tôi (Lịch sử đặt vé)
Tác nhân	Khách hàng
Mô tả	Khách hàng truy cập vào Vé của tôi để xem lịch sử đặt vé.
Điều kiện	Khách hàng đã đăng nhập tài khoản vào hệ thống.
Luồng sự kiện chính	- Người dùng nhấn vào mục “Vé của tôi”. - Hệ thống lấy dữ liệu danh sách vé của người dùng. - Hệ thống hiển thị danh sách các vé theo thứ tự từ mới nhất đến cũ nhất. - Người dùng có thể nhấn vào từng vé cụ thể để xem chi tiết mã vé, thông tin ghế và nhà xe.
Luồng ngoại lệ	- Trường hợp chưa có vé nào: Nếu khách hàng mới và chưa từng đặt vé, hệ thống hiển thị danh sách trống.

Kết quả	Danh sách các vé đã đặt được hiển thị đầy đủ trên màn hình cho khách hàng theo dõi.
----------------	---

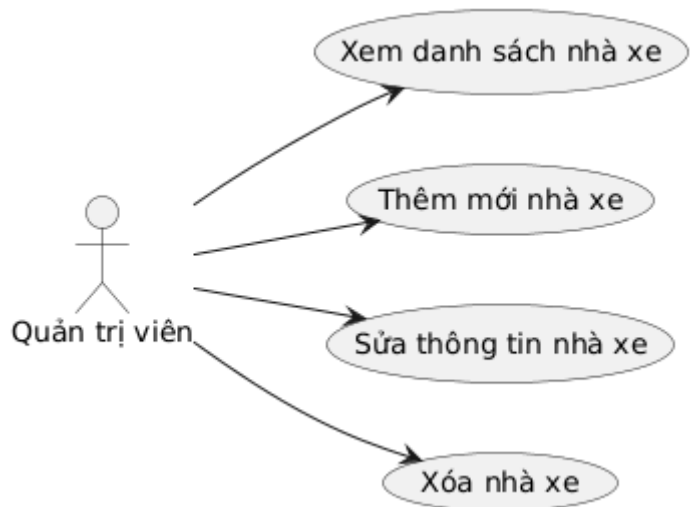
2.2.2. Nhóm chức năng phía quản trị viên (Admin)

2.2.2.1. Quản lý nhà xe

– **Mô tả chức năng**

Đây là chức năng dành cho Quản trị viên để theo dõi danh sách các nhà xe trên hệ thống Booking Hub. Quản trị viên có thể thêm các nhà xe, chỉnh sửa các thông tin nhà xe thay đổi hoặc xóa bỏ các nhà xe không còn hoạt động.

– **Biểu đồ Use Case**



Hình vẽ 2.6. Use Case Quản lý nhà xe

Bảng 2.6. Bảng mô tả Use Case Quản lý nhà xe

Tên Use Case	Quản lý nhà xe
Tác nhân	Quản trị viên
Mô tả	Quản trị viên thực hiện xem danh sách các nhà xe và thực hiện các thao tác cập nhật dữ liệu khi cần thiết.
Điều kiện	Quản trị viên đã đăng nhập thành công vào hệ thống quản trị.
Luồng sự kiện chính	- Quản trị viên chọn mục “Quản lý nhà xe”. - Hệ thống hiển thị dữ liệu danh sách nhà xe. - Để Thêm mới: Quản trị viên nhấn nút “Thêm”, điền thông tin và nhấn “Lưu”. - Để Chỉnh sửa: Quản trị viên chọn một nhà xe trong danh sách, thay đổi thông tin và nhấn “Cập nhật”. - Để Xóa: Quản trị viên chọn nút “Xóa” tại dòng tương ứng của nhà xe và xác nhận yêu cầu. - Hệ thống thực hiện cập nhật và thông báo thao tác thành công.

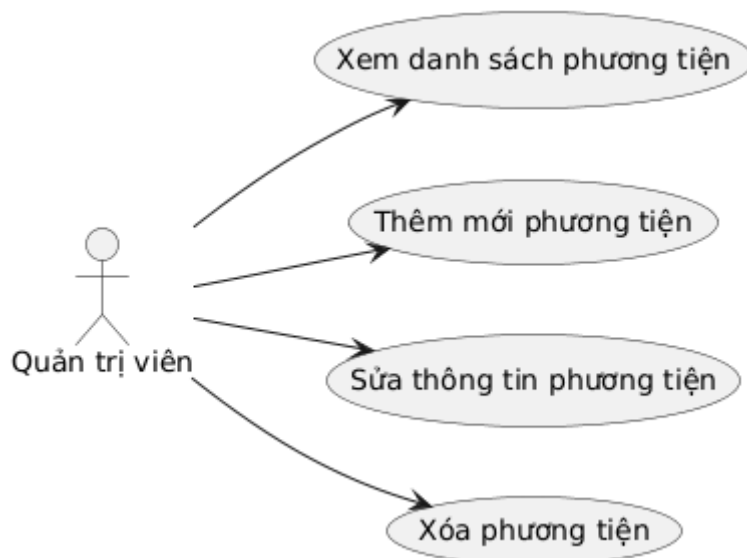
Luồng ngoại lệ	- Trường hợp dữ liệu không hợp lệ: Hệ thống báo lỗi nếu Quản trị viên để trống các ô bắt buộc khi Thêm/Sửa. - Trường hợp xác nhận xóa: Nếu Quản trị viên nhấn Xóa, hệ thống hiển thị thông báo "Bạn có chắc chắn muốn xóa không?" để tránh nhầm lẫn.
Kết quả	Danh sách nhà xe được hiển thị và cập nhật đúng thông tin.

2.2.2.2. Quản lý phương tiện

– Mô tả chức năng

Đây là chức năng cho phép Quản trị viên quản lý toàn bộ các xe. Chức năng này hỗ trợ các thao tác từ việc thêm xe mới, điều chỉnh thông tin của xe, cho đến việc xóa bỏ các xe đã cũ hoặc ngừng hoạt động. Đây là dữ liệu nền tảng để hệ thống có thể sắp xếp các chuyến xe và hiển thị sơ đồ ghé chính xác cho khách hàng.

– Biểu đồ Use Case



Hình vẽ 2.7. Use Case Quản lý phương tiện

Bảng 2.7. Bảng mô tả Use Case Quản lý phương tiện

Tên Use Case	Quản lý phương tiện
Tác nhân	Quản trị viên
Mô tả	Quản trị viên thực hiện xem danh sách và cập nhật thông tin phương tiện.
Điều kiện	Quản trị viên đã đăng nhập vào hệ thống và truy cập vào trang quản lý phương tiện.
Luồng sự kiện chính	- Quản trị viên vào trang “Quản lý phương tiện”. - Hệ thống hiển thị danh sách các phương tiện. - Để Thêm mới: Quản trị viên nhấn “Thêm” nhập các thông tin phương tiện sau đó nhấn "Lưu".

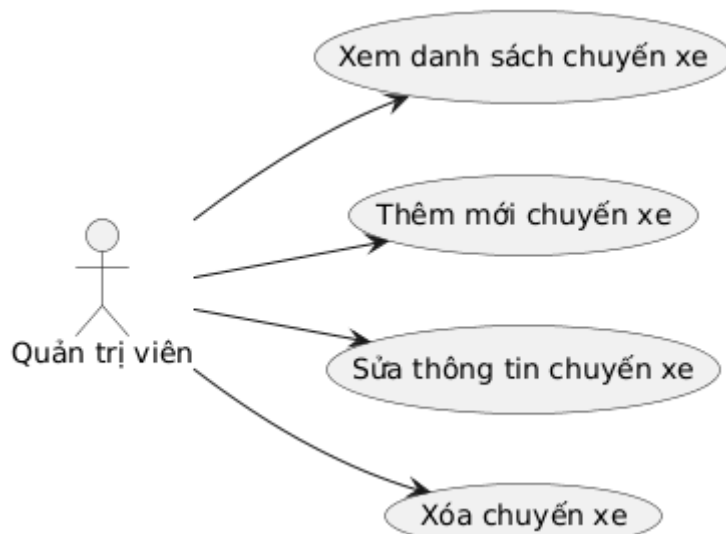
	- Đề Chỉnh sửa: Quản trị viên tìm xe cần sửa trong danh sách, thay đổi các thông tin và nhấn “Cập nhật”. - Đề Xóa: Quản trị viên nhấn vào nút “Xóa” tại dòng tương ứng của phương tiện và xác nhận yêu cầu xóa. - Hệ thống thực hiện kiểm tra các ràng buộc dữ liệu và cập nhật.
Luồng ngoại lệ	- Trường hợp nhập thiếu thông tin: Hệ thống hiển thị cảnh báo.
Kết quả	Danh sách phương tiện được hiển thị đầy đủ và cập nhật đúng thông tin.

2.2.2.3. Quản lý chuyến xe

– Mô tả chức năng

Đây là chức năng cho phép Quản trị viên thiết lập và điều phối các chuyến xe hoạt động hàng ngày. Một chuyến xe được tạo ra bằng cách kết hợp thông tin giữa một nhà xe cụ thể, một tuyến đường nhất định (điểm đi - điểm đến), một phương tiện và thời gian khởi hành cụ thể. Quản trị viên dựa vào chức năng này để vận hành hệ thống, giúp khách hàng có thể tìm thấy các chuyến xe phù hợp khi tra cứu. Chức năng hỗ trợ đầy đủ việc xem danh sách các chuyến xe đã thiết lập, thêm chuyến mới, điều chỉnh giờ chạy hoặc xóa các chuyến không còn hoạt động.

– Biểu đồ Use Case



Hình vẽ 2.8. Use Case Quản lý chuyến xe

Bảng 2.8. Bảng mô tả Use Case Quản lý chuyến xe

Tên Use Case	Quản lý chuyến xe
Tác nhân	Quản trị viên
Mô tả	Quản trị viên thực hiện thiết lập lịch trình các chuyến xe.

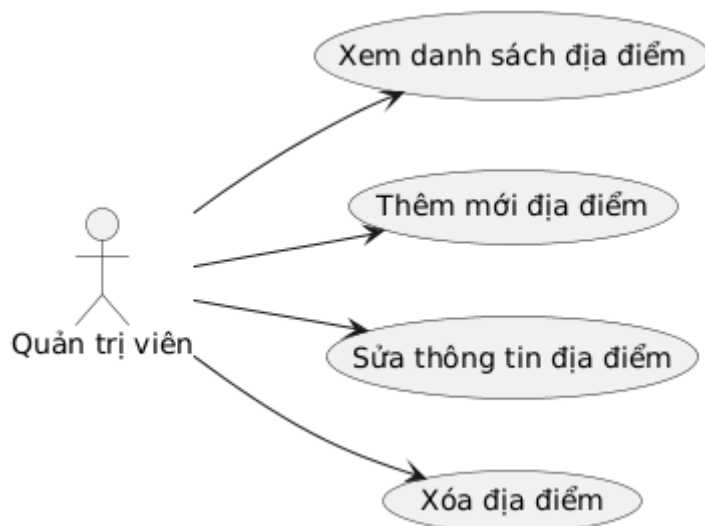
Điều kiện	Quản trị viên đã đăng nhập và hệ thống đã có sẵn dữ liệu về nhà xe, địa điểm và phương tiện.
Luồng sự kiện chính	- Quản trị viên vào trang “Quản lý chuyến xe”. - Hệ thống hiển thị danh sách các chueyens xe. - Đề Thêm mới: Quản trị viên nhấn nút “Thêm” nhập thông tin nhấn “Lưu”. - Đề Chỉnh sửa: Quản trị viên chọn chuyến xe cần thay đổi, cập nhật lại thông tin và nhấn “Cập nhật”. - Đề Xóa: Quản trị viên tìm chuyến xe cần xóa, nhấn nút “Xóa” và xác nhận yêu cầu. - Hệ thống kiểm tra tính hợp lệ và cập nhật dữ liệu vào hệ thống.
Luồng ngoại lệ	- Trường hợp trùng lịch: Hệ thống báo lỗi nếu một xe bị xếp vào hai chuyến chạy cùng một thời điểm. - Trường hợp xóa chuyến có khách: Nếu chuyến xe đã có khách hàng đặt vé thành công, hệ thống sẽ ngăn chặn việc xóa để bảo vệ quyền lợi khách hàng. - Trường hợp nhập thiếu: Hệ thống báo lỗi nếu Quản trị viên chưa chọn đủ các thông tin bắt buộc như giá vé hoặc thời gian chạy.
Kết quả	Hiển thị danh sách chuyến xe và cập nhật thông tin thành công.

2.2.2.4. Quản lý địa điểm

– Mô tả chức năng

Đây là chức năng quản lý danh mục địa danh trên hệ thống Booking Hub. Quản trị viên sử dụng chức năng này để thiết lập danh sách các tỉnh thành, quận huyện. Việc quản lý địa điểm một cách chuẩn hóa giúp khách hàng dễ dàng tìm kiếm hành trình trên thanh công cụ và giúp hệ thống gợi ý chính xác các điểm xuất phát/đích đến. Quản trị viên có quyền xem danh sách, thêm địa điểm mới, sửa tên địa điểm hoặc xóa các địa điểm không còn sử dụng.

– Biểu đồ Use Case



Hình vẽ 2.9. Use Case Quản lý địa điểm

Bảng 2.9. Bảng mô tả Use Case Quản lý địa điểm

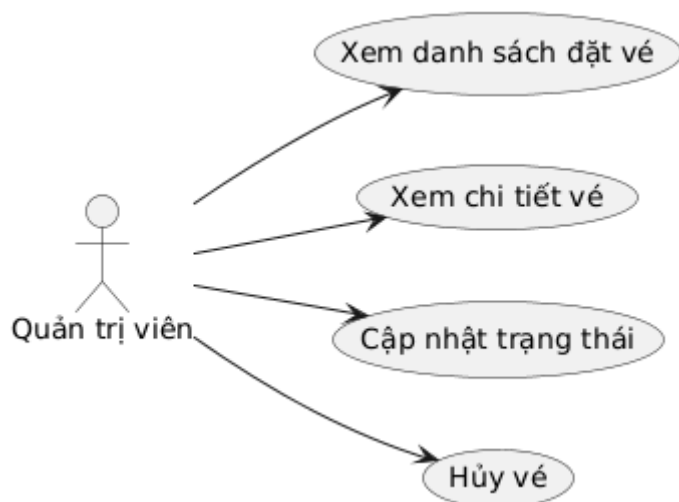
Tên Use Case	Quản lý địa điểm
Tác nhân	Quản trị viên
Mô tả	Quản trị viên thực hiện xem danh sách và cập nhật thông tin địa điểm.
Điều kiện	Quản trị viên đã đăng nhập thành công.
Luồng sự kiện chính	- Quản trị viên vào trang “Quản lý địa điểm”. - Hệ thống hiển thị danh sách các địa điểm. - Để Thêm mới: Quản trị viên nhấn “Thêm” nhập thông tin và nhấn “Lưu”. - Để Chỉnh sửa: Quản trị viên chọn địa điểm cần sửa, thay thông tin và nhấn “Cập nhật”. - Để Xóa: Quản trị viên chọn địa điểm muốn xóa, nhấn “Xóa” và xác nhận lại yêu cầu. - Hệ thống kiểm tra dữ liệu và cập nhật thông tin.
Luồng ngoại lệ	- Trường hợp để trống: Hệ thống hiển thị cảnh báo nếu Quản trị viên nhấn lưu mà chưa nhập các thông tin bắt buộc.
Kết quả	Hiển thị danh sách địa điểm và cập nhật thông tin thành công.

2.2.2.5. Quản lý đặt vé

– Mô tả chức năng

Đây là chức năng cho phép Quản trị viên theo dõi và xử lý toàn bộ vé đã được đặt. Quản trị viên có thể xem danh sách các vé đã được đặt, xem chi tiết từng vé để biết thông tin khách hàng và số ghế đã chọn. Ngoài ra, Quản trị viên còn có quyền cập nhật trạng thái đặt vé (ví dụ: xác nhận đã thanh toán) hoặc thực hiện hủy vé trong các trường hợp khách hàng yêu cầu hoặc có sự cố xảy ra.

– Biểu đồ Use Case



Hình vẽ 2.10. Use Case Quản lý đặt vé

Bảng 2.10. Bảng mô tả Use Case Quản lý đặt vé

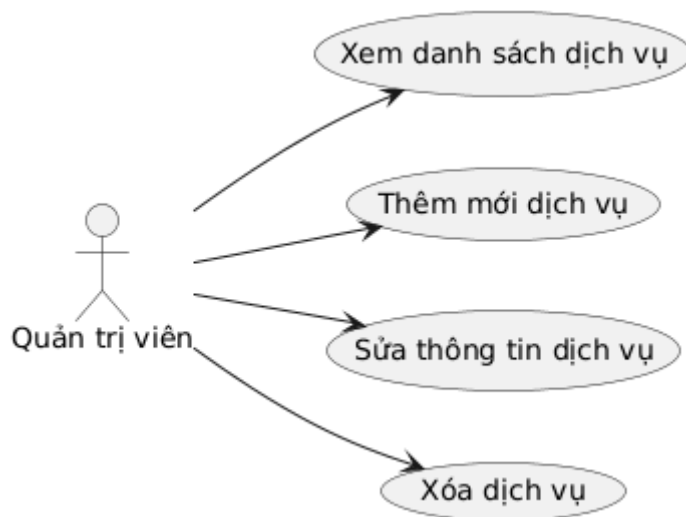
Tên Use Case	Quản lý đặt
Tác nhân	Quản trị viên
Mô tả	Quản trị viên xem thông tin vé và cập nhật trạng thái
Điều kiện	Quản trị viên đã đăng nhập thành công và hệ thống đã có dữ liệu đặt vé từ khách hàng.
Luồng sự kiện chính	- Quản trị viên truy cập vào trang “Quản lý đặt vé”. - Hệ thống hiển thị danh sách các vé đã đặt. - Để Xem chi tiết: Quản trị viên nhấn vào nút “Chi tiết” để xem thông tin chi tiết của vé. - Để Cập nhật trạng thái: Quản trị viên ấn vào nút “Xác nhận thanh toán” và xác nhận. - Để Hủy vé: Quản trị viên ấn vào nút “Hủy” và xác nhận. - Hệ thống cập nhật thông tin mới.
Luồng ngoại lệ	- Trường hợp vé trạng thái đã hủy: Không hiển thị nút Xác nhận thanh toán và Hủy vé.
Kết quả	Hiển thị danh sách vé và cập nhật thông tin thành công.

2.2.2.6. Quản lý dịch vụ

– Mô tả chức năng

Đây là chức năng giúp Quản trị viên quản lý các dịch vụ bổ sung trên hệ thống như suất ăn, bảo hiểm,... Quản trị viên có thể theo dõi danh sách, thêm dịch vụ mới, thay đổi thông tin hoặc xóa các dịch vụ không còn cung cấp để khách hàng có thể lựa chọn khi đặt vé.

– Biểu đồ Use Case



Hình vẽ 2.11. Use Case Quản lý dịch vụ

Bảng 2.11. Bảng mô tả Use Case Quản lý dịch vụ

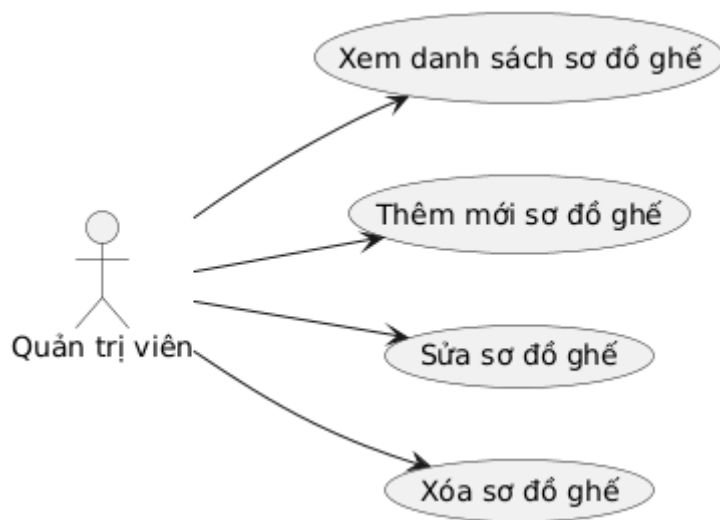
Tên Use Case	Quản lý dịch vụ
Tác nhân	Quản trị viên
Mô tả	Quản trị viên thực hiện xem danh sách và cập nhật thông tin các dịch vụ bổ sung.
Điều kiện	Quản trị viên đã đăng nhập thành công.
Luồng sự kiện chính	- Quản trị viên vào trang “Quản lý dịch vụ”. - Hệ thống hiển thị danh sách các dịch vụ. - Để Thêm mới: Quản trị viên nhấn “Thêm” nhập thông tin và nhấn “Lưu”. - Để Chỉnh sửa: Quản trị viên chọn dịch vụ cần sửa, thay thông tin và nhấn “Cập nhật”. - Để Xóa: Quản trị viên chọn dịch vụ muốn xóa, nhấn “Xóa” và xác nhận lại yêu cầu. - Hệ thống kiểm tra dữ liệu và cập nhật thông tin.
Luồng ngoại lệ	- Trường hợp để trống: Hệ thống hiển thị cảnh báo nếu Quản trị viên nhấn lưu mà chưa nhập các thông tin bắt buộc.
Kết quả	Hiển thị danh sách dịch vụ và cập nhật thông tin thành công.

2.2.2.7. Quản lý sơ đồ ghế

– Mô tả chức năng

Đây là chức năng cho phép Quản trị viên thiết kế và quản lý cấu trúc chỗ ngồi cho từng loại xe khách khác nhau (như xe ghế ngồi, xe giường nằm 1 tầng hoặc 2 tầng). Quản trị viên có thể tạo mới sơ đồ theo số lượng ghế, chỉnh sửa vị trí các ghế hoặc xóa các sơ đồ không còn sử dụng để đảm bảo khi khách hàng đặt vé sẽ hiển thị đúng giao diện thực tế của xe.

– Biểu đồ Use Case



Hình vẽ 2.12. Use Case Quản lý sơ đồ ghế

Bảng 2.12. Bảng mô tả Use Case Quản lý sơ đồ ghế

Tên Use Case	Quản lý sơ đồ ghế
Tác nhân	Quản trị viên
Mô tả	Quản trị viên thực hiện xem danh sách và cập nhật các mẫu sơ đồ ghế ngồi cho phương tiện.
Điều kiện	Quản trị viên đã đăng nhập thành công.
Luồng sự kiện chính	- Quản trị viên vào trang “Quản lý sơ đồ ghế”. - Hệ thống hiển thị danh sách các mẫu sơ đồ ghế đã tạo. - Để Thêm mới: Quản trị viên nhấn “Thêm” nhập thông tin và nhấn “Lưu”. - Để Chỉnh sửa: Quản trị viên chọn sơ đồ cần sửa, thay đổi cấu trúc hoặc số ghế và nhấn “Cập nhật”. - Để Xóa: Quản trị viên chọn sơ đồ muốn xóa, nhấn “Xóa” và xác nhận lại yêu cầu. - Hệ thống kiểm tra dữ liệu và cập nhật thông tin.
Luồng ngoại lệ	- Trường hợp để trống: Hệ thống hiển thị cảnh báo nếu Quản trị viên nhấn lưu mà chưa nhập các thông tin bắt buộc.
Kết quả	Hiển thị danh sách sơ đồ ghế và cập nhật thông tin thành công.

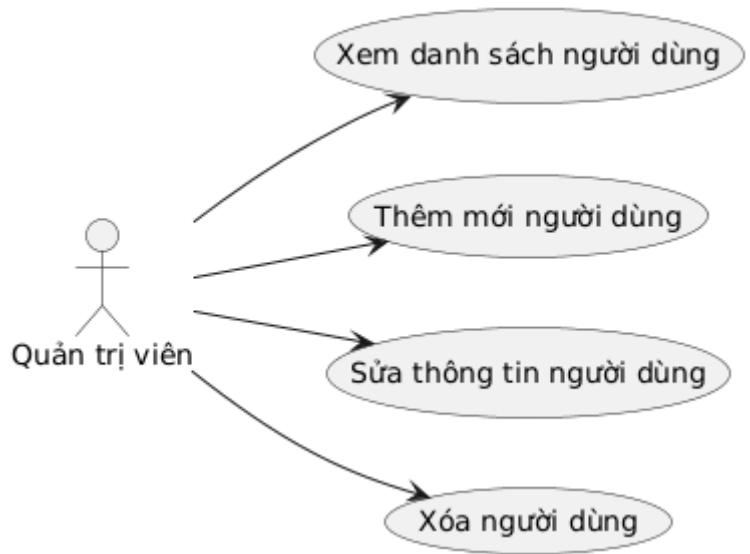
2.2.2.8. Quản lý người dùng

– Mô tả chức năng

Đây là chức năng giúp Quản trị viên theo dõi và quản lý toàn bộ danh sách tài khoản trên hệ thống. Quản trị viên có thể kiểm tra thông tin của tài khoản, thực hiện

thêm mới tài khoản, chỉnh sửa thông tin hoặc xóa các tài khoản của hệ thống Booking Hub.

– **Biểu đồ Use Case**



Hình vẽ 2.13. Use Case Quản lý người dùng

Bảng 2.13. Bảng mô tả Use Case Quản lý người dùng

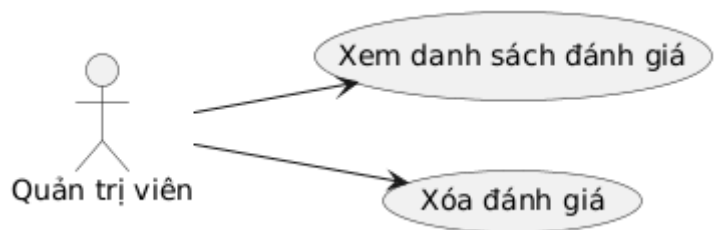
Tên Use Case	Quản lý người dùng
Tác nhân	Quản trị viên
Mô tả	Quản trị viên thực hiện xem danh sách và cập nhật thông tin các tài khoản trên hệ thống.
Điều kiện	Quản trị viên đã đăng nhập thành công.
Luồng sự kiện chính	- Quản trị viên vào trang “Quản lý người dùng”. - Hệ thống hiển thị danh sách các tài khoản. - Để Thêm mới: Quản trị viên nhấn “Thêm” nhập thông tin và nhấn “Lưu”. - Để Chỉnh sửa: Quản trị viên chọn tài khoản cần sửa, thay đổi thông tin và nhấn “Cập nhật”. - Để Xóa: Quản trị viên chọn tài khoản muốn xóa, nhấn “Xóa” và xác nhận lại yêu cầu. - Hệ thống kiểm tra dữ liệu và cập nhật thông tin.
Luồng ngoại lệ	- Trường hợp để trống: Hệ thống hiển thị cảnh báo nếu Quản trị viên nhấn lưu mà chưa nhập các thông tin bắt buộc.
Kết quả	Hiển thị danh sách người dùng và cập nhật thông tin thành công.

2.2.2.9. Quản lý đánh giá

– **Mô tả chức năng**

Đây là chức năng giúp Quản trị viên theo dõi các phản hồi và mức độ hài lòng của khách hàng sau khi sử dụng dịch vụ chuyển xe. Quản trị viên có thể xem toàn bộ nội dung đánh giá và số sao mà người dùng đã gửi. Trong trường hợp phát hiện các đánh giá có nội dung không phù hợp, vi phạm quy tắc cộng đồng hoặc có tính chất spam, Quản trị viên có quyền xóa bỏ để đảm bảo thông tin hiển thị trên hệ thống luôn khách quan.

– **Biểu đồ Use Case**



Hình vẽ 2.14. Use Case Quản lý đánh giá

Bảng 2.14. Bảng mô tả Use Case Quản lý đánh giá

Tên Use Case	Quản lý đánh giá
Tác nhân	Quản trị viên
Mô tả	Quản trị viên thực hiện xem các phản hồi của khách hàng và loại bỏ những đánh giá không phù hợp.
Điều kiện	Quản trị viên đã đăng nhập thành công.
Luồng sự kiện chính	- Quản trị viên vào trang “Quản lý đánh giá”. - Hệ thống hiển thị danh sách các đánh giá kèm thông tin khách hàng, nội dung và số sao. - Quản trị viên theo dõi và kiểm tra nội dung các phản hồi. - Để Xóa: Quản trị viên chọn đánh giá không phù hợp, nhấn “Xóa” và xác nhận lại yêu cầu. - Hệ thống thực hiện xóa dữ liệu khỏi cơ sở dữ liệu và thông báo thành công.
Luồng ngoại lệ	- Trường hợp xác nhận: Hệ thống yêu cầu xác nhận trước khi xóa để tránh thao tác nhầm.
Kết quả	Hiển thị danh sách đánh giá và xóa nội dung không phù hợp thành công.

2.3. Kịch bản kiểm thử

2.3.1. Kịch bản kiểm thử phía khách hàng

2.3.1.1. Kịch bản Trang chủ

- **Mô tả:** Người dùng mở trình duyệt và vào trang chủ
- **Các bước thực hiện:**
 1. Mở trình duyệt và mở trang Booking Hub của khách hàng

2. Kiểm tra trang tải thành công
 3. Kiểm tra hiển thị thông tin tìm chuyến và nút Tìm chuyến
 4. Nhập thông tin hoặc để trống thông tin tìm chuyến
 5. Nhấn nút Tìm chuyến
- **Kết quả mong muốn:**
 - Trang chủ hiển thị đầy đủ thông tin giao diện
 - Hiển thị thông báo khi thiếu dữ liệu
 - Chuyển sang trang tìm chuyến nếu dữ liệu hợp lệ

2.3.1.2. Kịch bản Đăng nhập

- **Mô tả:** Người dùng mở trình duyệt và vào trang đăng nhập
- **Các bước thực hiện:**
 1. Mở trình duyệt và mở trang đăng nhập
 2. Kiểm tra hiển thị thông tin các trường dữ liệu
 3. Nhập thông tin dữ liệu
 4. Thử thực hiện đăng nhập
- **Kết quả mong muốn:**
 - Đăng nhập sai hiển thị thông báo và vẫn ở lại trang đăng nhập
 - Đăng nhập đúng rời khỏi trang đăng nhập và điều hướng tới trang tương ứng

2.3.1.3. Kịch bản Đăng ký

- **Mô tả:** Người dùng mở trình duyệt và vào trang đăng ký
- **Các bước thực hiện:**
 1. Mở trang đăng ký
 2. Kiểm tra hiển thị thông tin các trường dữ liệu
 3. Nhập thông tin dữ liệu
 4. Thử thực hiện đăng ký
- **Kết quả mong muốn:**
 - Thông tin không hợp lệ thông báo lỗi và vẫn ở trang đăng ký
 - Thông tin hợp lệ đăng ký thành công và điều hướng về trang đăng nhập

2.3.1.4. Kịch bản Tìm chuyến

- **Mô tả:** Người dùng mở trang tìm chuyến và tìm kiếm chuyến xe
- **Các bước thực hiện:**
 1. Mở trang tìm chuyến
 2. Kiểm tra hiển thị thông tin các trường dữ liệu
 3. Nhập thông tin dữ liệu
 4. Thực hiện tìm chuyến
- **Kết quả mong muốn:**
 - Thiếu thông tin tìm chuyến hiển thị thông báo lỗi
 - Thông tin hợp lệ hiển thị các chuyến xe

2.3.1.5. Kịch bản Đặt vé

- **Mô tả:** Người dùng chọn chuyến xe và đặt vé
- **Các bước thực hiện:**

1. Tìm kiếm chuyến xe và chọn chuyến xe
 2. Kiểm tra hiển thị thông tin các trường dữ liệu
 3. Nhập các thông tin đặt vé
 4. Thực hiện đặt vé
- **Kết quả mong muốn:**
 - Thiếu thông tin đặt vé hiển thị thông báo lỗi
 - Ghế đã được đặt hiển thị thông báo lỗi
 - Đủ thông tin đặt vé thành công
 - Chưa đăng nhập sẽ chuyển về trang đăng nhập

2.3.1.6. Kịch bản Thanh toán

- **Mô tả:** Người dùng thanh toán vé
- **Các bước thực hiện:**
 1. Mở trang vé của tôi
 2. Kiểm tra hiển thị thông tin danh sách vé
 3. Thanh toán vé
- **Kết quả mong muốn:**
 - Hiển thị nút Thanh toán với vé chưa thanh toán
 - Chưa đăng nhập sẽ chuyển về trang đăng nhập
 - Thanh toán thành công hiển thị trạng thái vé đã thanh toán

2.3.1.7. Kịch bản Vé của tôi

- **Mô tả:** Người dùng vào trang vé của tôi
- **Các bước thực hiện:**
 1. Mở trang vé của tôi
 2. Kiểm tra giao diện
 3. Kiểm tra hiển thị danh sách vé
- **Kết quả mong muốn:**
 - Chưa đăng nhập sẽ chuyển về trang đăng nhập
 - Có vé sẽ hiển thị danh sách vé
 - Không có vé sẽ hiển thị rỗng

2.3.1.8. Kịch bản Hồ sơ

- **Mô tả:** Người dùng vào trang hồ sơ
- **Các bước thực hiện:**
 1. Đăng nhập
 2. Mở trang hồ sơ
 3. Kiểm tra hiển thị thông tin người dùng
- **Kết quả mong muốn:**
 - Hiển thị đúng thông tin người dùng

2.3.2. Kịch bản kiểm thử phía quản trị viên

2.3.2.1. Kịch bản Đăng nhập

- **Mô tả:** Quản trị viên mở trình duyệt và vào trang đăng nhập
- **Các bước thực hiện:**

1. Mở trình duyệt và mở trang đăng nhập
 2. Kiểm tra hiển thị thông tin các trường dữ liệu
 3. Nhập thông tin dữ liệu
 4. Thử thực hiện đăng nhập
- **Kết quả mong muốn:**
 - Đăng nhập sai hiển thị thông báo và vẫn ở lại trang đăng nhập
 - Đăng nhập đúng rời khỏi trang đăng nhập và điều hướng tới trang tương ứng

2.3.2.2. Kịch bản Dashboard

- **Mô tả:** Quản trị viên mở trình duyệt và vào trang dashboard
- **Các bước thực hiện:**
 1. Mở trình duyệt
 2. Vào trang dashboard
 3. Kiểm tra thông tin hiển thị
- **Kết quả mong muốn:**
 - Hiển thị đúng thông tin trang dashboard
 - Nếu chưa đăng nhập hoặc hết phiên đăng nhập về lại trang đăng nhập

2.3.2.3. Kịch bản quản lý địa điểm

- **Mô tả:** Quản trị viên mở trình duyệt và vào trang quản lý địa điểm
- **Các bước thực hiện:**
 1. Mở trình duyệt
 2. Vào trang quản lý địa điểm
 3. Kiểm tra thông tin hiển thị địa điểm
 4. Thử thêm, sửa, xóa, tìm kiếm địa điểm
- **Kết quả mong muốn:**
 - Hiển thị đúng thông tin danh sách địa điểm
 - Tìm kiếm được địa điểm
 - Thêm, sửa, xóa được địa điểm
 - Nếu nhập thiếu thông tin dữ liệu thông báo lỗi
 - Nếu chưa đăng nhập hoặc hết phiên đăng nhập về lại trang đăng nhập

2.3.2.4. Kịch bản quản lý nhà xe

- **Mô tả:** Quản trị viên mở trình duyệt và vào trang quản lý nhà xe
- **Các bước thực hiện:**
 1. Mở trình duyệt
 2. Vào trang quản lý nhà xe
 3. Kiểm tra thông tin hiển thị nhà xe
 4. Thử thêm, sửa, xóa, tìm kiếm nhà xe
- **Kết quả mong muốn:**
 - Hiển thị đúng thông tin danh sách nhà xe
 - Tìm kiếm được nhà xe
 - Thêm, sửa, xóa được nhà xe
 - Nếu nhập thiếu thông tin dữ liệu thông báo lỗi
 - Nếu chưa đăng nhập hoặc hết phiên đăng nhập về lại trang đăng nhập

2.3.2.5. Kịch bản Quản lý phương tiện

- **Mô tả:** Quản trị viên mở trình duyệt và vào trang Quản lý phương tiện
- **Các bước thực hiện:**
 1. Mở trình duyệt
 2. Vào trang Quản lý phương tiện
 3. Kiểm tra thông tin hiển thị xe
 4. Thử thêm, sửa, xóa, tìm kiếm xe
- **Kết quả mong muốn:**
 - Hiển thị đúng thông tin danh sách xe
 - Tìm kiếm được xe
 - Thêm, sửa, xóa được xe
 - Nếu nhập thiếu thông tin dữ liệu thông báo lỗi
 - Nếu chưa đăng nhập hoặc hết phiên đăng nhập về lại trang đăng nhập

2.3.2.6. Kịch bản quản lý chuyển xe

- **Mô tả:** Quản trị viên mở trình duyệt và vào trang quản lý chuyển xe
- **Các bước thực hiện:**
 1. Mở trình duyệt
 2. Vào trang quản lý chuyển xe
 3. Kiểm tra thông tin hiển thị chuyển xe
 4. Thử thêm, sửa, xóa, tìm kiếm chuyển xe
- **Kết quả mong muốn:**
 - Hiển thị đúng thông tin danh sách chuyển xe
 - Tìm kiếm được chuyển xe
 - Thêm, sửa, xóa được chuyển xe
 - Nếu nhập thiếu thông tin dữ liệu thông báo lỗi
 - Nếu chưa đăng nhập hoặc hết phiên đăng nhập về lại trang đăng nhập

2.3.2.7. Kịch bản quản lý người dùng

- **Mô tả:** Quản trị viên mở trình duyệt và vào trang quản lý người dùng
- **Các bước thực hiện:**
 1. Mở trình duyệt
 2. Vào trang quản lý người dùng
 3. Kiểm tra thông tin hiển thị người dùng
 4. Thử thêm, sửa, xóa, tìm kiếm người dùng
- **Kết quả mong muốn:**
 - Hiển thị đúng thông tin danh sách người dùng
 - Tìm kiếm được người dùng
 - Thêm, sửa, xóa được người dùng
 - Nếu nhập thiếu thông tin dữ liệu thông báo lỗi
 - Nếu chưa đăng nhập hoặc hết phiên đăng nhập về lại trang đăng nhập

2.3.2.8. Kịch bản quản lý dịch vụ

- **Mô tả:** Quản trị viên mở trình duyệt và vào trang quản lý dịch vụ
- **Các bước thực hiện:**

1. Mở trình duyệt
 2. Vào trang quản lý dịch vụ
 3. Kiểm tra thông tin hiển thị dịch vụ
 4. Thử thêm, sửa, xóa, tìm kiếm dịch vụ
- **Kết quả mong muốn:**
 - Hiển thị đúng thông tin danh sách dịch vụ
 - Tìm kiếm được dịch vụ
 - Thêm, sửa, xóa được dịch vụ
 - Nếu nhập thiếu thông tin dữ liệu thông báo lỗi
 - Nếu chưa đăng nhập hoặc hết phiên đăng nhập về lại trang đăng nhập

2.3.2.9. Kịch bản quản lý đánh giá

- **Mô tả:** Quản trị viên mở trình duyệt và vào trang quản lý đánh giá
- **Các bước thực hiện:**
 1. Mở trình duyệt
 2. Vào trang quản lý đánh giá
 3. Kiểm tra thông tin hiển thị đánh giá
- **Kết quả mong muốn:**
 - Hiển thị đúng thông tin danh sách đánh giá
 - Tìm kiếm được đánh giá
 - Nếu chưa đăng nhập hoặc hết phiên đăng nhập về lại trang đăng nhập

2.3.2.10. Kịch bản quản lý thanh toán

- **Mô tả:** Quản trị viên mở trình duyệt và vào trang quản lý thanh toán
- **Các bước thực hiện:**
 1. Mở trình duyệt
 2. Vào trang quản lý thanh toán
 3. Kiểm tra thông tin hiển thị thanh toán
- **Kết quả mong muốn:**
 - Hiển thị đúng thông tin danh sách thanh toán
 - Tìm kiếm được thanh toán
 - Nếu chưa đăng nhập hoặc hết phiên đăng nhập về lại trang đăng nhập

2.3.2.11. Kịch bản quản lý thông báo

- **Mô tả:** Quản trị viên mở trình duyệt và vào trang quản lý thông báo
- **Các bước thực hiện:**
 1. Mở trình duyệt
 2. Vào trang quản lý thông báo
 3. Kiểm tra thông tin hiển thị thông báo
- **Kết quả mong muốn:**
 - Hiển thị đúng thông tin danh sách thông báo
 - Tìm kiếm được thông báo
 - Nếu chưa đăng nhập hoặc hết phiên đăng nhập về lại trang đăng nhập

2.3.2.12. Kịch bản quản lý đặt vé

- **Mô tả:** Quản trị viên mở trình duyệt và vào trang quản lý đặt vé

- **Các bước thực hiện:**
 1. Mở trình duyệt
 2. Vào trang quản lý đặt vé
 3. Kiểm tra thông tin hiển thị đặt vé
 4. Xác nhận hoặc hủy vé
- **Kết quả mong muốn:**
 - Hiển thị đúng thông tin danh sách vé
 - Tìm kiếm được vé
 - Có thể xác nhận và hủy vé
 - Nếu chưa đăng nhập hoặc hết phiên đăng nhập về lại trang đăng nhập

CHƯƠNG 3 : THỰC HIỆN KIỂM THỬ TỰ ĐỘNG BẰNG SELENIUM

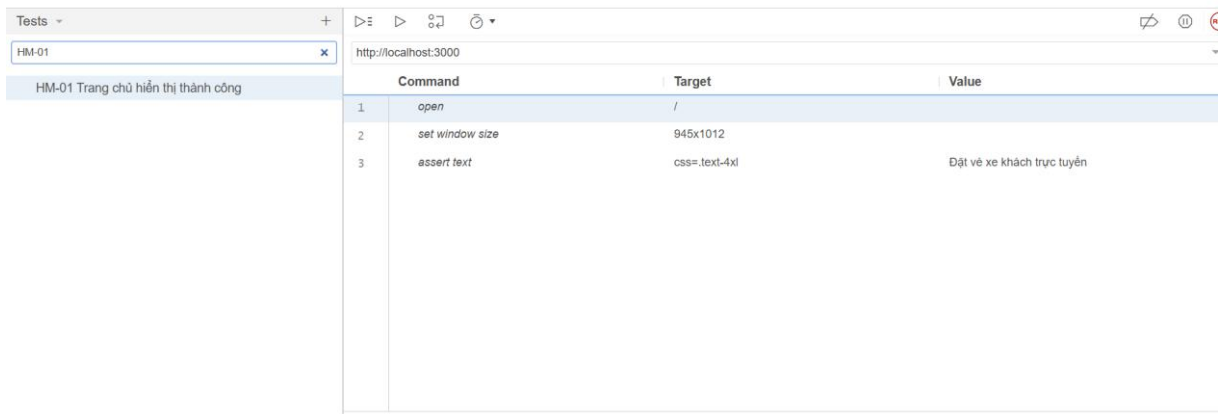
3.1. Kiểm thử sử dụng Selenium IDE

3.1.1. Kiểm thử phía khách hàng

3.1.1.1. Kiểm thử trang Trang chủ

Test case ID: HM-01

Test case: Trang chủ hiển thị thành công



Hình vẽ 3.1. Test case HM-01

Bảng 3.1. Test case HM-01

Các bước thực hiện	Kết quả mong muốn	Kết quả
1. Mở trang chủ 2. Kiểm tra trang chủ	Trang chủ hiển thị được thành công	Đạt

Test case ID: HM-07

Test case: Tìm kiếm chuyến xe trên trang chủ

HM-07

http://localhost:3000

HM-07 Ấn tìm kiếm chuyển sang trang tìm kiếm

Command	Target	Value
1	open	/
2	set window size	945x1012
3	mouse over	css=:disabled()3Aopacity-50
4	mouse out	css=:disabled()3Aopacity-50
5	click	id=:departureLocationId
6	select	id=:departureLocationId label=Hà Nội - Long Biên
7	click	id=:arrivalLocationId
8	select	id=:arrivalLocationId label=Hải Phòng - Lê Chân
9	click	id=:departureDate
10	click	css=:react-datepicker__day-014
11	click	css=:disabled()3Aopacity-50
12	mouse over	css=:disabled()3Aopacity-50
13	assert title	Tìm chuyến xe - BookingHub

Hình vẽ 3.2. Test case HM-07

Bảng 3.2. Test case HM-07

Các bước thực hiện	Kết quả mong muốn	Kết quả
1. Nhập thông tin tìm kiếm 2. Nhấn nút Tìm kiếm 3. Chuyển sang trang tìm kiếm chuyến xe	Chuyển sang trang tìm kiếm chuyến xe và hiển thị danh sách chuyến nếu có	Đạt

Test case ID: HM-10
Test case: Tìm kiếm lỗi khi chưa nhập thông tin

HM-10	http://localhost:3000		
HM-10 Bấm tìm kiếm khi chưa chọn thông tin	Command	Target	Value
	1	open	/
	2	set window size	945x1012
	3	click	css=.:disabled()3Aopacity-50
	4	assert text	css=.:go3958317564 Vui lòng điền đầy đủ thông tin

Hình vẽ 3.3. Test case HM-10

Bảng 3.3. Test case HM-10

Các bước thực hiện	Kết quả mong muốn	Kết quả
1. Mở trang chủ 2. Để trống các ô chọn 3. Nhấn Tìm kiếm 4. Kiểm tra thông báo lỗi xuất hiện	Trang chủ hiển thị thông báo lỗi "Vui lòng nhập đầy đủ thông tin"	Đạt

3.1.1.2. Kiểm thử trang Đăng nhập

Test case ID: LG-01

Test case: Kiểm tra trang đăng nhập hiển thị giao diện form đăng nhập

LG-01 Trang login hiển thị form đăng nhập	Command	Target	Value
1	open	/	
2	set window size	945x1012	
3	execute script	return !document.querySelector(' btn-login')	isLoginNeeded
4	if	!\$(isLoginNeeded)	
5	click	css= py-2:nth-child(1)	
6	click	css= hover:3A bg-red-50	
7	mouse over	css= hover:3A bg-red-50	
8	mouse out	css= hover:3A bg-red-50	
9	click	css= bg-red-600	
10	end		
11	click	linkText=Đăng nhập	
12	assert element present	css= text-3xl	

Hình vẽ 3.4. Test case LG-01

Bảng 3.4. Test case LG-01

Các bước thực hiện	Kết quả mong muốn	Kết quả
1. Nhấn Đăng nhập	Form đăng nhập hiển thị thành công	Đạt
2. Kiểm tra form đăng nhập		

Test case ID: LG-10

Test case: Không nhập thông tin đăng nhập

LG-10 Gửi thông tin form trống vẫn ở t	Command	Target	Value
1	open	/	
2	set window size	945x1012	
3	execute script	return !document.querySelector(' btn-login')	isLoginNeeded
4	if	!\$(isLoginNeeded)	
5	click	css= py-2:nth-child(1)	
6	click	css= hover:3A bg-red-50	
7	mouse over	css= hover:3A bg-red-50	
8	mouse out	css= hover:3A bg-red-50	
9	click	css= bg-red-600	
10	end		
11	click	linkText=Đăng nhập	
12	assert text	css= text-3xl	Đăng nhập
13	assert element present	id=usernameOrEmail	
14	assert element present	id=password	
15	click	css= disabled:3Aopacity-50	
16	assert editable	id=usernameOrEmail	

Hình vẽ 3.5. Test case LG-10

Bảng 3.5. Test case LG-10

Các bước thực hiện	Kết quả mong muốn	Kết quả
1. Mở trang đăng nhập	Hiển thị thông báo nhập dữ liệu ở trường bắt buộc	Đạt

2. Không nhập thông tin		
3. Ấn đăng nhập		

Test case ID: LG-11

Test case: Đăng nhập thành công

LG-11 Đăng nhập đúng tài khoản rồi K	Command	Target	Value
1	open	/	
2	set window size	945x1012	
3	execute script	return !document.querySelector('btn-login')	isLoginNeeded
4	if	!\$[isLoginNeeded]	
5	click	css= py-2:nth-child(1)	
6	click	css= hover(3A bg-red-50	
7	mouse over	css= hover(3A bg-red-50	
8	mouse out	css= hover(3A bg-red-50	
9	click	css= bg-red-600	
10	end		
11	click	linkText=Đăng nhập	
12	assert text	css= text-3xl	Đăng nhập
13	assert element present	id=usernameOrEmail	
14	assert element present	id=password	
15	click	id=usernameOrEmail	
16	type	id=usernameOrEmail	customer
17	type	id=password	123456aA@
18	click	css= disabled(3Aopacity-50	
19	assert text	css= go3958317564	Đăng nhập thành công!
20	close		

Hình vẽ 3.6. Test case LG-11

Bảng 3.6. Test case LG-11

Các bước thực hiện	Kết quả mong muốn	Kết quả
1. Mở trang đăng nhập	Hiển thị thông báo đăng nhập thành công	Đạt
2. Nhập tài khoản đúng		
3. Ấn đăng nhập		

3.1.1.3. Kiểm thử trang Đăng ký

Test case ID: RG-04

Test case: Kiểm tra hiển thị các trường nhập liệu trang đăng ký

RG-04 Trang đăng ký hiển thị đủ thông tin	Command	Target	Value
1	open	/	
2	set window size	945x1012	
3	execute script	return !document.querySelector('.btn-login')	isLoginNeeded
4	if	!\$[isLoginNeeded]	
5	click	css= py-2 nth-child(1)	
6	click	css= hover\3A bg-red-50	
7	mouse over	css= hover\3A bg-red-50	
8	mouse out	css= hover\3A bg-red-50	
9	click	css= bg-red-600	
10	end		
11	click	linkText=Đăng ký	
12	assert element present	id=username	
13	click	id=email	
14	assert element present	id=email	
15	click	id=password	
16	assert element present	id=password	
17	click	id=name	
18	click	id=name	
19	assert element present	id=name	
20	click	id=phone	
21	assert element present	id=phone	

Hình vẽ 3.7. Test case RG-04

Bảng 3.7. Test case RG-04

Các bước thực hiện	Kết quả mong muốn	Kết quả
1. Vào trang đăng ký 2. Kiểm tra các ô nhập liệu	Các ô nhập liệu hiển thị đủ	Đạt

Test case ID: RG-07
Test case: Không nhập thông tin ẩn đăng ký

RG-07 Gửi thông tin trống form vẫn hiển thị	Command	Target	Value
1	open	/	
2	set window size	945x1012	
3	execute script	return !document.querySelector('.btn-login')	isLoginNeeded
4	if	!\$[isLoginNeeded]	
5	click	css= py-2 nth-child(1)	
6	click	css= hover\3A bg-red-50	
7	mouse over	css= hover\3A bg-red-50	
8	mouse out	css= hover\3A bg-red-50	
9	click	css= bg-red-600	
10	end		
11	click	linkText=Đăng ký	
12	click	css= disabled\3A opacity-50	
13	assert editable	id=username	

Hình vẽ 3.8. Test case RG-07

Bảng 3.8. Test case RG-07

Các bước thực hiện	Kết quả mong muốn	Kết quả
1. Vào trang đăng ký 2. Ẩn đăng ký	Thông báo bắt buộc nhập	Đạt

Test case ID: RG-10

Test case: Kiểm tra trùng Username

RG-10 Kiểm tra trùng username nếu đ	Command	Target	Value
1	open	/	
2	set window size	945x1012	
3	execute script	return !document.querySelector("btn-login")	isLoginNeeded
4	if	!\$(isLoginNeeded)	
5	click	css= py-2:nth-child(1)	
6	click	css= hover!3A bg-red-50	
7	mouse over	css= hover!3A bg-red-50	
8	mouse out	css= hover!3A bg-red-50	
9	click	css= bg-red-600	
10	end		
11	click	linkText=Đăng ký	
12	click	id=username	
13	type	id=username	teest
14	click	id=password	
15	type	id=password	123456
16	click	id=name	
17	type	id=name	Nguyễn Văn B
18	click	id=phone	
19	type	id=phone	0333333333
20	click	id=email	
21	type	id=email	a@gmail.com
22	click	css= disabled!3Aopacity-50	
23	assert text	css= go3958317564	Username đã tồn tại

Hình vẽ 3.9. Test case RG-10

Bảng 3.9. Test case RG-10

Các bước thực hiện	Kết quả mong muốn	Kết quả
1. Vào trang đăng ký 2. Nhập username đã tồn tại 3. Ấn đăng ký	Thông báo đã tồn tại username	Đạt

Test case ID: RG-13

Test case: Đăng ký thành công

RG-13 Nhập đúng thông tin đăng ký th	Command	Target	Value
1	open	/	
2	set window size	945x1012	
3	execute script	return !document.querySelector("btn-login")	isLoginNeeded
4	if	!\$(isLoginNeeded)	
5	click	css= py-2:nth-child(1)	
6	click	css= hover!3A bg-red-50	
7	mouse over	css= hover!3A bg-red-50	
8	mouse out	css= hover!3A bg-red-50	
9	click	css= bg-red-600	
10	end		
11	click	linkText=Đăng ký	
12	click	id=username	
13	type	id=username	teest11111
14	click	id=password	
15	type	id=password	123456
16	click	id=name	
17	type	id=name	Nguyễn Văn B
18	click	id=phone	
19	type	id=phone	0333333334
20	click	id=email	
21	type	id=email	abcdcd@gmail.com
22	click	css= disabled!3Aopacity-50	
23	assert text	css= go3958317564	Đăng ký thành công!

Hình vẽ 3.10. Test case RG-13

Bảng 3.10. Test case RG-13

Các bước thực hiện	Kết quả mong muốn	Kết quả
1. Vào trang đăng ký 2. Nhập thông tin hợp lệ 3. Ấn đăng ký	Thông báo đăng ký thành công	Đạt

3.1.1.4. Kiểm thử trang Tìm chuyến

Test case ID: SR-05

Test case: Kiểm tra hiển thị các trường nhập liệu trang tìm chuyến

SR-05 Form tìm kiếm có đủ trường dữ	Command	Target
1	open	/
2	set window size	945x1012
3	click	css= flex:nth-child(2) > span
4	mouse over	css= .text-primary > span
5	mouse out	css= .text-primary > span
6	assert element present	id=search-departureLocationId
7	assert element present	id=search-arrivalLocationId
8	click	id=search-departureDate
9	assert element present	id=search-departureDate

Hình vẽ 3.11. Test case SR-05

Bảng 3.11. Test case SR-05

Các bước thực hiện	Kết quả mong muốn	Kết quả
1. Vào trang tìm chuyến 2. Kiểm tra form tìm chuyến	Hiển thị đủ điểm đi, điểm đến, ngày đi	Đạt

Test case ID: SR-09

Test case: Chỉ chọn điểm đi và ấn tìm kiếm

SR-09 Chỉ chọn điểm đi rồi tìm kiếm	Command	Target	Value
1	open	/	
2	set window size	945x1012	
3	click	css= flex:nth-child(2) > span	
4	mouse over	css= .text-primary > span	
5	mouse out	css= .text-primary > span	
6	click	id=search-departureLocationId	
7	select	id=search-departureLocationId	label=Hà Nội - Hoàng Mai
8	click	css= .px-6	
9	mouse over	css= .px-6	
10	mouse out	css= .px-6	
11	assert text	css= go3958317564	Vui lòng điền đầy đủ thông tin tìm kiếm

Hình vẽ 3.12. Test case SR-09

Bảng 3.12. Test case SR-09

Các bước thực hiện	Kết quả mong muốn	Kết quả
1. Vào trang tìm chuyến 2. Chọn điểm đi 3. Ấn tìm chuyến	Thông báo lỗi Vui lòng điền đầy đủ thông tin tìm kiếm	Đạt

Test case ID: SR-12

Test case: Tìm kiếm chuyến thành công

SR-12 Nhập đủ thông tin tìm kiếm thành công	Command	Target	Value
1	open	/	
2	set window size	945x1012	
3	click	css= flex:nth-child(2) > span	
4	mouse over	css= text-primary > span	
5	mouse out	css= text-primary > span	
6	click	id=search-departureLocationId	
7	select	id=search-departureLocationId	label=Hà Nội - Long Biên
8	click	id=search-arrivalLocationId	
9	select	id=search-arrivalLocationId	label=Hải Phòng - Lê Chân
10	click	id=search-departureDate	
11	click	css= react-datepicker__week--keyboard-selected	
12	click	css= react-datepicker__day--015	
13	click	css= px-6	
14	mouse over	css= px-6	
15	mouse out	css= disabled3Aopacity-50	

Hình vẽ 3.13. Test case SR-12

Bảng 3.13. Test case SR-12

Các bước thực hiện	Kết quả mong muốn	Kết quả
1. Vào trang tìm chuyến 2. Nhập đủ thông tin tìm chuyến 3. Ấn tìm chuyến	Hiển thị danh sách chuyến phù hợp với thông tin tìm chuyến	Đạt

3.1.1.5. Kiểm thử trang Đặt vé

Test case ID: BK-01

Test case: Kiểm tra hiển thị trang đặt vé

BK-01 Trang đặt vé hiển thị và có tiêu	Command	Target	Value
1	open	/	
2	set window size	945x1012	
3	execute script	return !document.querySelector('btn-login')	isLoginNeeded
4	if	\$(isLoginNeeded)	
5	click	linkText=Đăng nhập	
6	click	id=usernameOrEmail	
7	type	id=usernameOrEmail	customer
8	type	id=password	123456aA@
9	click	css= disabled:3Aopacity-50	
10	mouse over	css= disabled:3Aopacity-50	
11	end		
12	click	id=departureLocationId	
13	select	id=departureLocationId	label=Hà Nội - Long Biên
14	click	id=arrivalLocationId	
15	select	id=arrivalLocationId	label=Hải Phòng - Lê Chân
16	click	id=departureDate	
17	click	css= react-datepicker__day--014	
18	click	css= disabled:3Aopacity-50	
19	click	css= px-6 nth-child(2)	
20	assert text	xpath=/h1[contains(, 'Đặt vé xe khách')]	Đặt vé xe khách

Hình vẽ 3.14. Test case BK-01

Bảng 3.14. Test case BK-01

Các bước thực hiện	Kết quả mong muốn	Kết quả
1.Tìm kiếm chuyến xe 2. Chọn chuyến xe 3. Kiểm tra trang đặt vé hiển thị và có tiêu đề	Trang đặt vé hiển thị và có tiêu đề	Đạt

Test case ID: BK-04
Test case: Kiểm tra các trường thông tin liên hệ

BK-04 Form có hiển thị các trường thông tin	Command	Target	Value
1	open	/	
2	set window size	945x1012	
3	execute script	return !document.querySelector('btn-login')	isLoginNeeded
4	if	\$(isLoginNeeded)	
5	click	linkText=Đăng nhập	
6	click	id=usernameOrEmail	
7	type	id=usernameOrEmail	customer
8	type	id=password	123456aA@
9	click	css= disabled:3Aopacity-50	
10	mouse over	css= disabled:3Aopacity-50	
11	end		
12	click	id=departureLocationId	
13	select	id=departureLocationId	label=Hà Nội - Long Biên
14	click	id=arrivalLocationId	
15	select	id=arrivalLocationId	label=Hải Phòng - Lê Chân
16	click	id=departureDate	
17	click	css= react-datepicker__day--014	
18	click	css= disabled:3Aopacity-50	
19	click	css= px-6 nth-child(2)	
20	assert text	xpath=/h1[contains(, 'Đặt vé xe khách')]	Đặt vé xe khách
21	assert text	css= mb-4 > .font-bold	Thông tin liên hệ
22	assert element present	name=customerName	
23	assert element present	name=customerPhone	
24	assert element present	name=customerEmail	
25	assert element present	name=paymentMethod	
26	assert element present	css= px-6	

Hình vẽ 3.15. Test case BK-04

Bảng 3.15. Test case BK-04

Các bước thực hiện	Kết quả mong muốn	Kết quả
1. Tìm kiếm chuyến xe 2. Chọn chuyến xe 3. Kiểm tra trang đặt vé 4. Trang đặt vé hiển thị đủ các trường thông tin liên hệ	Trang đặt vé hiển thị đủ các trường thông tin liên hệ	Đạt

Test case ID: BK-08

Test case: Kiểm tra nút đặt vé

BK-08 Nút đặt vé hiển thị	Command	Target	Value
1	open	/	
2	set window size	945x1012	
3	click	id=departureLocationId	
4	select	id=departureLocationId	label=Hà Nội - Long Biên
5	click	id=arrivalLocationId	
6	select	id=arrivalLocationId	label=Hải Phòng - Lê Chân
7	click	id=departureDate	
8	click	css= react-datepicker__day--014	
9	click	css= disabled3Aopacity-50	
10	mouse over	css= disabled3Aopacity-50	
11	mouse out	css= disabled3Aopacity-50	
12	click	css= px-6 nth-child(2)	
13	assert text	css= px-6	Vui lòng chọn ghế

Hình vẽ 3.16. Test case BK-08

Bảng 3.16. Test case BK-08

Các bước thực hiện	Kết quả mong muốn	Kết quả
1. Tìm kiếm chuyến xe 2. Chọn chuyến xe 3. Kiểm tra nút đặt vé	Nút đặt vé hiển thị Vui lòng chọn ghế khi chưa chọn ghế	Đạt

Test case ID: BK-09

Test case: Đặt vé thành công

BK-09 Đặt vé thành công	Command	Target	Value
1	open	/	
2	set window size	945x1012	
3	click	id=departureLocationId	
4	select	id=departureLocationId	label=Hà Nội - Long Biên
5	click	id=arrivalLocationId	
6	select	id=arrivalLocationId	label=Hải Phòng - Lê Chân
7	click	id=departureDate	
8	click	css= react-datepicker__day--014	
9	click	css= disabled3Aopacity-50	
10	mouse over	css= disabled3Aopacity-50	
11	mouse out	css= disabled3Aopacity-50	
12	click	css= px-6 nth-child(2)	
13	click	css= grid nth-child(3) > p-2 nth-child(3)	
14	click	css= px-6	
15	click	css= mb-4 > text-2xl	
16	assert text	css= mb-4 > text-2xl	Đặt vé thành công!

Hình vẽ 3.17. Test case BK-09

Bảng 3.17. Test case BK-09

Các bước thực hiện	Kết quả mong muốn	Kết quả
1. Tìm kiếm chuyến xe 2. Chọn chuyến xe 3. Nhập đầy đủ thông tin 4. Nhấn nút Đặt vé 5. Kiểm tra thông đặt vé thành công	Hiện thị thông báo Đặt vé thành công	Đạt

3.1.1.6. Kiểm thử trang Thanh toán

Test case ID: PM-01

Test case: Kiểm tra hiển thị trang thanh toán

PM-01 Trang thanh toán hiển thị	Command	Target	Value
1	open	/	
2	set window size	945x1012	
3	execute script	return !!document.querySelector(' btn-login')	isLoginNeeded
4	if	\$(isLoginNeeded)	
5	click	linkText=Đăng nhập	
6	click	id=usernameOrEmail	
7	type	id=usernameOrEmail	customer
8	click	id=password	
9	type	id=password	123456aA@
10	click	css= disabled3Aopacity-50	
11	end		
12	click	css= flex:nth-child(3) > span	
13	mouse over	css= flex:nth-child(3) > span	
14	mouse over	css= text-primary > span	
15	mouse out	css= text-primary > span	
16	wait for element visible	xpath=//div[//span[contains(., 'Chờ thanh toán')]]/button[contains(., 'Thanh toán')]	30000
17	verify element present	xpath=//div[//span[contains(., 'Chờ thanh toán')]]/button[contains(., 'Thanh toán')]	
18	click	xpath=//div[//span[contains(., 'Chờ thanh toán')]]/button[contains(., 'Thanh toán')]	
19	assert text	css= text-3xl	Thanh toán

Hình vẽ 3.18. Test case PM-01

Bảng 3.18. Test case PM-01

Các bước thực hiện	Kết quả mong muốn	Kết quả
1. Đăng nhập 2. Vào mục Vé của tôi 3. Ấn nút Thanh toán 4. Kiểm tra trang thanh toán hiển thị	Trang thanh toán hiển thị thành công	Đạt

Test case ID: PM-07

Test case: Kiểm tra các phương thức thanh toán

PM-07 Hiển thị phương thức thanh toán	Command	Target	Value
1	open	/	
2	set window size	945x1012	
3	execute script	return !document.querySelector('btn-login')	isLoginNeeded
4	if	\$(isLoginNeeded)	
5	click	linkText=Đăng nhập	
6	click	id=usernameOrEmail	
7	type	id=usernameOrEmail	customer
8	click	id=password	
9	type	id=password	123456aA@
10	click	css= disabled:3Aopacity-50	
11	end		
12	click	css= flex:nth-child(3) > span	
13	mouse over	css= flex:nth-child(3) > span	
14	mouse over	css= text-primary > span	
15	mouse out	css= text-primary > span	
16	wait for element visible	xpath=//div[//span[contains(,"Chờ thanh toán")]]/button[contains(,"Thanh toán")]	30000
17	verify element present	xpath=//div[//span[contains(,"Chờ thanh toán")]]/button[contains(,"Thanh toán")]	
18	click	xpath=//div[//span[contains(,"Chờ thanh toán")]]/button[contains(,"Thanh toán")]	
19	assert text	css= bg-white:nth-child(2) > .text-xl	Phương thức thanh toán

Hình vẽ 3.19. Test case PM-07

Bảng 3.19. Test case PM-07

Các bước thực hiện	Kết quả mong muốn	Kết quả
1. Đăng nhập 2. Vào mục Vé của tôi 3. Ấn nút Thanh toán 4. Kiểm tra phương thức thanh toán hiển thị	Phương thức thanh toán hiển thị thành công	Đạt

Test case ID: PM-10
Test case: Thanh toán trả tiền mặt khi lên xe

PM-10 Thanh toán bằng phương thức tiền	Command	Target	Value
5	click	linkText=Đăng nhập	
6	click	id=usernameOrEmail	
7	type	id=usernameOrEmail	customer
8	click	id=password	
9	type	id=password	123456aA@
10	click	css= disabled:3Aopacity-50	
11	end		
12	click	css= flex:nth-child(3) > span	
13	mouse over	css= flex:nth-child(3) > span	
14	mouse over	css= text-primary > span	
15	mouse out	css= text-primary > span	
16	wait for element visible	xpath=//div[//span[contains(,"Chờ thanh toán")]]/button[contains(,"Thanh toán")]	30000
17	verify element present	xpath=//div[//span[contains(,"Chờ thanh toán")]]/button[contains(,"Thanh toán")]	
18	click	xpath=//div[//span[contains(,"Chờ thanh toán")]]/button[contains(,"Thanh toán")]	
19	wait for element visible	xpath=//button[contains(normalize-space(,"Xác nhận thanh toán")]	30000
20	click	xpath=//button[contains(normalize-space(,"Xác nhận thanh toán")]	
21	assert text	css= go3958317564	Đặt vé thành công! Vui lòng thanh toán khi lên xe

Hình vẽ 3.20. Test case PM-10

Bảng 3.20. Test case PM-10

Các bước thực hiện	Kết quả mong muốn	Kết quả
1. Đăng nhập 2. Vào mục Vé của tôi	Hiện thị thông báo "Đặt vé thành công! Vui lòng thanh toán khi lên xe"	Đạt

3. Ấn nút Thanh toán		
4. Chọn phương thức tiền mặt		
5. Ấn xác nhận thanh toán		

3.1.1.7. Kiểm thử trang Vé của tôi

Test case ID: MB-01

Test case: Kiểm tra hiển thị trang vé của tôi

MB-01 Trang Vé của tôi hiển thị đúng	Command	Target	Value
1	open	/	
2	set window size	945x1012	
3	execute script	return !!document.querySelector('btn-login')	isLoginNeeded
4	if	\$(isLoginNeeded)	
5	click	linkText=Đăng nhập	
6	click	id=usernameOrEmail	
7	type	id=usernameOrEmail	customer
8	click	id=password	
9	type	id=password	123456aA@
10	click	css=.disabled:3Aopacity-50	
11	end		
12	click	css= flex:nth-child(3) > span	
13	mouse over	css= flex:nth-child(3) > span	
14	mouse over	css= text-primary > span	
15	mouse out	css= text-primary > span	
16	assert text	css= flex > .text-3xl	Vé của tôi

Hình vẽ 3.21. Test case MB-01

Bảng 3.21. Test case MB-01

Các bước thực hiện	Kết quả mong muốn	Kết quả
1. Đăng nhập	Trang vé của tôi hiển thị	Đạt
2. Vào mục Vé của tôi		
3. Kiểm tra trang		

Test case ID: MB-08

Test case: Thực hiện hủy vé

MB-08 Hủy vé thành công nếu có nút hủy	Command	Target	Value
1	open	/	
2	set window size	945x1012	
3	execute script	return !document.querySelector(" btn-login")	isLoginNeeded
4	if	!isLoginNeeded	
5	click	linkText=Đăng nhập	
6	click	id=usernameOrEmail	
7	type	id=usernameOrEmail	customer
8	click	id=password	
9	type	id=password	123456aA@
10	click	css= disabled3Aopacity-50	
11	end		
12	click	css= flex:nth-child(3) > span	
13	mouse over	css= flex:nth-child(3) > span	
14	mouse over	css= text-primary > span	
15	mouse out	css= text-primary > span	
16	wait for element visible	xpath=//div[//span[contains(,"Chò thanh toán")]]/button[contains(,"Hủy")][1]	30000
17	click	xpath=//div[//span[contains(,"Chò thanh toán")]]/button[contains(,"Hủy")][1]	
18	click	xpath=//button[contains(,"Xác nhận hủy")]	
19	assert text	css= go4109123758:nth-child(2) go3958317564	Hủy vé thành công!

Hình vẽ 3.22. Test case MB-08

Bảng 3.22. Test case MB-08

Các bước thực hiện	Kết quả mong muốn	Kết quả
1. Đăng nhập 2. Vào mục Vé của tôi 3. Nhấn nút Hủy vé 4. Vé được hủy thành công	Vé được hủy thành công	Đạt

Test case ID: MB-10
Test case: Xem chi tiết vé

MB-10 Xem chi tiết của vé	Command	Target	Value
1	open	/	
2	set window size	945x1012	
3	execute script	return !document.querySelector(" btn-login")	isLoginNeeded
4	if	!isLoginNeeded	
5	click	linkText=Đăng nhập	
6	click	id=usernameOrEmail	
7	type	id=usernameOrEmail	customer
8	click	id=password	
9	type	id=password	123456aA@
10	click	css= disabled3Aopacity-50	
11	end		
12	click	css= flex:nth-child(3) > span	
13	mouse over	css= flex:nth-child(3) > span	
14	mouse over	css= text-primary > span	
15	mouse out	css= text-primary > span	
16	assert element present	css= bg-white:nth-child(1) .w-full:nth-child(1)	
17	assert text	css= bg-white:nth-child(1) .w-full:nth-child(1)	Xem chi tiết
18	click	css= bg-white:nth-child(1) .w-full:nth-child(1)	
19	assert text	css= sticky > text-2xl	Chi tiết vé

Hình vẽ 3.23. Test case MB-10

Bảng 3.23. Test case MB-10

Các bước thực hiện	Kết quả mong muốn	Kết quả
1. Đăng nhập 2. Vào mục Vé của tôi	Chi tiết vé hiển thị thành công	Đạt

3. Ấn nút Xem chi tiết		
4. Hiện thị thông tin chi tiết		

Test case ID: MB-14
Test case: Ấn nút thanh toán chuyển qua trang thanh toán

MB-14 Ấn nút thanh toán hiển thị trang thanh toán	Command	Target	Value
1	open	/	
2	set window size	945x1012	
3	execute script	return !document.querySelector('btn-login')	isLoginNeeded
4	if	\$(isLoginNeeded)	
5	click	linkText=Đăng nhập	
6	click	id=usernameOrEmail	
7	type	id=usernameOrEmail	customer
8	click	id=password	
9	type	id=password	123456aA@
10	click	css=.disabled3Aopacity-50	
11	end		
12	click	css=.flex:nth-child(3) > span	
13	mouse over	css=.flex:nth-child(3) > span	
14	mouse over	css=.text-primary > span	
15	mouse out	css=.text-primary > span	
16	wait for element visible	xpath=//*[@div//span[contains(,Chờ thanh toán)]]/button[contains(,Thanh toán)]	30000
17	verify element present	xpath=//*[@div//span[contains(,Chờ thanh toán)]]/button[contains(,Thanh toán)]	
18	click	xpath=//*[@div//span[contains(,Chờ thanh toán)]]/button[contains(,Thanh toán)]	

Hình vẽ 3.24. Test case MB-14

Bảng 3.24. Test case MB-14

Các bước thực hiện	Kết quả mong muốn	Kết quả
1. Đăng nhập 2. Vào mục Vé của tôi 3. Ấn nút Thanh toán 4. Kiểm tra trang thanh toán hiển thị	Điều hướng sang trang thanh toán	Đạt

3.1.1.8. Kiểm thử trang Hồ sơ
Test case ID: PF-01
Test case: Kiểm tra hiển thị trang hồ sơ

PF-01 Trang thông tin cá nhân hiển thị sau	Command	Target	Value
1	open	/	
2	set window size	945x1012	
3	execute script	return !!document.querySelector(' btn-login')	isLoginNeeded
4	if	\$(isLoginNeeded)	
5	click	linkText=Đăng nhập	
6	click	id=usernameOrEmail	
7	type	id=usernameOrEmail	customer
8	click	id=password	
9	type	id=password	123456aA@
10	click	css= disabled3Aopacity-50	
11	end		
12	click	css= sm3Apx-6 nth-child(1)	
13	click	css= py-2 nth-child(1)	
14	click	linkText=Tài khoản	
15	assert title	Thông tin cá nhân - BookingHub	

Hình vẽ 3.25. Test case PF-01

Bảng 3.25. Test case PF-01

Các bước thực hiện	Kết quả mong muốn	Kết quả
1. Đăng nhập 2. Vào trang thông tin cá nhân	Trang thông tin cá nhân hiển thị thành công	Đạt

Test case ID: PF-08
Test case: Thay đổi Họ tên

PF-08 Có thể điền lại họ tên và lưu lại	Command	Target	Value
1	open	/	
2	set window size	945x1012	
3	execute script	return !!document.querySelector(' btn-login')	isLoginNeeded
4	if	\$(isLoginNeeded)	
5	click	linkText=Đăng nhập	
6	click	id=usernameOrEmail	
7	type	id=usernameOrEmail	customer
8	click	id=password	
9	type	id=password	123456aA@
10	click	css= disabled3Aopacity-50	
11	end		
12	click	css= sm3Apx-6 nth-child(1)	
13	click	css= py-2 nth-child(1)	
14	click	linkText=Tài khoản	
15	click	name=name	
16	type	name=name	Nguyễn Văn B
17	click	css= bg-blue-600	
18	mouse over	css= bg-blue-600	
19	mouse out	css= bg-blue-600	
20	assert text	css= go3958317564	Cập nhật profile thành công!

Hình vẽ 3.26. Test case PF-08

Bảng 3.26. Test case PF-08

Các bước thực hiện	Kết quả mong muốn	Kết quả
1. Đăng nhập 2. Vào trang thông tin cá nhân 3. Nhập họ tên mới 4. Lưu thay đổi	Thông tin được lưu thành công	Đạt

Test case ID: PF-09

Test case: Thay đổi email

PF-09 Có thể điền lại email và lưu lại	Command	Target	Value
1	open	/	
2	set window size	945x1012	
3	execute script	return !!document.querySelector(".btn-login")	isLoginNeeded
4	if	\$(isLoginNeeded)	
5	click	linkText=Đăng nhập	
6	click	id=usernameOrEmail	
7	type	id=usernameOrEmail	customer
8	click	id=password	
9	type	id=password	123456aA@
10	click	css= disabled3Aopacity-50	
11	end		
12	click	css= sm3Apx-6 nth-child(1)	
13	click	css= py-2 nth-child(1)	
14	click	linkText=Tài khoản	
15	click	name=email	
16	type	name=email	customer1@gmail.com
17	click	css= bg-blue-600	
18	mouse over	css= bg-blue-600	
19	mouse out	css= bg-blue-600	
20	assert text	css= go3958317564	Cập nhật profile thành công!

Hình vẽ 3.27. Test case PF-09

Bảng 3.27. Test case PF-09

Các bước thực hiện	Kết quả mong muốn	Kết quả
1. Đăng nhập 2. Vào trang thông tin cá nhân 3. Nhập email mới 4. Lưu thay đổi	Thông tin được lưu thành công	Đạt

Test case ID: PF-13
Test case: Kiểm tra lỗi để trống SĐT

PF-13 Để trống thông tin SĐT thông báo lỗi	Command	Target	Value
1	open	/	
2	set window size	945x1012	
3	execute script	return !!document.querySelector(".btn-login")	isLoginNeeded
4	if	\$(isLoginNeeded)	
5	click	linkText=Đăng nhập	
6	click	id=usernameOrEmail	
7	type	id=usernameOrEmail	customer
8	click	id=password	
9	type	id=password	123456aA@
10	click	css= disabled3Aopacity-50	
11	end		
12	click	css= sm3Apx-6 nth-child(1)	
13	click	css= py-2 nth-child(1)	
14	click	linkText=Tài khoản	
15	click	name=phone	
16	click	name=phone	
17	double click	name=phone	
18	type	name=phone	
19	click	css= bg-blue-600	
20	assert editable	name=phone	

Hình vẽ 3.28. Test case PF-13

Bảng 3.28. Test case PF-13

Các bước thực hiện	Kết quả mong muốn	Kết quả
--------------------	-------------------	---------

1. Đăng nhập 2. Vào trang thông tin cá nhân 3. Xóa SĐT 4. Lưu thay đổi	Thông báo bắt buộc nhập	Đạt
---	-------------------------	-----

3.1.2. Kiểm thử phía quản trị viên

3.1.2.1. Kiểm thử trang Đăng nhập

Test case ID: AL-01

Test case: Kiểm tra hiển thị đăng nhập

	Command	Target	Value
AL-01 Trang admin login hiển thị form đăng	1. <i>open</i>	/	
AL-02 Tiêu đề trang đăng nhập hiển thị đủ	2. <i>set window size</i>	945x1012	
AL-03 Đăng nhập sai báo lỗi sai thông tin	3. <i>store xpath count</i>	<i>xpath=//button[contains(@data-testid,'logout')]</i>	logoutCount
AL-04 Đăng nhập đúng thông báo đăng nh	4. <i>if</i>	<i>\${logoutCount} > 0</i>	
	5. <i>click</i>	<i>css=[data-testid="admin-header-logout"]</i>	
	6. <i>wait for element visible</i>	<i>xpath=//button[contains(.,'Đăng xuất')]</i>	30000
	7. <i>click</i>	<i>xpath=//button[contains(.,'Đăng xuất')]</i>	
	8. <i>end</i>		
	9. <i>assert element present</i>	<i>id=admin-usernameOrEmail</i>	
	10. <i>assert element present</i>	<i>id=admin-password</i>	
	11. <i>close</i>		

Hình vẽ 3.29. Test case AL-01

Bảng 3.29. Test case AL-01

Các bước thực hiện	Kết quả mong muốn	Kết quả
1. Mở trang đăng nhập 2. Kiểm tra form đăng nhập	Form đăng nhập hiển thị đủ	Đạt

Test case ID: AL-03

Test case: Nhập sai tài khoản mật khẩu

1	open	/	
2	set window size	945x1012	
3	store xpath count	xpath=//button[contains(@data-testid,'logout')]	logoutCount
4	if	`\${logoutCount}` > 0	
5	click	css=[data-testid="admin-header-logout"]	
6	wait for element visible	xpath=//button[contains(., 'Đăng xuất')]	30000
7	click	xpath=//button[contains(., 'Đăng xuất')]	
8	end		
9	click	id=admin-usernameOrEmail	
10	type	id=admin-usernameOrEmail	anccnncnc
11	type	id=admin-password	12ss
12	click	css=.hover\3A bg-blue-700	
13	assert text	css=.go3958317564	Sai tài khoản hoặc mật khẩu
14	close		

Hình vẽ 3.30. Test case AL-03

Bảng 3.30. Test case AL-03

Các bước thực hiện	Kết quả mong muốn	Kết quả
1. Mở trang đăng nhập 2. Nhập thông tin đăng nhập sai 3. Kiểm tra thông báo	Thông báo sai tài khoản hoặc mật khẩu	Không đạt

Test case ID: AL-04
Test case: Đăng nhập đúng thông báo đăng nhập thành công

Command	Target	Value
1	open	/
2	set window size	945x1012
3	store xpath count	xpath=//button[contains(@data-testid,'logout')]logoutCount
4	if	`\${logoutCount}` > 0
5	click	css=[data-testid="admin-header-logout"]
6	wait for element visible	xpath=//button[contains(., 'Đăng xuất')]30000
7	click	xpath=//button[contains(., 'Đăng xuất')]
8	end	
9	click	id=admin-usernameOrEmail
10	type	id=admin-usernameOrEmailadmin
11	type	id=admin-password123456aA@
12	click	css=.hover\3A bg-blue-700
13	assert text	css=.go3958317564Đăng nhập thành công!
14	close	

Hình vẽ 3.31. Test case AL-04

Bảng 3.31. Test case AL-04

Các bước thực hiện	Kết quả mong muốn	Kết quả
1. Mở trang đăng nhập 2. Nhập thông tin đúng 3. Kiểm tra thông báo	Đăng nhập thành công	Đạt

3.1.2.2. Kiểm thử trang Dashboad

Test case ID: AD-01
Test case: Kiểm tra hiển thị trang dashboard

	Command	Target	Value
1	open	/	
2	set window size	945x1012	
3	run	Đăng nhập Admin nhà xe	
4	assert title	Dashboard - BookingHub Admin	
5	close		

Hình vẽ 3.32. Test case AD-01

Bảng 3.32. Test case AD-01

Các bước thực hiện	Kết quả mong muốn	Kết quả
1. Mở trang Dashboard 2. Kiểm tra trang	Trang hiển thị thành công	Đạt

Test case ID: AD-04
Test case: Kiểm tra hiển thị trang dashboard

	Command	Target	Value
1	open	/	
2	set window size	945x1012	
3	run	Đăng nhập Admin nhà xe	
4	assert text	css=.mb-8 > .text-xl	Thông kê đặt vé
5	close		

Hình vẽ 3.33. Test case AD-04

Bảng 3.33. Test case AD-04

Các bước thực hiện	Kết quả mong muốn	Kết quả
1. Mở trang Dashboard 2. Kiểm tra hiển thị Thông kê đặt vé	Thông tin thông kê đặt vé hiển thị đúng	Đạt

3.1.2.3. Kiểm thử trang Quản lý địa điểm

Test case ID: LC-01
Test case: Kiểm tra hiển thị quản lý địa điểm

	Command	Target	Value
1	open	/	
2	set window size	945x1012	
3	run	Đăng nhập Super Admin	
4	click	css=div:nth-child(3) > .w-full	
5	click	linkText=Quản lý địa điểm	
6	assert title	Quản lý Địa điểm - BookingHub	
7	close		

Hình vẽ 3.34. Test case LC-01

Bảng 3.34. Test case LC-01

Các bước thực hiện	Kết quả mong muốn	Kết quả
1. Mở trang quản lý địa điểm	Trang hiển thị thành công	Đạt
2. Kiểm tra trang		

Test case ID: LC-07

Test case: Nhập đủ dữ liệu và thêm địa điểm mới

	Command	Target	Value
1	open	/	
2	set window size	945x1012	
3	run	Đăng nhập Super Admin	
4	click	css=div:nth-child(3) > .w-full	
5	click	linkText=Quản lý địa điểm	
6	wait for element visible	css=[data-testid="admin-locations-btn-add"]	30000
7	click	css=[data-testid="admin-locations-btn-add"]	
8	click	name=city	
9	type	name=city	Thành phố test
10	click	name=district	
11	type	name=district	Test
12	click	id=latitude	
13	type	id=latitude	21.028222
14	click	id=longitude	
15	type	id=longitude	105.88888
16	click	css=gap-2 > .bg-blue-600	
17	assert text	css=go3958317564	Tạo địa điểm thành công!
18	close		

Hình vẽ 3.35. Test case LC-07

Bảng 3.35. Test case LC-07

Các bước thực hiện	Kết quả mong muốn	Kết quả
1. Mở trang quản lý địa điểm	Thêm bản ghi thành công	Đạt
2. Ấn thêm địa điểm		
3. Nhập đầy đủ thông tin		
4. Ấn lưu lại		
5. Kiểm tra thông báo		

Test case ID: LC-08

Test case: Chỉnh sửa địa điểm

	Command	Target	Value
1	open	/	
2	set window size	945x1012	
3	run	Đăng nhập Super Admin	
4	click	css=div:nth-child(3) > .w-full	
5	click	linkText=Quản lý địa điểm	
6	click	css=.hover\:3A bg-gray-50:nth-child(1) .p-1:nth-child(1) path	
7	type	name=district	Test1
8	click	css=.gap-2 > .bg-blue-600	
9	assert text	css=.go3958317564	Cập nhật thành công!
10	close		

Hình vẽ 3.36. Test case LC-08

Bảng 3.36. Test case LC-08

Các bước thực hiện	Kết quả mong muốn	Kết quả
1. Mở trang quản lý địa điểm 2. Ấn sửa địa điểm 3. Chỉnh sửa thông tin 4. Ấn lưu lại 5. Kiểm tra thông báo	Cập nhật bản ghi thành công	Đạt

Test case ID: LC-09

Test case: Xóa địa điểm

	Command	Target	Value
1	open	/	
2	set window size	945x1012	
3	run	Đăng nhập Super Admin	
4	click	css=div:nth-child(3) > .w-full	
5	click	linkText=Quản lý địa điểm	
6	click	css=.hover\:3A bg-gray-50:nth-child(1) .hover\:3A bg-red-50	
7	click	css=.bg-red-600	
8	assert text	css=.go3958317564	Xóa thành công!
9	close		

Hình vẽ 3.37. Test case LC-09

Bảng 3.37. Test case LC-09

Các bước thực hiện	Kết quả mong muốn	Kết quả
1. Mở trang quản lý địa điểm 2. Ấn xóa địa điểm 3. Xác nhận xóa 4. Kiểm tra thông báo	Xóa thành công, bảng dữ liệu không hiển thị bản ghi vừa xóa	Đạt

3.1.2.4. Kiểm thử trang Quản lý nhà xe

Test case ID: BC-01

Test case: Kiểm tra hiển thị quản lý nhà xe

	Command	Target	Value
1	open	/	
2	set window size	945x1012	
3	run	Đăng nhập Admin nhà xe	
4	click	linkText=Nhà xe	
5	assert title	Quản lý Nhà xe - BookingHub	
6	close		

Hình vẽ 3.38. Test case BC-01

Bảng 3.38. Test case BC-01

Các bước thực hiện	Kết quả mong muốn	Kết quả
1. Mở trang quản lý nhà xe	Trang quản lý nhà xe hiển thị thành công	Đạt
2. Kiểm tra trang quản lý nhà xe		

Test case ID: BC-06

Test case: Nhập đủ dữ liệu và thêm nhà xe mới

	Command	Target	Value
1	open	/	
2	set window size	945x1012	
3	run	Đăng nhập Super Admin	
4	click	linkText=Nhà xe	
5	wait for element visible	css=[data-testid="admin-bus-companies-btn-add"]	30000
6	click	css=[data-testid="admin-bus-companies-btn-add"]	
7	click	name=name	
8	type	name=name	Test
9	type	name=email	ackckkc@test.com
10	type	name=phone	0333333333
11	type	name=address	Test
12	click	css=[data-testid="admin-bus-companies-form-submit"]	
13	assert text	css=.go3958317564	Tạo nhà xe thành công!
14	close		

Hình vẽ 3.39. Test case BC-06

Bảng 3.39. Test case BC-06

Các bước thực hiện	Kết quả mong muốn	Kết quả
1. Mở trang quản lý nhà xe	Thêm bản ghi thành công	Đạt
2. Ấn nút thêm nhà xe		
3. Nhập đầy đủ thông tin		
4. Ấn lưu lại		

Test case ID: BC-07

Test case: Cập nhật nhà xe

	Command	Target	Value
1	open	/	
2	set window size	945x1012	
3	run	Đăng nhập Super Admin	
4	click	linkText=Nhà xe	
5	click	css= hover3A bg-gray-50:nth-child(1) .p-1:nth-child(1) path	
6	click	name=name	
7	type	name=name	Test1
8	click	css= gap-2 > .bg-blue-600	
9	assert text	css= go3958317564	Cập nhật thành công!
10	close		

Hình vẽ 3.40. Test case BC-07

Bảng 3.40. Test case BC-07

Các bước thực hiện	Kết quả mong muốn	Kết quả
1. Mở trang quản lý nhà xe 2. Ấn nút cập nhật nhà xe 3. Thay đổi thông tin nhà xe 4. Ấn lưu lại	Cập nhật bản ghi thành công	Đạt

Test case ID: BC-08

Test case: Xóa nhà xe

	Command	Target	Value
1	open	/	
2	set window size	945x1012	
3	run	Đăng nhập Super Admin	
4	click	linkText=Nhà xe	
5	click	css= hover3A bg-gray-50:nth-child(1) path:nth-child(2)	
6	click	css= .bg-red-600	
7	assert text	css= go3958317564	Xóa thành công!
8	close		

Hình vẽ 3.41. Test case BC-08

Bảng 3.41. Test case BC-08

Các bước thực hiện	Kết quả mong muốn	Kết quả
1. Mở trang quản lý nhà xe 2. Ấn nút xóa nhà xe 3. Xác nhận xóa 4. Kiểm tra thông báo	Xóa thành công	Đạt

Test case ID: BC-11

Test case: Tìm kiếm nhà xe theo từ khóa

	Command	Target	Value
1	open	/	
2	set window size	945x1012	
3	run	Đăng nhập Super Admin	
4	click	linkText=Nhà xe	
5	click	css=.p-6	
6	click	css=.sm\3A flex-1	
7	type	css=.sm\3A flex-1	Test
8	click	css=.shrink-0	
9	close		

Hình vẽ 3.42. Test case BC-11

Bảng 3.42. Test case BC-11

Các bước thực hiện	Kết quả mong muốn	Kết quả
1. Mở trang quản lý nhà xe 2. Nhập thông tin tìm kiếm 3. Ấn tìm kiếm 4. Kiểm tra bảng dữ liệu	Tìm kiếm thành công, bảng dữ liệu hiển thị tương ứng với thông tin tìm kiếm	Đạt

3.1.2.5. Kiểm thử trang Quản lý phương tiện

Test case ID: VH-10

Test case: Thêm mới phương tiện thành công

	Command	Target	Value
1	open	/	
2	set window size	945x1012	
3	run	Đăng nhập Admin nhà xe	
4	click	linkText=Phương tiện	
5	click	css=.bg-blue-600	
6	click	name=busName	
7	type	name=busName	Xe 16 chỗ
8	click	name=busType	
9	select	name=busType	label=Limousine
10	click	name=licensePlate	
11	type	name=licensePlate	30A12345
12	click	name=layoutTemplateId	
13	select	name=layoutTemplateId	label=Xe 16 chỗ (16 chỗ)
14	click	name=totalSeats	
15	type	name=totalSeats	16
16	click	css=.gap-2 > .bg-blue-600	
17	assert text	css=.go4109123758:nth-child(2) .go3958317564	Tạo xe thành công!
18	close		

Hình vẽ 3.43. Test case VH-10

Bảng 3.43. Test case VH-10

Các bước thực hiện	Kết quả mong muốn	Kết quả
1. Mở trang quản lý phương tiện 2. Ấn thêm mới	Thêm bản ghi thành công	Đạt

3. Nhập thông tin		
4. Ấn Lưu		
5. Kiểm tra thông báo		

Test case ID: VH-11

Test case: Chỉnh sửa phương tiện thành công

	Command	Target	Value
1	open	/	
2	set window size	945x1012	
3	run	Đăng nhập Admin nhà xe	
4	click	linkText=Phương tiện	
5	click	css=hover\3A bg-gray-50:nth-child(1) p-1:nth-child(1) > .lucide	
6	click	name=busName	
7	type	name=busName	Xe 16 chỗ 1
8	click	css=gap-2 > .bg-blue-600	
9	assert text	css=go4109123758:nth-child(2) .go3958317564	Cập nhật thành công!
10	close		

Hình vẽ 3.44. Test case VH-11

Bảng 3.44. Test case VH-11

Các bước thực hiện	Kết quả mong muốn	Kết quả
1. Mở trang quản lý phương tiện 2. Ấn sửa 3. Thay đổi thông tin 4. Ấn Lưu 5. Kiểm tra thông báo	Cập nhật bản ghi thành công	Đạt

Test case ID: VH-12

Test case: Xóa phương tiện

	Command	Target	Value
1	open	/	
2	set window size	945x1012	
3	run	Đăng nhập Admin nhà xe	
4	click	linkText=Phương tiện	
5	click	css=hover\3A bg-gray-50:nth-child(1) p-1:nth-child(2) > .lucide	
6	click	css=.bg-red-600	
7	assert text	css=go3958317564	Xóa thành công!
8	close		

Hình vẽ 3.45. Test case VH-12

Bảng 3.45. Test case VH-12

Các bước thực hiện	Kết quả mong muốn	Kết quả
1. Mở trang quản lý phương tiện 2. Ấn xóa 3. Ấn Xác nhận xóa 4. Kiểm tra thông báo	Xóa thành công	Đạt

3.1.2.6. Kiểm thử trang quản lý chuyển xe

Test case ID: TR-15

Test case: Xóa chuyển xe

http://localhost:3001		
Command	Target	Value
1	open	/
2	set window size	945x1012
3	run	Đăng nhập Admin nhà xe
4	click	linkText=Quản lý chuyển
5	click	css=.hover\3A bg-gray-50:nth-child(1) path:nth-child(2)
6	click	css= bg-red-600
7	assert text	css= go3958317564 Xóa thành công!
8	close	

Các bước thực hiện	Kết quả mong muốn	Kết quả
1. Mở trang quản lý chuyển xe 2. Xóa chuyển xe 3. Xác nhận xóa 4. Kiểm tra thông báo	Xóa thành công	Không đạt

Test case ID: TR-16

Test case: Sửa chuyển xe thành công

Command	Target	Value
1	open	/
2	set window size	945x1012
3	run	Đăng nhập Admin nhà xe
4	click	linkText=Quản lý chuyển
5	click	css=.hover\3A bg-gray-50:nth-child(1) .hover\3A bg-blue-50
6	click	name=departureTime
7	click	name=departureTime
8	click	name=departureTime
9	type	name=departureTime 2026-04-29T09:00
10	click	css= gap-2 > bg-blue-600
11	assert text	css= go3958317564 Cập nhật thành công!
12	close	

Các bước thực hiện	Kết quả mong muốn	Kết quả
1. Mở trang quản lý chuyển xe 2. Ấn nút sửa chuyển 3. Nhập thông tin thay đổi 4. Ấn Lưu 5. Kiểm tra thông báo	Cập nhật bản ghi thành công	Đạt

Test case ID: TR-17

Test case: Tạo chuyển mới thành công

Command		Target	Value
1	open	/	
2	set window size	945x1012	
3	run	Đăng nhập Admin nhà xe	
4	click	linkText=Quản lý chuyển	
5	wait for element present	css= bg-blue-600	30000
6	click	css= bg-blue-600	
7	click	name=vehicleId	
8	select	name=vehicleId	label=Xe 16 (Ghế ngồi)
9	click	name=departureLocationId	
10	select	name=departureLocationId	label=Hồ Chí Minh - Bình Tân
11	click	name=arrivalLocationId	
12	select	name=arrivalLocationId	label=Đà Nẵng - Thanh Khê
13	click	id=bulkCreate	
14	click	css= react-datepicker-wrapper:nth-child(2) .w-full	
15	click	css= react-datepicker__time-list-item:nth-child(99)	
16	click	css=div:nth-child(1) > .relative .text-sm	
17	click	css= react-datepicker__day--016	
18	click	css=div:nth-child(2) > .relative .w-full	
19	click	css= react-datepicker__day--019	
20	click	css= w-full:nth-child(3)	
21	click	css= disabled:3Aopacity-50	
22	run script	window.scrollTo(0,0)	
23	assert text	css= go3958317564	Tạo chuyển thành công!
24	close		

Các bước thực hiện	Kết quả mong muốn	Kết quả
1. Mở trang quản lý chuyển xe 2. Ấn nút tạo chuyển 3. Nhập thông tin chuyển 4. Ấn tạo chuyển 5. Kiểm tra thông báo	Tạo chuyển thành công	Đạt

3.1.2.7. Kiểm thử trang Quản lý người dùng

Test case ID: US-07

Test case: Thêm người dùng mới thành công

Command		Target	Value
1	open	/	
2	set window size	945x1012	
3	run	Đăng nhập Admin nhà xe	
4	click	css= flex:nth-child(8) > span	
5	click	css= bg-blue-600	
6	assert text	css= bg-white > .text-xl	Thêm người dùng
7	execute script	return 'user_' + Date.now()	username
8	execute script	return \$(username) + '@gmail.com'	mail
9	click	css=div:nth-child(1) > .px-3	
10	type	css=div:nth-child(1) > .px-3	\$(username)
11	click	css=div:nth-child(2) > .border	
12	type	css=div:nth-child(2) > .border	\$(mail)
13	click	css=div:nth-child(3) > .px-3	
14	type	css=div:nth-child(3) > .px-3	test
15	click	css=div:nth-child(4) > .border	
16	select	css=div:nth-child(4) > .border	label=company_admin
17	click	css=div:nth-child(5) > .w-full	
18	type	css=div:nth-child(5) > .w-full	123456aA@
19	click	css= px-4:nth-child(2)	
20	assert text	css= go2072408551	Tạo người dùng thành công!
21	close		

Các bước thực hiện	Kết quả mong muốn	Kết quả
1. Mở trang quản lý người dùng 2. Ấn nút thêm người dùng 3. Nhập thông tin người dùng 4. Ấn Lưu	Thêm bản ghi thành công	Đạt

5. Kiểm tra thông báo		
-----------------------	--	--

Test case ID: US-08

Test case: Cập nhật người dùng

Command	Target	Value
1. open	/	
2. set window size	945x1012	
3. run	Đăng nhập Admin nhà xe	
4. click	css= flex:nth-child(8) > span	
5. click	css= border-t:nth-child(1) .hover(3A bg-blue-50 > .lucide	
6. click	css=div:nth-child(3) > .px-3	
7. type	css=div:nth-child(3) > .px-3	Admin11
8. click	css= px-4:nth-child(2)	
9. assert text	css= go3958317564	Cập nhật người dùng thành công!
10. close		

Các bước thực hiện	Kết quả mong muốn	Kết quả
1. Mở trang quản lý người dùng 2. Ấn nút sửa 3. Nhập thông tin sửa 4. Ấn Lưu 5. Kiểm tra thông báo	Cập nhật bản ghi thành công	Đạt

Test case ID: US-09

Test case: Xóa người dùng

Command	Target	Value
1. open	/	
2. set window size	945x1012	
3. run	Đăng nhập Admin nhà xe	
4. click	css=.flex:nth-child(8) > span	
5. click	css=.border-t:nth-child(5) .p-2:nth-child(2) path:nth-child(2)	
6. click	css=.bg-red-600	
7. assert text	css=.go3958317564	Xóa người dùng thành công!
8. close		

Các bước thực hiện	Kết quả mong muốn	Kết quả
1. Mở trang quản lý người dùng 2. Xóa người dùng 3. Xác nhận xóa 4. Kiểm tra thông báo	Xóa thành công	Đạt

3.1.2.8. Kiểm thử trang Quản lý dịch vụ

Test case ID: VS-08

Test case: Nhập đủ dữ liệu và thêm dịch vụ mới

	Command	Target	Value
1	open	/	
2	set window size	945x1012	
3	run	Đăng nhập Admin nhà xe	
4	click	css=div:nth-child(4) > .w-full	
5	click	linkText=Dịch vụ bổ sung	
6	click	css=.bg-blue-600	
7	click	css=div:nth-child(1) > .px-3	
8	type	css=div:nth-child(1) > .px-3	Aqua
9	click	css=div:nth-child(2) > .px-3	
10	select	css=div:nth-child(2) > .px-3	label=Nước uống
11	click	css=div:nth-child(3) > .px-3	
12	type	css=div:nth-child(3) > .px-3	1000
13	click	css=.disabled\3Aopacity-50:nth-child(2)	
14	assert text	css=.go2072408551	Tạo dịch vụ thành công!
15	close		

Các bước thực hiện	Kết quả mong muốn	Kết quả
1. Mở trang quản lý dịch vụ 2. Ấn thêm mới 3. Nhập thông tin dịch vụ 4. Ấn Lưu 5. Kiểm tra thông báo	Thêm bản ghi thành công	Đạt

Test case ID: VS-09

Test case: Cập nhật dịch vụ

	Command	Target	Value
1	open	/	
2	set window size	945x1012	
3	run	Đăng nhập Admin nhà xe	
4	click	css=div:nth-child(4) > .w-full	
5	click	linkText=Dịch vụ bổ sung	
6	click	css=.border-t:nth-child(1) .text-blue-600	
7	click	css=div:nth-child(3) > .px-3	
8	type	css=div:nth-child(3) > .px-3	10000
9	click	css=.disabled\3Aopacity-50:nth-child(2)	
10	assert text	css=.go2072408551	Cập nhật dịch vụ thành công!
11	close		

Các bước thực hiện	Kết quả mong muốn	Kết quả
1. Mở trang quản lý dịch vụ 2. Ấn Sửa 3. Nhập thông tin dịch vụ 4. Ấn Lưu 5. Kiểm tra thông báo	Cập nhật bản ghi thành công	Đạt

Test case ID: VS-10

Test case: Xóa dịch vụ thành công

	Command	Target	Value
1	open	/	
2	set window size	945x1012	
3	run	Đăng nhập Admin nhà xe	
4	click	css=div:nth-child(4) > .w-full	
5	click	linkText=Dịch vụ bổ sung	
6	click	css=.border-t:nth-child(1) .text-red-600	
7	click	css=.bg-red-600	
8	assert text	css=.go2072408551	Xóa dịch vụ thành công!
9	close		

Các bước thực hiện	Kết quả mong muốn	Kết quả
1. Mở trang quản lý dịch vụ 2. Ấn Xóa 3. Ấn Xác nhận xóa 4. Kiểm tra thông báo	Xóa thành công	Đạt

3.1.2.9. Kiểm thử trang Quản lý đánh giá

Test case ID: RV-04

Test case: Nhập thông tin và tìm kiếm đánh giá

	Command	Target	Value
1	open	/	
2	set window size	945x1012	
3	run	Đăng nhập Admin nhà xe	
4	click	linkText=Đánh giá	
5	click	css=.sm\:3A flex-1	
6	type	css=.sm\:3A flex-1	BK202510220008
7	click	css=.shrink-0	
8	close		

Các bước thực hiện	Kết quả mong muốn	Kết quả
1. Mở trang quản lý đánh giá 2. Nhập thông tin tìm kiếm 3. Ấn tìm kiếm 4. Kiểm tra bảng dữ liệu	Tìm kiếm thành công, bảng dữ liệu hiển thị tương ứng với thông tin tìm kiếm	Đạt

Test case ID: RV-06

Test case: Xóa đánh giá

	Command	Target	Value
1	open	/	
2	set window size	945x1012	
3	run	Đăng nhập Admin nhà xe	
4	click	linkText=Đánh giá	
5	click	css=.lucide-trash2	
6	click	css=.bg-red-600	
7	assert text	css=.go3958317564	Xóa đánh giá thành công!
8	close		

Các bước thực hiện	Kết quả mong muốn	Kết quả
1. Mở trang quản lý đánh giá 2. Xóa đánh giá 3. Xác nhận xóa 4. Kiểm tra thông báo	Xóa thành công	Đạt

3.1.2.10. Kiểm thử trang Quản lý thông báo

Test case ID: NT-06

Test case: Đánh dấu đã đọc tất cả thông báo

	Command	Target	Value
1	open	/	
2	set window size	945x1012	
3	run	Đăng nhập Admin nhà xe	
4	click	css=.flex:nth-child(6)	
5	click	css=.bg-green-600	
6	assert text	css=.go3958317564	Đã đánh dấu tất cả thông báo là đã đọc
7	close		

Các bước thực hiện	Kết quả mong muốn	Kết quả
1. Mở trang thông báo 2. Ấn nút Đánh dấu đã đọc 3. Kiểm tra thông báo	Đánh dấu đã đọc thành công	Đạt

Test case ID: NT-07

Test case: Tải lại thông báo

	Command	Target	Value
1	open	/	
2	set window size	945x1012	
3	run	Đăng nhập Admin nhà xe	
4	click	css=.flex:nth-child(6)	
5	click	css=.bg-blue-600	
6	assert text	css=.go3958317564	Đã tải lại thông báo
7	close		

Các bước thực hiện	Kết quả mong muốn	Kết quả
--------------------	-------------------	---------

1. Mở trang thông báo 2. Ấn nút Tải lại 3. Kiểm tra thông báo	Dữ liệu được tải lại thành công	Đạt
---	---------------------------------	-----

3.1.2.11. Kiểm thử trang Quản lý đặt vé

Test case ID: AB-08

Test case: Ấn xem chi tiết thông tin vé hiển thị

	Command	Target	Value
1	<i>open</i>	/	
2	<i>set window size</i>	945x1012	
3	<i>run</i>	Đăng nhập Admin nhà xe	
4	<i>click</i>	linkText=Quản lý đặt vé	
5	<i>click</i>	css=.hover\3A bg-gray-50:nth-child(1) path	
6	<i>run script</i>	window.scrollTo(0,0)	
7	<i>assert text</i>	css=.text-gray-800	Chi tiết đặt vé
8	<i>close</i>		

Các bước thực hiện	Kết quả mong muốn	Kết quả
1. Mở trang quản lý đặt vé 2. Ấn xem chi tiết vé 3. Kiểm tra thông tin chi tiết vé	Thông tin chi tiết vé hiển thị thành công	Đạt

Test case ID: AB-09

Test case: Xác nhận thanh toán cho vé

	Command	Target	Value
1	<i>open</i>	/	
2	<i>set window size</i>	945x1012	
3	<i>run</i>	Đăng nhập Admin nhà xe	
4	<i>click</i>	linkText=Quản lý đặt vé	
5	<i>click</i>	css=.sm\3A flex-1	
6	<i>type</i>	css=.sm\3A flex-1	BK202510220008
7	<i>click</i>	css=.shrink-0	
8	<i>click</i>	css=.lucide-check-circle	
9	<i>click</i>	css=.bg-yellow-600	
10	<i>assert text</i>	css=.go3958317564	Xác nhận thanh toán thành công!
11	<i>close</i>		

Các bước thực hiện	Kết quả mong muốn	Kết quả
1. Mở trang quản lý đặt vé 2. Ấn xác nhận thanh toán cho vé 3. Kiểm tra thông báo sau khi xác nhận	Xác nhận thanh toán thành công	Đạt

Test case ID: AB-11
Test case: Hủy vé thành công

	Command	Target	Value
1	open	/	
2	set window size	945x1012	
3	run	Đăng nhập Admin nhà xe	
4	click	linkText=Quản lý đặt vé	
5	click	css=.sm\3A flex-1	
6	type	css=.sm\3A flex-1	BK202604140004
7	click	css=.shrink-0	
8	click	css=.lucide-x	
9	click	css=.bg-red-600	
10	assert text	css=.go3958317564	Hủy vé thành công!
11	close		

Các bước thực hiện	Kết quả mong muốn	Kết quả
1. Mở trang quản lý đặt vé 2. Chọn vé 3. Ấn hủy vé 4. Kiểm tra thông báo hủy vé	Hủy vé thành công	Đạt

3.2. Kiểm thử sử dụng Selenium WebDriver

3.2.1. Kiểm thử phía khách hàng

3.2.1.1. Kiểm thử trang Trang chủ

Test case ID: HM-01
Test case: Trang chủ hiển thị thành công

```
@Test(description = "HM-01 Trang chủ hiển thị thành công")
public void case_HM_001() {
    driver.get(Config.getBaseUrl() + "/");
    driver.manage().window().setSize(new Dimension(945, 1012));
    Assert.assertTrue(wait
        .until(ExpectedConditions
            .presenceOfElementLocated(By.cssSelector("[data-testid='customer-home-heading']")))
        .getText().contains("Đặt vé xe khách trực tuyến"));
}
```

Các bước thực hiện	Kết quả mong muốn	Kết quả
1. Mở trang chủ 2. Kiểm tra trang chủ	Trang chủ hiển thị được thành công	Đạt

Test case ID: HM-07
Test case: Tìm kiếm chuyến xe trên trang chủ

```

@Test(description = "HM-07 Ấn tìm kiếm chuyển sang trang tìm kiếm")
public void case_HM_007() {
    driver.get(Config.getBaseUrl() + "/");
    driver.manage().window().setSize(new Dimension(945, 1012));
    wait.until(ExpectedConditions.elementToBeClickable(By.id("departureLocationId"))).click();
    new Select(wait.until(ExpectedConditions.presenceOfElementLocated(By.id("departureLocationId"))))
        .selectByVisibleText("Hà Nội - Long Biên");
    wait.until(ExpectedConditions.elementToBeClickable(By.id("arrivalLocationId"))).click();
    new Select(wait.until(ExpectedConditions.presenceOfElementLocated(By.id("arrivalLocationId"))))
        .selectByVisibleText("Hải Phòng - Lê Chân");
    wait.until(ExpectedConditions.elementToBeClickable(By.id("departureDate"))).click();
    wait.until(ExpectedConditions.elementToBeClickable(By.cssSelector(".react-datepicker__day--016"))).click();
    wait.until(ExpectedConditions.elementToBeClickable(By.cssSelector("[data-testid='home-search-submit']"))).click();
    wait.until(ExpectedConditions.titleIs("Tìm chuyến xe - BookingHub"));
    Assert.assertEquals(driver.getTitle(), "Tìm chuyến xe - BookingHub");
}

```

Các bước thực hiện	Kết quả mong muốn	Kết quả
1. Nhập thông tin tìm kiếm 2. Nhấn nút Tìm kiếm 3. Chuyển sang trang tìm kiếm chuyến xe	Chuyển sang trang tìm kiếm chuyến xe và hiển thị danh sách chuyến nếu có	Đạt

Test case ID: HM-10

Test case: Tìm kiếm lỗi khi chưa nhập thông tin

```

@Test(description = "HM-10 Bấm tìm kiếm khi chưa chọn thông tin")
public void case_HM_010() {
    driver.get(Config.getBaseUrl() + "/");
    driver.manage().window().setSize(new Dimension(945, 1012));
    wait.until(ExpectedConditions.elementToBeClickable(By.cssSelector("[data-testid='home-search-submit']"))).click();
    Assert.assertFalse(
        driver.findElements(By.xpath("//*[contains(normalize-space(.),\"Vui lòng điền đầy đủ thông tin\")]]"))
            .isEmpty());
}

```

Các bước thực hiện	Kết quả mong muốn	Kết quả
1. Mở trang chủ 2. Để trống các ô chọn 3. Nhấn Tìm kiếm 4. Kiểm tra thông báo lỗi xuất hiện	Trang chủ hiển thị thông báo lỗi "Vui lòng nhập đầy đủ thông tin"	Đạt

3.2.1.2. Kiểm thử trang Đăng nhập

Test case ID: LG-01

Test case: Kiểm tra trang đăng nhập hiển thị giao diện form đăng nhập

```

@Test(description = "LG-01 Trang login hiển thị form đăng nhập")
public void case_LG_001() {
    driver.get(Config.getBaseUrl() + "/");
    driver.manage().window().setSize(new Dimension(945, 1012));

    logoutBefore();

    wait.until(ExpectedConditions.elementToBeClickable(By.cssSelector(".btn-login"))).click();
    Assert.assertFalse(driver.findElements(By.cssSelector(".text-3x1")).isEmpty(), "Missing element");
}

```

Các bước thực hiện	Kết quả mong muốn	Kết quả
1. Nhấn Đăng nhập	Form đăng nhập hiển thị thành công	Đạt
2. Kiểm tra form đăng nhập		

Test case ID: LG-10

Test case: Không nhập thông tin đăng nhập

```

@Test(description = "LG-10 Gửi thông tin form trống vẫn ở trang login và báo bắt buộc nhập")
public void case_LG_010() {
    driver.get(Config.getBaseUrl() + "/");
    driver.manage().window().setSize(new Dimension(945, 1012));
    logoutBefore();

    wait.until(ExpectedConditions.elementToBeClickable(By.cssSelector(".btn-login"))).click();
    wait.until(ExpectedConditions.visibilityOfElementLocated(By.id("usernameOrEmail")));
    wait.until(ExpectedConditions.visibilityOfElementLocated(By.id("password")));
    Assert.assertFalse(driver.findElements(By.id("usernameOrEmail")).isEmpty(), "Missing element");
    Assert.assertFalse(driver.findElements(By.id("password")).isEmpty(), "Missing element");
    wait.until(ExpectedConditions.elementToBeClickable(By.cssSelector("[data-testid='login-submit']"))).click();
    WebElement username = driver.findElement(By.id("usernameOrEmail"));

    Boolean isValid = (Boolean) ((JavascriptExecutor) driver)
        .executeScript("return arguments[0].checkValidity();", username);

    Assert.assertFalse(isValid);
}

```

Các bước thực hiện	Kết quả mong muốn	Kết quả
1. Mở trang đăng nhập	Hiển thị thông báo nhập dữ liệu ở trường bắt buộc	Đạt
2. Không nhập thông tin		
3. Ấn đăng nhập		

Test case ID: LG-11

Test case: Đăng nhập thành công

```

@Test(description = "LG-11 Đăng nhập đúng tài khoản rời khỏi trang login")
public void case_LG_011() {
    driver.get(Config.getBaseUrl() + "/");
    driver.manage().window().setSize(new Dimension(945, 1012));
    logoutBefore();

    wait.until(ExpectedConditions.elementToBeClickable(By.cssSelector(".btn-login"))).click();
    wait.until(ExpectedConditions.visibilityOfElementLocated(By.id("usernameOrEmail"))).clear();
    wait.until(ExpectedConditions.visibilityOfElementLocated(By.id("usernameOrEmail"))).sendKeys("customer");
    wait.until(ExpectedConditions.visibilityOfElementLocated(By.id("password"))).clear();
    wait.until(ExpectedConditions.visibilityOfElementLocated(By.id("password"))).sendKeys("123456aA@");
    wait.until(ExpectedConditions.elementToBeClickable(By.cssSelector("[data-testid='login-submit']"))).click();
    WebElement toast = wait.until(ExpectedConditions.presenceOfElementLocated(
        By.xpath("//*[contains(normalize-space(.),'Đăng nhập thành công!']")
    ));

    Assert.assertTrue(toast.getText().contains("Đăng nhập thành công!"));
}

```

Các bước thực hiện	Kết quả mong muốn	Kết quả
1. Mở trang đăng nhập 2. Nhập tài khoản đúng 3. Ấn đăng nhập	Hiện thị thông báo đăng nhập thành công	Đạt

3.2.1.3. Kiểm thử trang Đăng ký

Test case ID: RG-04

Test case: Kiểm tra hiển thị các trường nhập liệu trang đăng ký

}

```
@Test(description = "RG-04 Trang đăng ký hiển thị đủ thông tin")
public void case_RG_004() {
    goToRegisterPage();

    wait.until(ExpectedConditions.visibilityOfElementLocated(By.id("username")));
    wait.until(ExpectedConditions.visibilityOfElementLocated(By.id("email")));
    wait.until(ExpectedConditions.visibilityOfElementLocated(By.id("password")));
    wait.until(ExpectedConditions.visibilityOfElementLocated(By.id("name")));
    wait.until(ExpectedConditions.visibilityOfElementLocated(By.id("phone")));

    Assert.assertTrue(true);
}
```

Các bước thực hiện	Kết quả mong muốn	Kết quả
1. Vào trang đăng ký 2. Kiểm tra các ô nhập liệu	Các ô nhập liệu hiển thị đủ	Đạt

Test case ID: RG-07

Test case: Không nhập thông tin ấn đăng ký

```
@Test(description = "RG-07 Gửi thông tin trống form vẫn hiển thị và thông báo nhập các trường bắt buộc")
public void case_RG_007() {
    goToRegisterPage();

    wait.until(ExpectedConditions.elementToBeClickable(By.cssSelector("[data-testid='register-submit']"))).click();

    WebElement username = wait.until(ExpectedConditions.visibilityOfElementLocated(By.id("username")));

    Boolean isValid = (Boolean) ((JavascriptExecutor) driver).executeScript("return arguments[0].checkValidity();",
        username);

    Assert.assertFalse(isValid);
}
```

Các bước thực hiện	Kết quả mong muốn	Kết quả
1. Vào trang đăng ký 2. Ấn đăng ký	Thông báo bắt buộc nhập	Đạt

Test case ID: RG-10

Test case: Kiểm tra trùng Username

```

@Test(description = "RG-10 Kiểm tra trùng username nếu đã tồn tại")
public void case_RG_010() {
    goToRegisterPage();

    WebElement username = wait.until(ExpectedConditions.visibilityOfElementLocated(By.id("username")));
    username.clear();
    username.sendKeys("teest");

    WebElement password = wait.until(ExpectedConditions.visibilityOfElementLocated(By.id("password")));
    password.clear();
    password.sendKeys("123456");

    WebElement name = wait.until(ExpectedConditions.visibilityOfElementLocated(By.id("name")));
    name.clear();
    name.sendKeys("Nguyễn Văn B");

    WebElement phone = wait.until(ExpectedConditions.visibilityOfElementLocated(By.id("phone")));
    phone.clear();
    phone.sendKeys("0333333333");

    WebElement email = wait.until(ExpectedConditions.visibilityOfElementLocated(By.id("email")));
    email.clear();
    email.sendKeys("a@gmail.com");

    wait.until(ExpectedConditions.elementToBeClickable(By.cssSelector("[data-testid='register-submit']"))).click();

    waitForToast("Username đã tồn tại");
}

```

Các bước thực hiện	Kết quả mong muốn	Kết quả
1. Vào trang đăng ký 2. Nhập username đã tồn tại 3. Ấn đăng ký	Thông báo đã tồn tại username	Đạt

Test case ID: RG-13

Test case: Đăng ký thành công

```

@Test(description = "RG-13 Nhập đúng thông tin đăng ký thành công")
public void case_RG_013() {
    goToRegisterPage();

    WebElement username = wait.until(ExpectedConditions.visibilityOfElementLocated(By.id("username")));
    username.clear();
    username.sendKeys("teest11111");

    WebElement password = wait.until(ExpectedConditions.visibilityOfElementLocated(By.id("password")));
    password.clear();
    password.sendKeys("123456");

    WebElement name = wait.until(ExpectedConditions.visibilityOfElementLocated(By.id("name")));
    name.clear();
    name.sendKeys("Nguyễn Văn B");

    WebElement phone = wait.until(ExpectedConditions.visibilityOfElementLocated(By.id("phone")));
    phone.clear();
    phone.sendKeys("0333333334");

    WebElement email = wait.until(ExpectedConditions.visibilityOfElementLocated(By.id("email")));
    email.clear();
    email.sendKeys("abdc@gmail.com");

    wait.until(ExpectedConditions.elementToBeClickable(
        By.cssSelector("[data-testid='register-submit']")
    )).click();

    waitForToast("Đăng ký thành công!");
}

```

Các bước thực hiện	Kết quả mong muốn	Kết quả
1. Vào trang đăng ký	Thông báo đăng ký thành công	Đạt

2. Nhập thông tin hợp lệ		
3. Ấn đăng ký		

3.2.1.4. Kiểm thử trang Tìm chuyến

Test case ID: SR-05

Test case: Kiểm tra hiển thị các trường nhập liệu trang tìm chuyến

```
@Test(description = "SR-05 Form tìm kiếm có đủ trường dữ liệu")
Run | Debug
public void case_SR_005() {
    goToSearchPage();

    wait.until(ExpectedConditions.visibilityOfElementLocated(By.id("search-departureLocationId")));
    wait.until(ExpectedConditions.visibilityOfElementLocated(By.id("search-arrivalLocationId")));
    wait.until(ExpectedConditions.visibilityOfElementLocated(By.id("search-departureDate")));

    Assert.assertTrue(true);
}
```

Các bước thực hiện	Kết quả mong muốn	Kết quả
1. Vào trang tìm chuyến	Hiển thị đủ điểm đi, điểm đến, ngày đi	Đạt
2. Kiểm tra form tìm chuyến		

Test case ID: SR-09

Test case: Chỉ chọn điểm đi và ấn tìm kiếm

```
@Test(description = "SR-09 Chỉ chọn điểm đi rồi tìm kiếm hiển thị lỗi thiếu thông tin")
Run | Debug
public void case_SR_009() {
    goToSearchPage();

    new Select(wait.until(ExpectedConditions.visibilityOfElementLocated(By.id("search-departureLocationId"))))
        .selectByVisibleText("Hà Nội - Hoàng Mai");

    wait.until(ExpectedConditions.elementToBeClickable(By.cssSelector(".px-6"))).click();

    waitForError("Vui lòng điền đầy đủ thông tin tìm kiếm");
}
```

Các bước thực hiện	Kết quả mong muốn	Kết quả
1. Vào trang tìm chuyến	Thông báo lỗi Vui lòng điền đầy đủ thông tin tìm kiếm	Đạt
2. Chọn điểm đi		
3. Ấn tìm chuyến		

Test case ID: SR-12

Test case: Tìm kiếm chuyến thành công

```

@Test(description = "SR-12 Nhập đủ thông tin tìm kiếm thành công")
Run / Debug
public void case_SR_012() {
    goToSearchPage();

    new Select(wait.until(ExpectedConditions.visibilityOfElementLocated(By.id("search-departureLocationId"))))
        .selectByVisibleText("Hà Nội - Long Biên");

    new Select(wait.until(ExpectedConditions.visibilityOfElementLocated(By.id("search-arrivalLocationId"))))
        .selectByVisibleText("Hải Phòng - Lê Chân");

    wait.until(ExpectedConditions.elementToBeClickable(By.id("search-departureDate"))).click();

    wait.until(ExpectedConditions.elementToBeClickable(By.cssSelector(".react-datepicker__day--015"))).click();

    wait.until(ExpectedConditions.elementToBeClickable(By.cssSelector(".px-6"))).click();

    wait.until(ExpectedConditions.titleContains("Tìm chuyến xe"));
}

```

Các bước thực hiện	Kết quả mong muốn	Kết quả
1. Vào trang tìm chuyến 2. Nhập đủ thông tin tìm chuyến 3. Ấn tìm chuyến	Hiển thị danh sách chuyến phù hợp với thông tin tìm chuyến	Đạt

3.2.1.5. Kiểm thử trang Đặt vé

Test case ID: BK-01

Test case: Kiểm tra hiển thị trang đặt vé

```

@Test(description = "BK-01 Trang đặt vé hiển thị và có tiêu đề")
Run / Debug
public void case_BK_001() {
    loginCustomer();
    searchTrip();
    openBooking();

    Assert.assertTrue(driver.findElement(By.xpath("//h1[contains(., 'Đặt vé xe khách')]")).getText()
        .contains("Đặt vé xe khách"));
}

```

Các bước thực hiện	Kết quả mong muốn	Kết quả
1.Tìm kiếm chuyến xe 2. Chọn chuyến xe 3. Kiểm tra trang đặt vé hiển thị và có tiêu đề	Trang đặt vé hiển thị và có tiêu đề	Đạt

Test case ID: BK-04

Test case: Kiểm tra các trường thông tin liên hệ


```

@Test(description = "BK-04 Form có hiển thị các trường thông tin")
Run / Debug
public void case_BK_004() {
    loginCustomer();
    searchTrip();
    openBooking();

    Assert.assertTrue(
        wait.until(ExpectedConditions.visibilityOfElementLocated(By.name("customerName"))).isDisplayed());

    Assert.assertTrue(
        wait.until(ExpectedConditions.visibilityOfElementLocated(By.name("customerPhone"))).isDisplayed());

    Assert.assertTrue(
        wait.until(ExpectedConditions.visibilityOfElementLocated(By.name("customerEmail"))).isDisplayed());

    Assert.assertTrue(
        wait.until(ExpectedConditions.visibilityOfElementLocated(By.name("paymentMethod"))).isDisplayed());
}

```

Các bước thực hiện	Kết quả mong muốn	Kết quả
1. Tìm kiếm chuyến xe 2. Chọn chuyến xe 3. Kiểm tra trang đặt vé 4. Trang đặt vé hiển thị đủ các trường thông tin liên hệ	Trang đặt vé hiển thị đủ các trường thông tin liên hệ	Đạt

Test case ID: BK-08

Test case: Kiểm tra nút đặt vé

```

148
149 @Test(description = "BK-08 Nút đặt vé hiển thị")
Run / Debug
150 public void case_BK_008() {
151     loginCustomer();
152     searchTrip();
153     openBooking();
154
155     By btn = By.cssSelector("[data-testid='booking-submit']");
156
157     Assert.assertTrue(
158         wait.until(ExpectedConditions.visibilityOfElementLocated(btn)).getText().contains("Vui lòng chọn ghế"));
159
160 }
161

```

Các bước thực hiện	Kết quả mong muốn	Kết quả
1. Tìm kiếm chuyến xe 2. Chọn chuyến xe 3. Kiểm tra nút đặt vé	Nút đặt vé hiển thị Vui lòng chọn ghế khi chưa chọn ghế	Đạt

Test case ID: BK-09

Test case: Đặt vé thành công

```

@Test(description = "BK-09 Đặt vé thành công")
Run / Debug
public void case_BK_009() {
    loginCustomer();
    searchTrip();
    openBooking();

    wait.until(ExpectedConditions.elementToBeClickable(By.cssSelector(".grid:nth-child(3) .p-2"))).click();

    wait.until(ExpectedConditions.elementToBeClickable(By.cssSelector(".px-6"))).click();

    Assert.assertTrue(wait.until(ExpectedConditions.visibilityOfElementLocated(By.cssSelector(".mb-4 > .text-2xl"))).getText().contains("Đặt vé thành công"));
}

```

Các bước thực hiện	Kết quả mong muốn	Kết quả
1. Tìm kiếm chuyến xe 2. Chọn chuyến xe 3. Nhập đầy đủ thông tin	Hiển thị thông báo Đặt vé thành công	Đạt

4. Nhấn nút Đặt vé 5. Kiểm tra thông đặt vé thành công		
---	--	--

3.2.1.6. Kiểm thử trang Thanh toán

Test case ID: PM-01

Test case: Kiểm tra hiển thị trang thanh toán

```
@Test(description = "PM-01 Trang thanh toán hiển thị")
Run / Debug
public void case_PM_001() {
    loginCustomer();

    openPaymentPage();

    WebElement heading = wait.until(ExpectedConditions
        .visibilityOfElementLocated(By.cssSelector("[data-testid='customer-payment-heading']")));

    Assert.assertTrue(heading.getText().contains("Thanh toán"));
}
```

Các bước thực hiện	Kết quả mong muốn	Kết quả
1. Đăng nhập 2. Vào mục Vé của tôi 3. Ấn nút Thanh toán 4. Kiểm tra trang thanh toán hiển thị	Trang thanh toán hiển thị thành công	Đạt

Test case ID: PM-07

Test case: Kiểm tra các phương thức thanh toán

```
@Test(description = "PM-07 Hiển thị phương thức thanh toán")
Run / Debug
public void case_PM_007() {
    loginCustomer();
    openPaymentPage();

    By paymentMethod = By.cssSelector("[data-testid='customer-payment-methods']");

    WebElement el = wait.until(ExpectedConditions.visibilityOfElementLocated(paymentMethod));
    Assert.assertTrue(el.isDisplayed());
}
```

Các bước thực hiện	Kết quả mong muốn	Kết quả
1. Đăng nhập 2. Vào mục Vé của tôi 3. Ấn nút Thanh toán 4. Kiểm tra phương thức thanh toán hiển thị	Phương thức thanh toán hiển thị thành công	Đạt

Test case ID: PM-10

Test case: Thanh toán trả tiền mặt khi lên xe

```

@Test(description = "PM-10 Thanh toán bằng phương thức tiền mặt và Xác nhận thanh toán thành công")
Run | Debug
public void case_PM_010() {
    loginCustomer();
    openPaymentPage();

    By confirmBtn = By.cssSelector("[data-testid='customer-payment-confirm']");

    WebElement btn = wait.until(ExpectedConditions.elementToBeClickable(confirmBtn));
    btn.click();

    // Assert.assertFalse(driver
    // .findElements(By.xpath(
    // "//*[contains(normalize-space(.),\nĐặt vé thành công! Vui lòng thanh toán khi lên xe\n)])"))
    // .isEmpty());
    waitForToast("Đặt vé thành công! Vui lòng thanh toán khi lên xe");
}

```

Các bước thực hiện	Kết quả mong muốn	Kết quả
1. Đăng nhập 2. Vào mục Vé của tôi 3. Ấn nút Thanh toán 4. Chọn phương thức tiền mặt 5. Ấn xác nhận thanh toán	Hiện thị thông báo "Đặt vé thành công! Vui lòng thanh toán khi lên xe"	Đạt

3.2.1.7. Kiểm thử trang Vé của tôi

Test case ID: MB-01

Test case: Kiểm tra hiển thị trang vé của tôi

```

@Test(description = "MB-01 Trang Vé của tôi hiển thị đúng")
Run | Debug
public void case_MB_001() {
    loginCustomer();
    openMyBookingsPage();

    WebElement heading = wait.until(ExpectedConditions
        .visibilityOfElementLocated(By.cssSelector("[data-testid='customer-my-bookings-heading']")));

    Assert.assertTrue(heading.getText().contains("Vé của tôi"));
}

```

Các bước thực hiện	Kết quả mong muốn	Kết quả
1. Đăng nhập 2. Vào mục Vé của tôi 3. Kiểm tra trang	Trang vé của tôi hiển thị	Đạt

Test case ID: MB-08

Test case: Thực hiện hủy vé

```

@Test(description = "MB-08 Hủy vé thành công nếu có nút hủy vé")
Run | Debug
public void case_MB_008() {
    loginCustomer();
    openMyBookingsPage();

    wait.until(ExpectedConditions.elementToBeClickable(
        By.xpath("//div[.//span[contains(., 'Chờ thanh toán')]]//button[contains(., 'Hủy')][1]"))).click();
    wait.until(ExpectedConditions.elementToBeClickable(By.xpath("//button[contains(., 'Xác nhận hủy')]))).click();
    waitForToast("Hủy vé thành công!");
}

```

Các bước thực hiện	Kết quả mong muốn	Kết quả
1. Đăng nhập 2. Vào mục Vé của tôi 3. Nhấn nút Hủy vé	Vé được hủy thành công	Đạt

4. Vé được hủy thành công		
---------------------------	--	--

Test case ID: MB-10

Test case: Xem chi tiết vé

```
@Test(description = "MB-10 Xem chi tiết của vé")
Run | Debug
public void case_MB_010() {
    loginCustomer();
    openMyBookingsPage();

    Assert.assertFalse(driver.findElements(By.cssSelector(".bg-white:nth-child(1) .w-full:nth-child(1)")).isEmpty(),
        "Missing element");
    Assert.assertFalse(
        driver.findElements(By.xpath("//*[contains(normalize-space(.),\"Xem chi tiết\")]")).isEmpty());
    Assert.assertTrue(wait.until(ExpectedConditions.presenceOfElementLocated(By.cssSelector(".sticky > .text-2xl"))))
        .getText().contains("Chi tiết vé");
}
```

Các bước thực hiện	Kết quả mong muốn	Kết quả
1. Đăng nhập 2. Vào mục Vé của tôi 3. Ấn nút Xem chi tiết 4. Hiện thị thông tin chi tiết	Chi tiết vé hiển thị thành công	Đạt

Test case ID: MB-14

Test case: Ấn nút thanh toán chuyển qua trang thanh toán

```
@Test(description = "MB-14 Ấn nút thanh toán hiển thị trang thanh toán")
Run | Debug
public void case_MB_014() {
    loginCustomer();
    openMyBookingsPage();

    wait.until(ExpectedConditions.visibilityOfElementLocated(
        By.xpath("(//div[//span[contains(.,'Chờ thanh toán')]]//button[contains(.,'Thanh toán')])[1]")));
    Assert.assertFalse(driver
        .findElements(
            By.xpath("(//div[//span[contains(.,'Chờ thanh toán')]]//button[contains(.,'Thanh toán')])[1]"))
        .isEmpty(), "Missing element");
    wait.until(ExpectedConditions.elementToBeClickable(
        By.xpath("(//div[//span[contains(.,'Chờ thanh toán')]]//button[contains(.,'Thanh toán')])[1]")))
        .click();

    wait.until(ExpectedConditions
        .visibilityOfElementLocated(By.cssSelector("[data-testid='customer-payment-heading']")));
}
```

Các bước thực hiện	Kết quả mong muốn	Kết quả
1. Đăng nhập 2. Vào mục Vé của tôi 3. Ấn nút Thanh toán 4. Kiểm tra trang thanh toán hiển thị	Điều hướng sang trang thanh toán	Đạt

3.2.1.8. Kiểm thử trang Hồ sơ

Test case ID: PF-01

Test case: Kiểm tra hiển thị trang hồ sơ

```

@Test(description = "PF-01 Trang thông tin cá nhân hiển thị sau khi đăng nhập")
Run | Debug
public void case_PF_001() {
    loginCustomer();
    openProfilePage();

    Assert.assertEquals(driver.getTitle(), "Thông tin cá nhân - BookingHub");
}

```

Các bước thực hiện	Kết quả mong muốn	Kết quả
1. Đăng nhập 2. Vào trang thông tin cá nhân	Trang thông tin cá nhân hiển thị thành công	Đạt

Test case ID: PF-08

Test case: Thay đổi Họ tên

```

@Test(description = "PF-08 Có thể điền lại họ tên và lưu lại")
Run | Debug
public void case_PF_008() {
    loginCustomer();
    openProfilePage();

    wait.until(ExpectedConditions.elementToBeClickable(By.name("name"))).click();
    wait.until(ExpectedConditions.visibilityOfElementLocated(By.name("name"))).clear();
    wait.until(ExpectedConditions.visibilityOfElementLocated(By.name("name"))).sendKeys("Nguyễn Văn B");
    wait.until(ExpectedConditions.elementToBeClickable(By.cssSelector("[data-testid='customer-profile-submit']")))
        .click();
    waitForToast("Cập nhật profile thành công!");
}

```

Các bước thực hiện	Kết quả mong muốn	Kết quả
1. Đăng nhập 2. Vào trang thông tin cá nhân 3. Nhập họ tên mới 4. Lưu thay đổi	Thông tin được lưu thành công	Đạt

Test case ID: PF-09

Test case: Thay đổi email

```

@Test(description = "PF-09 Có thể điền lại email và lưu lại")
Run | Debug
public void case_PF_009() {
    loginCustomer();
    openProfilePage();

    wait.until(ExpectedConditions.elementToBeClickable(By.name("email"))).click();
    wait.until(ExpectedConditions.visibilityOfElementLocated(By.name("email"))).clear();
    wait.until(ExpectedConditions.visibilityOfElementLocated(By.name("email"))).sendKeys("customer1@gmail.com");
    wait.until(ExpectedConditions.elementToBeClickable(By.cssSelector("[data-testid='customer-profile-submit']")))
        .click();
    waitForToast("Cập nhật profile thành công!");
}

```

Các bước thực hiện	Kết quả mong muốn	Kết quả
1. Đăng nhập 2. Vào trang thông tin cá nhân 3. Nhập email mới 4. Lưu thay đổi	Thông tin được lưu thành công	Đạt

Test case ID: PF-13

Test case: Kiểm tra lỗi để trống SĐT

```

@Test(description = "PF-13 Để trống thông tin SĐT thông báo lỗi")
Run / Debug
public void case_PF_013() {
    loginCustomer();
    openProfilePage();

    wait.until(ExpectedConditions.elementToBeClickable(By.name("phone"))).click();
    wait.until(ExpectedConditions.elementToBeClickable(By.name("phone"))).click();
    new Actions(driver).doubleClick(wait.until(ExpectedConditions.presenceOfElementLocated(By.name("phone"))))
        .perform();
    wait.until(ExpectedConditions.visibilityOfElementLocated(By.name("phone"))).clear();
    wait.until(ExpectedConditions.visibilityOfElementLocated(By.name("phone"))).sendKeys("");
    wait.until(ExpectedConditions.elementToBeClickable(By.cssSelector("[data-testid='customer-profile-submit']")));
    WebElement phone = wait.until(ExpectedConditions.presenceOfElementLocated(By.name("phone")));

    Boolean isValid = (Boolean) ((JavascriptExecutor) driver).executeScript("return arguments[0].checkValidity();",
        phone);

    Assert.assertFalse(isValid);
}

```

Các bước thực hiện	Kết quả mong muốn	Kết quả
1. Đăng nhập 2. Vào trang thông tin cá nhân 3. Xóa SĐT 4. Lưu thay đổi	Thông báo bắt buộc nhập	Đạt

3.2.2. Kiểm thử phía quản trị viên

3.2.2.1. Kiểm thử trang Đăng nhập

Test case ID: AL-01

Test case: Kiểm tra hiển thị đăng nhập

```

@Test(description = "AL-01 Trang admin login hiển thị form đăng nhập")
Run / Debug
public void case_AL_001() {
    openAdmin();
    logoutAdmin();

    Assert.assertTrue(driver.findElement(By.id("admin-usernameOrEmail")).isDisplayed());
    Assert.assertTrue(driver.findElement(By.id("admin-password")).isDisplayed());
}

```

Các bước thực hiện	Kết quả mong muốn	Kết quả
1. Mở trang đăng nhập 2. Kiểm tra form đăng nhập	Form đăng nhập hiển thị đủ	Đạt

Test case ID: AL-03

Test case: Nhập sai tài khoản mật khẩu

```

}

@Test(description = "AL-03 Đăng nhập sai báo lỗi sai thông tin")
Run / Debug
public void case_AL_003() {
    openAdmin();
    logoutAdmin();

    wait.until(ExpectedConditions.visibilityOfElementLocated(By.id("admin-usernameOrEmail"))).sendKeys("wrong");
    wait.until(ExpectedConditions.visibilityOfElementLocated(By.id("admin-password"))).sendKeys("wrong");

    wait.until(ExpectedConditions.elementToBeClickable(By.cssSelector("[data-testid='admin-login-submit']")))
        .click();

    Assert.assertFalse(driver
        .findElements(By.xpath("//*[contains(normalize-space(.),'Sai tài khoản hoặc mật khẩu']")).isEmpty());
}

```

Các bước thực hiện	Kết quả mong muốn	Kết quả
1. Mở trang đăng nhập 2. Nhập thông tin đăng nhập sai 3. Kiểm tra thông báo	Thông báo sai tài khoản hoặc mật khẩu	Không đạt

Test case ID: AL-04

Test case: Đăng nhập đúng thông báo đăng nhập thành công

```
@Test(description = "AL-04 Đăng nhập đúng thông báo đăng nhập thành công")
Run | Debug
public void case_AL_004() {
    openAdmin();
    logoutAdmin();

    wait.until(ExpectedConditions.visibilityOfElementLocated(By.id("admin-usernameOrEmail"))).sendKeys("admin");
    wait.until(ExpectedConditions.visibilityOfElementLocated(By.id("admin-password"))).sendKeys("123456aA@");

    wait.until(ExpectedConditions.elementToBeClickable(By.cssSelector("[data-testid='admin-login-submit']"))).click();

    Assert.assertTrue(wait.until(ExpectedConditions
        .visibilityOfElementLocated(By.xpath("//*[contains(normalize-space(.),'Đăng nhập thành công')]"))
        .isDisplayed()));
}
```

Các bước thực hiện	Kết quả mong muốn	Kết quả
1. Mở trang đăng nhập 2. Nhập thông tin đúng 3. Kiểm tra thông báo	Đăng nhập thành công	Đạt

3.2.2.2. Kiểm thử trang Dashboad

Test case ID: AD-01

Test case: Kiểm tra hiển thị trang dashboard

```
@Test(description = "AD-01 Dashboard hiển thị sau khi đăng nhập")
Run | Debug
public void case_AD_001() {
    loginAdmin();

    wait.until(ExpectedConditions.titleIs("Dashboard - BookingHub Admin"));
    Assert.assertEquals(driver.getTitle(), "Dashboard - BookingHub Admin");
}
```

Các bước thực hiện	Kết quả mong muốn	Kết quả
1. Mở trang Dashboard 2. Kiểm tra trang	Trang hiển thị thành công	Đạt

Test case ID: AD-04

Test case: Kiểm tra hiển thị thông kê đặt vé

```
@Test(description = "AD-04 Kiểm tra hiển thị thông tin Thống kê đặt vé")
Run | Debug
public void case_AD_004() {
    loginAdmin();

    Assert.assertTrue(wait
        .until(ExpectedConditions
            .visibilityOfElementLocated(By.xpath("//*[contains(normalize-space(.),'Thống kê đặt vé')]"))
            .isDisplayed()));
}
```

Các bước thực hiện	Kết quả mong muốn	Kết quả
--------------------	-------------------	---------

1. Mở trang Dashboard 2. Kiểm tra hiển thị Thông kê đặt vé	Thông tin thống kê đặt vé hiển thị đúng	Đạt
---	---	-----

3.2.2.3. Kiểm thử trang Quản lý địa điểm

Test case ID: LC-01

Test case: Kiểm tra hiển thị quản lý địa điểm

```
@Test(description = "LC-01 Kiểm tra hiển thị trang quản lý địa điểm")
Run | Debug
public void case_LC_001() {
    loginSuperAdmin();
    openLocationPage();

    Assert.assertEquals(driver.getTitle(), "Quản lý Địa điểm - BookingHub");
}
```

Các bước thực hiện	Kết quả mong muốn	Kết quả
1. Mở trang quản lý địa điểm 2. Kiểm tra trang	Trang hiển thị thành công	Đạt

Test case ID: LC-07

Test case: Nhập đủ dữ liệu và thêm địa điểm mới

```
@Test(description = "LC-07 Nhập đầy đủ thông tin và lưu địa điểm")
Run | Debug
public void case_LC_007() {
    loginSuperAdmin();
    openLocationPage();

    wait.until(ExpectedConditions.elementToBeClickable(By.cssSelector("[data-testid='admin-locations-btn-add']")))
        .click();

    wait.until(ExpectedConditions.visibilityOfElementLocated(By.name("city"))).sendKeys("Thành phố test");

    driver.findElement(By.name("district")).sendKeys("Test");
    driver.findElement(By.id("latitude")).sendKeys("21.028222");
    driver.findElement(By.id("longitude")).sendKeys("105.88888");

    By saveBtn = By.cssSelector("[data-testid='admin-locations-form-submit']");

    wait.until(ExpectedConditions.visibilityOfElementLocated(saveBtn));
    wait.until(ExpectedConditions.elementToBeClickable(saveBtn)).click();

    WebElement toast = wait.until(ExpectedConditions
        .visibilityOfElementLocated(By.xpath("//*[contains(normalize-space(.),'Tạo địa điểm thành công')]")));

    Assert.assertTrue(toast.isDisplayed());
}
```

Các bước thực hiện	Kết quả mong muốn	Kết quả
1. Mở trang quản lý địa điểm 2. Ấn thêm địa điểm 3. Nhập đầy đủ thông tin 4. Ấn lưu lại 5. Kiểm tra thông báo	Thêm bản ghi thành công	Đạt

Test case ID: LC-08

Test case: Chỉnh sửa địa điểm


```

@Test(description = "LC-08 Chỉnh sửa địa điểm")
Run / Debug
public void case_LC_008() {
    loginSuperAdmin();
    openLocationPage();

    wait.until(
        ExpectedConditions.elementToBeClickable(By.cssSelector("[data-testid='admin-locations-btn-edit-']")))
        .click();

    WebElement district = wait.until(ExpectedConditions.visibilityOfElementLocated(By.name("district")));
    district.clear();
    district.sendKeys("Test1");

    By saveBtn = By.cssSelector("[data-testid='admin-locations-form-submit']");

    wait.until(ExpectedConditions.visibilityOfElementLocated(saveBtn));
    wait.until(ExpectedConditions.elementToBeClickable(saveBtn)).click();

    WebElement toast = wait.until(ExpectedConditions
        .visibilityOfElementLocated(By.xpath("//*[contains(normalize-space(.),'Cập nhật thành công']"))));

    Assert.assertTrue(toast.isDisplayed());
}

```

Các bước thực hiện	Kết quả mong muốn	Kết quả
1. Mở trang quản lý địa điểm 2. Ấn sửa địa điểm 3. Chỉnh sửa thông tin 4. Ấn lưu lại 5. Kiểm tra thông báo	Cập nhật bản ghi thành công	Đạt

Test case ID: LC-09
Test case: Xóa địa điểm

```

213
214 @Test(description = "LC-09 Xóa địa điểm")
Run / Debug
215 public void case_LC_009() {
216     loginSuperAdmin();
217     openLocationPage();
218
219     wait.until(
220         ExpectedConditions.elementToBeClickable(By.cssSelector("[data-testid='admin-locations-btn-delete-']")))
221         .click();
222
223     wait.until(ExpectedConditions.elementToBeClickable(By.cssSelector("[data-testid='confirm-modal-confirm']")))
224         .click();
225
226     WebElement toast = wait.until(ExpectedConditions
227         .visibilityOfElementLocated(By.xpath("//*[contains(normalize-space(.),'Xóa thành công']"))));
228
229     Assert.assertTrue(toast.isDisplayed());
230
231 }
232

```

Các bước thực hiện	Kết quả mong muốn	Kết quả
1. Mở trang quản lý địa điểm 2. Ấn xóa địa điểm 3. Xác nhận xóa 4. Kiểm tra thông báo	Xóa thành công, bảng dữ liệu không hiển thị bản ghi vừa xóa	Đạt

3.2.2.4. Kiểm thử trang Quản lý nhà xe

Test case ID: BC-01

Test case: Kiểm tra hiển thị quản lý nhà xe

```

@Test(description = "BC-01 Trang Quản lý Nhà xe hiển thị")
Run | Debug
public void case_BC_001() {
    loginSuperAdmin();
    openBusCompaniesPage();
    Assert.assertEquals(driver.getTitle(), "Quản lý Nhà xe - BookingHub");
}

```

Các bước thực hiện	Kết quả mong muốn	Kết quả
1. Mở trang quản lý nhà xe	Trang quản lý nhà xe hiển thị thành công	Đạt
2. Kiểm tra trang quản lý nhà xe		

Test case ID: BC-06

Test case: Nhập đủ dữ liệu và thêm nhà xe mới

```

@Test(description = "BC-06 Thêm nhà xe nhập đầy đủ thông tin và lưu lại")
Run | Debug
public void case_BC_006() {
    loginSuperAdmin();
    openBusCompaniesPage();
    wait.until(ExpectedConditions.visibilityOfElementLocated(By.cssSelector("[data-testid='admin-bus-companies-btn-add']")));
    wait.until(ExpectedConditions.elementToBeClickable(By.cssSelector("[data-testid='admin-bus-companies-btn-add']"))).click();
    wait.until(ExpectedConditions.elementToBeClickable(By.name("name"))).click();
    wait.until(ExpectedConditions.visibilityOfElementLocated(By.name("name"))).clear();
    wait.until(ExpectedConditions.visibilityOfElementLocated(By.name("name"))).sendKeys("Test");
    wait.until(ExpectedConditions.visibilityOfElementLocated(By.name("email"))).clear();
    wait.until(ExpectedConditions.visibilityOfElementLocated(By.name("email"))).sendKeys("ackckkc@test.com");
    wait.until(ExpectedConditions.visibilityOfElementLocated(By.name("phone"))).clear();
    wait.until(ExpectedConditions.visibilityOfElementLocated(By.name("phone"))).sendKeys("0333333333");
    wait.until(ExpectedConditions.visibilityOfElementLocated(By.name("address"))).clear();
    wait.until(ExpectedConditions.visibilityOfElementLocated(By.name("address"))).sendKeys("Test");
    wait.until(ExpectedConditions.elementToBeClickable(By.cssSelector("[data-testid='admin-bus-companies-form-submit']"))).click();
    Assert.assertFalse(driver.findElements(By.xpath("//*[contains(normalize-space(.), 'Tạo nhà xe thành công!')]")).isEmpty());
}

```

Các bước thực hiện	Kết quả mong muốn	Kết quả
1. Mở trang quản lý nhà xe	Thêm bản ghi thành công	Đạt
2. Ấn nút thêm nhà xe		
3. Nhập đầy đủ thông tin		
4. Ấn lưu lại		

Test case ID: BC-07

Test case: Cập nhật nhà xe

```

@Test(description = "BC-07 Cập nhật nhà xe")
Run | Debug
public void case_BC_007() {
    loginSuperAdmin();
    openBusCompaniesPage();
    wait.until(ExpectedConditions.elementToBeClickable(By.cssSelector("[data-testid='admin-bus-companies-btn-edit']"))).click();
    wait.until(ExpectedConditions.elementToBeClickable(By.name("name"))).click();
    wait.until(ExpectedConditions.visibilityOfElementLocated(By.name("name"))).clear();
    wait.until(ExpectedConditions.visibilityOfElementLocated(By.name("name"))).sendKeys("Test1");
    wait.until(ExpectedConditions.elementToBeClickable(By.xpath("//*[button[contains(., 'Lưu') or contains(., 'Cập nhật') or contains(., 'Thêm') or contains(., 'Tạo')]]"))).click();
    Assert.assertFalse(driver.findElements(By.xpath("//*[contains(normalize-space(.), 'Cập nhật thành công!')]")).isEmpty());
}

```

Các bước thực hiện	Kết quả mong muốn	Kết quả
1. Mở trang quản lý nhà xe	Cập nhật bản ghi thành công	Đạt

2. Ấn nút cập nhật nhà xe		
3. Thay đổi thông tin nhà xe		
4. Ấn lưu lại		

Test case ID: BC-08

Test case: Xóa nhà xe

```
@Test(description = "BC-08 Xóa nhà xe")
Run | Debug
public void case_BC_008() {
    loginSuperAdmin();
    openBusCompaniesPage();
    wait.until(ExpectedConditions
        .elementToBeClickable(By.cssSelector("[data-testid='admin-bus-companies-btn-delete-']"))).click();
    wait.until(ExpectedConditions.elementToBeClickable(By.cssSelector("[data-testid='confirm-modal-confirm']")))
        .click();
    Assert.assertFalse(
        driver.findElements(By.xpath("//*[contains(normalize-space(.),\"Xóa thành công!\"]")).isEmpty());
}
```

Các bước thực hiện	Kết quả mong muốn	Kết quả
1. Mở trang quản lý nhà xe	Xóa thành công	Đạt
2. Ấn nút xóa nhà xe		
3. Xác nhận xóa		
4. Kiểm tra thông báo		

Test case ID: BC-11

Test case: Tìm kiếm nhà xe theo từ khóa

```
@Test(description = "BC-11 Tìm kiếm nhà xe theo từ khóa")
Run | Debug
public void case_BC_011() {
    loginSuperAdmin();
    openBusCompaniesPage();
    wait.until(ExpectedConditions.elementToBeClickable(By.cssSelector(".p-6"))).click();
    wait.until(ExpectedConditions
        .elementToBeClickable(By.cssSelector("[data-testid='admin-bus-companies-search-input']"))).click();
    wait.until(ExpectedConditions
        .visibilityOfElementLocated(By.cssSelector("[data-testid='admin-bus-companies-search-input']")))
        .clear();
    wait.until(ExpectedConditions
        .visibilityOfElementLocated(By.cssSelector("[data-testid='admin-bus-companies-search-input']")))
        .sendKeys("Test");
    wait.until(ExpectedConditions
        .elementToBeClickable(By.cssSelector("[data-testid='admin-bus-companies-search-submit']"))).click();
}
```

Các bước thực hiện	Kết quả mong muốn	Kết quả
1. Mở trang quản lý nhà xe	Tìm kiếm thành công, bảng dữ liệu hiển thị tương ứng với thông tin tìm kiếm	Đạt
2. Nhập thông tin tìm kiếm		
3. Ấn tìm kiếm		
4. Kiểm tra bảng dữ liệu		

3.2.2.5. Kiểm thử trang Quản lý phương tiện

Test case ID: VH-10

Test case: Thêm mới phương tiện thành công

```

@Test(description = "VH-10 Thêm mới phương tiện thành công")
Run | Debug
public void case_VH_010() {
    loginAdmin();
    openVehiclesPage();
    wait.until(ExpectedConditions.elementToBeClickable(By.cssSelector("[data-testid='admin-vehicles-btn-add']")))
        .click();
    wait.until(ExpectedConditions.elementToBeClickable(By.name("busName"))).click();
    wait.until(ExpectedConditions.visibilityOfElementLocated(By.name("busName"))).clear();
    wait.until(ExpectedConditions.visibilityOfElementLocated(By.name("busName"))).sendKeys("Xe 16 chỗ");
    wait.until(ExpectedConditions.elementToBeClickable(By.name("busType"))).click();
    new Select(wait.until(ExpectedConditions.presenceOfElementLocated(By.name("busType"))))
        .selectByVisibleText("Limousine");
    wait.until(ExpectedConditions.elementToBeClickable(By.name("licensePlate"))).click();
    wait.until(ExpectedConditions.visibilityOfElementLocated(By.name("licensePlate"))).clear();
    wait.until(ExpectedConditions.visibilityOfElementLocated(By.name("licensePlate"))).sendKeys("30A12345");
    wait.until(ExpectedConditions.elementToBeClickable(By.name("layoutTemplateId"))).click();
    new Select(wait.until(ExpectedConditions.presenceOfElementLocated(By.name("layoutTemplateId"))))
        .selectByVisibleText("Xe 16 chỗ (16 chỗ)");
    wait.until(ExpectedConditions.elementToBeClickable(By.name("totalSeats"))).click();
    wait.until(ExpectedConditions.visibilityOfElementLocated(By.name("totalSeats"))).clear();
    wait.until(ExpectedConditions.visibilityOfElementLocated(By.name("totalSeats"))).sendKeys("16");
    By submitBtn = By.cssSelector("[data-testid='admin-vehicles-form-submit']");

    wait.until(ExpectedConditions.elementToBeClickable(submitBtn)).click();
    Assert.assertFalse(
        driver.findElements(By.xpath("//*[contains(normalize-space(.),\"Tạo xe thành công!\")]")).isEmpty());
}

```

Các bước thực hiện	Kết quả mong muốn	Kết quả
1. Mở trang quản lý phương tiện 2. Ấn thêm mới 3. Nhập thông tin 4. Ấn Lưu 5. Kiểm tra thông báo	Thêm bản ghi thành công	Đạt

Test case ID: VH-11

Test case: Chỉnh sửa phương tiện thành công

```

@Test(description = "VH-11 Chỉnh sửa phương tiện thành công")
Run | Debug
public void case_VH_011() {
    loginAdmin();
    openVehiclesPage();
    wait.until(ExpectedConditions.elementToBeClickable(By.cssSelector("[data-testid='admin-vehicles-btn-edit-']")))
        .click();
    wait.until(ExpectedConditions.elementToBeClickable(By.name("busName"))).click();
    wait.until(ExpectedConditions.visibilityOfElementLocated(By.name("busName"))).clear();
    wait.until(ExpectedConditions.visibilityOfElementLocated(By.name("busName"))).sendKeys("Xe 16 chỗ 1");
    By submitBtn = By.cssSelector("[data-testid='admin-vehicles-form-submit']");

    wait.until(ExpectedConditions.elementToBeClickable(submitBtn)).click();
    Assert.assertFalse(
        driver.findElements(By.xpath("//*[contains(normalize-space(.),\"Cập nhật thành công!\")]")).isEmpty());
}

```

Các bước thực hiện	Kết quả mong muốn	Kết quả
1. Mở trang quản lý phương tiện 2. Ấn sửa 3. Thay đổi thông tin 4. Ấn Lưu 5. Kiểm tra thông báo	Cập nhật bản ghi thành công	Đạt

Test case ID: VH-12

Test case: Xóa phương tiện

```

@Test(description = "VH-12 Xóa phương tiện")
Run | Debug
public void case_VH_012() {
    loginAdmin();
    openVehiclesPage();
    wait.until(
        ExpectedConditions.elementToBeClickable(By.cssSelector("[data-testid='admin-vehicles-btn-delete-']")))
        .click();
    wait.until(ExpectedConditions.elementToBeClickable(By.cssSelector("[data-testid='confirm-modal-confirm']")))
        .click();
    Assert.assertFalse(
        driver.findElements(By.xpath("//*[contains(normalize-space(.),\"Xóa thành công!\"]")).isEmpty());
    }

```

Các bước thực hiện	Kết quả mong muốn	Kết quả
1. Mở trang quản lý phương tiện 2. Ấn xóa 3. Ấn Xác nhận xóa 4. Kiểm tra thông báo	Xóa thành công	Đạt

3.2.2.6. Kiểm thử trang quản lý chuyển xe

Test case ID: TR-15

Test case: Xóa chuyển xe

```

@Test(description = "TR-15 Xóa chuyển xe")
Run | Debug
public void case_TR_015() {
    loginAdmin();
    openTripsPage();
    wait.until(ExpectedConditions.elementToBeClickable(By.cssSelector("[data-testid='admin-trips-btn-delete-']")))
        .click();
    wait.until(ExpectedConditions.elementToBeClickable(By.cssSelector("[data-testid='confirm-modal-confirm']")))
        .click();
    Assert.assertFalse(
        driver.findElements(By.xpath("//*[contains(normalize-space(.),\"Xóa thành công!\"]")).isEmpty());
    }

```

Các bước thực hiện	Kết quả mong muốn	Kết quả
1. Mở trang quản lý chuyển xe 2. Xóa chuyển xe 3. Xác nhận xóa 4. Kiểm tra thông báo	Xóa thành công	Không đạt

Test case ID: TR-16

Test case: Sửa chuyển xe thành công

```

@Test(description = "TR-16 Sửa chuyển xe thành công")
Run | Debug
public void case_TR_016() {
    loginAdmin();
    openTripsPage();
    wait.until(ExpectedConditions.elementToBeClickable(By.xpath("//*[contains(@data-testid, 'btn-edit-')][1]")))
        .click();
    wait.until(ExpectedConditions.elementToBeClickable(By.name("departureTime"))).click();
    wait.until(ExpectedConditions.elementToBeClickable(By.name("departureTime"))).click();
    wait.until(ExpectedConditions.elementToBeClickable(By.name("departureTime"))).click();
    wait.until(ExpectedConditions.visibilityOfElementLocated(By.name("departureTime"))).clear();
    wait.until(ExpectedConditions.visibilityOfElementLocated(By.name("departureTime")))
        .sendKeys("2026-04-29T09:00");
    By submitBtn = By.cssSelector("[data-testid='admin-trips-form-submit']");

    wait.until(ExpectedConditions.elementToBeClickable(submitBtn)).click();
    Assert.assertFalse(
        driver.findElements(By.xpath("//*[contains(normalize-space(.),\"Cập nhật thành công!\"]")).isEmpty());
    }

```

Các bước thực hiện	Kết quả mong muốn	Kết quả
--------------------	-------------------	---------

1. Mở trang quản lý chuyến xe 2. Ấn nút sửa chuyến 3. Nhập thông tin thay đổi 4. Ấn Lưu 5. Kiểm tra thông báo	Cập nhật bản ghi thành công	Đạt
---	-----------------------------	-----

Test case ID: TR-17

Test case: Tạo chuyến mới thành công

```
@Test(description = "TR-17 Tạo chuyến mới thành công")
Run | Debug
public void case_TR_017() {
    loginAdmin();
    openTripsPage();
    wait.until(
        ExpectedConditions.presenceOfElementLocated(By.cssSelector("[data-testid='admin-trips-btn-create']")));
    wait.until(ExpectedConditions.elementToBeClickable(By.cssSelector("[data-testid='admin-trips-btn-create']")))
        .click();
    wait.until(ExpectedConditions.elementToBeClickable(By.name("vehicleId"))).click();
    new Select(wait.until(ExpectedConditions.presenceOfElementLocated(By.name("vehicleId"))))
        .selectByVisibleText("Xe 16 (Ghế ngồi)");
    wait.until(ExpectedConditions.elementToBeClickable(By.name("departureLocationId"))).click();
    new Select(wait.until(ExpectedConditions.presenceOfElementLocated(By.name("departureLocationId"))))
        .selectByVisibleText("Hồ Chí Minh - Bình Tân");
    wait.until(ExpectedConditions.elementToBeClickable(By.name("arrivalLocationId"))).click();
    new Select(wait.until(ExpectedConditions.presenceOfElementLocated(By.name("arrivalLocationId"))))
        .selectByVisibleText("Đà Nẵng - Thanh Khê");
    wait.until(ExpectedConditions.elementToBeClickable(By.id("bulkCreate"))).click();
    wait.until(ExpectedConditions
        .elementToBeClickable(By.cssSelector(".react-datepicker-wrapper:nth-child(2) .w-full"))).click();
    wait.until(ExpectedConditions
        .elementToBeClickable(By.cssSelector(".react-datepicker__time-list-item:nth-child(99)"))).click();
    wait.until(ExpectedConditions.elementToBeClickable(By.cssSelector("div:nth-child(1) > .relative .text-sm")))
        .click();
    wait.until(ExpectedConditions.elementToBeClickable(By.cssSelector(".react-datepicker__day--016"))).click();
    wait.until(ExpectedConditions.elementToBeClickable(By.cssSelector("div:nth-child(2) > .relative .w-full")))
        .click();
    wait.until(ExpectedConditions.elementToBeClickable(By.cssSelector(".react-datepicker__day--019"))).click();
    wait.until(ExpectedConditions.elementToBeClickable(By.cssSelector(".w-full:nth-child(3)"))).click();
    wait.until(ExpectedConditions.elementToBeClickable(By.cssSelector(".disabled\\3Aopacity-50"))).click();
    By submitBtn = By.cssSelector("[data-testid='admin-trip-create-submit']");

    wait.until(ExpectedConditions.elementToBeClickable(submitBtn)).click();
    Assert.assertFalse(driver.findElements(By.xpath("//*[contains(normalize-space(.),\"Tạo chuyến thành công!\")]"))
        .isEmpty());
}
```

Các bước thực hiện	Kết quả mong muốn	Kết quả
1. Mở trang quản lý chuyến xe 2. Ấn nút tạo chuyến 3. Nhập thông tin chuyến 4. Ấn tạo chuyến 5. Kiểm tra thông báo	Tạo chuyến thành công	Đạt

3.2.2.7. Kiểm thử trang Quản lý người dùng

Test case ID: US-07

Test case: Thêm người dùng mới thành công


```

@Test(description = "US-07 Thêm người dùng mới thành công")
Run | Debug
public void case_US_007() {
    loginAdmin();
    openUsersPage();
    wait.until(ExpectedConditions.elementToBeClickable(By.cssSelector("[data-testid='admin-users-btn-add']")))
        .click();
    Assert.assertTrue(
        wait.until(ExpectedConditions.presenceOfElementLocated(By.cssSelector(".bg-white > .text-xl")))
            .getText().contains("Thêm người dùng"));
    wait.until(ExpectedConditions.elementToBeClickable(By.cssSelector("div:nth-child(1) > .px-3")))
        .click();
    wait.until(ExpectedConditions.visibilityOfElementLocated(By.cssSelector("div:nth-child(1) > .px-3")))
        .clear();
    wait.until(ExpectedConditions.visibilityOfElementLocated(By.cssSelector("div:nth-child(1) > .px-3")))
        .sendKeys("test");
    wait.until(ExpectedConditions.elementToBeClickable(By.cssSelector("div:nth-child(2) > .border")))
        .click();
    wait.until(ExpectedConditions.visibilityOfElementLocated(By.cssSelector("div:nth-child(2) > .border")))
        .clear();
    wait.until(ExpectedConditions.visibilityOfElementLocated(By.cssSelector("div:nth-child(2) > .border")))
        .sendKeys("test@gmail.com");
    wait.until(ExpectedConditions.elementToBeClickable(By.cssSelector("div:nth-child(3) > .px-3")))
        .click();
    wait.until(ExpectedConditions.visibilityOfElementLocated(By.cssSelector("div:nth-child(3) > .px-3")))
        .clear();
    wait.until(ExpectedConditions.visibilityOfElementLocated(By.cssSelector("div:nth-child(3) > .px-3")))
        .sendKeys("test");
    wait.until(ExpectedConditions.elementToBeClickable(By.cssSelector("div:nth-child(4) > .border")))
        .click();
    new Select(
        wait.until(ExpectedConditions.presenceOfElementLocated(By.cssSelector("div:nth-child(4) > .border")))
            .selectByVisibleText("company_admin"));
    wait.until(ExpectedConditions.elementToBeClickable(By.cssSelector("div:nth-child(6) > .w-full")))
        .click();
    wait.until(ExpectedConditions.visibilityOfElementLocated(By.cssSelector("div:nth-child(6) > .w-full")))
        .clear();
    wait.until(ExpectedConditions.visibilityOfElementLocated(By.cssSelector("div:nth-child(6) > .w-full")))
        .sendKeys("123456aA@");
    wait.until(ExpectedConditions.elementToBeClickable(By.cssSelector(".px-4:nth-child(2)")))
        .click();
    By submitBtn = By.cssSelector("[data-testid='admin-users-btn-save']");
    wait.until(ExpectedConditions.elementToBeClickable(submitBtn))
        .click();
    Assert.assertFalse(driver
        .findElements(By.xpath("//*[contains(normalize-space(.),\"Tạo người dùng thành công!\")]")).isEmpty());
}

```

Các bước thực hiện	Kết quả mong muốn	Kết quả
- Mở trang quản lý người dùng - Ấn nút thêm người dùng - Nhập thông tin người dùng - Ấn Lưu - Kiểm tra thông báo	Thêm bản ghi thành công	Đạt

Test case ID: US-08

Test case: Cập nhật người dùng

```

@Test(description = "US-08 Sửa người dùng")
Run | Debug
public void case_US_008() {
    loginAdmin();
    openUsersPage();
    wait.until(ExpectedConditions
        .elementToBeClickable(By.cssSelector(".border-t:nth-child(1) .hover\\3A bg-blue-50 > .lucide")))
        .click();
    wait.until(ExpectedConditions.elementToBeClickable(By.cssSelector("div:nth-child(3) > .px-3")))
        .click();
    wait.until(ExpectedConditions.visibilityOfElementLocated(By.cssSelector("div:nth-child(3) > .px-3")))
        .clear();
    wait.until(ExpectedConditions.visibilityOfElementLocated(By.cssSelector("div:nth-child(3) > .px-3")))
        .sendKeys("Admin11");
    // wait.until(ExpectedConditions.elementToBeClickable(By.cssSelector(".px-4:nth-child(2)")))
    //     .click();
    By submitBtn = By.cssSelector("[data-testid='admin-users-btn-save']");
    wait.until(ExpectedConditions.elementToBeClickable(submitBtn))
        .click();
    Assert.assertFalse(
        driver.findElements(By.xpath("//*[contains(normalize-space(.),\"Cập nhật người dùng thành công!\")]")).isEmpty());
}

```

Các bước thực hiện	Kết quả mong muốn	Kết quả
- Mở trang quản lý người dùng - Ấn nút sửa - Nhập thông tin sửa - Ấn Lưu - Kiểm tra thông báo	Cập nhật bản ghi thành công	Đạt

Test case ID: US-09
Test case: Xóa người dùng

```
@Test(description = "US-09 Xóa người dùng")
Run | Debug
public void case_US_009() {
    loginAdmin();
    openUsersPage();
    wait.until(ExpectedConditions.elementToBeClickable(By.cssSelector("[data-testid='admin-users-btn-delete-']")))
        .click();
    wait.until(ExpectedConditions.elementToBeClickable(By.cssSelector("[data-testid='confirm-modal-confirm']")))
        .click();
    Assert.assertFalse(driver
        .findElements(By.xpath("//*[contains(normalize-space(.),\"Xóa người dùng thành công!\")]")).isEmpty());
}
```

Các bước thực hiện	Kết quả mong muốn	Kết quả
1. Mở trang quản lý người dùng 2. Xóa người dùng 3. Xác nhận xóa 4. Kiểm tra thông báo	Xóa thành công	Đạt

3.2.2.8. Kiểm thử trang Quản lý dịch vụ

Test case ID: VS-08
Test case: Nhập đủ dữ liệu và thêm dịch vụ mới

```
@Test(description = "VS-08 Thêm mới dịch vụ thành công")
Run | Debug
public void case_VS_008() {
    loginAdmin();
    openVasPage();
    wait.until(ExpectedConditions.elementToBeClickable(By.cssSelector("[data-testid='admin-vas-btn-add']")))
        .click();
    wait.until(ExpectedConditions.elementToBeClickable(By.cssSelector("div:nth-child(1) > .px-3")))
        .click();
    wait.until(ExpectedConditions.visibilityOfElementLocated(By.cssSelector("div:nth-child(1) > .px-3")))
        .clear();
    wait.until(ExpectedConditions.visibilityOfElementLocated(By.cssSelector("div:nth-child(1) > .px-3")))
        .sendKeys("Aqua");
    wait.until(ExpectedConditions.elementToBeClickable(By.cssSelector("div:nth-child(2) > .px-3")))
        .click();
    new Select(wait.until(ExpectedConditions.presenceOfElementLocated(By.cssSelector("div:nth-child(2) > .px-3"))))
        .selectByVisibleText("Nước uống");
    wait.until(ExpectedConditions.elementToBeClickable(By.cssSelector("div:nth-child(3) > .px-3")))
        .click();
    wait.until(ExpectedConditions.visibilityOfElementLocated(By.cssSelector("div:nth-child(3) > .px-3")))
        .clear();
    wait.until(ExpectedConditions.visibilityOfElementLocated(By.cssSelector("div:nth-child(3) > .px-3")))
        .sendKeys("1000");
    wait.until(ExpectedConditions.elementToBeClickable(By.cssSelector(".disabled\\3Aopacity-50:nth-child(2)")))
        .click();
    Assert.assertFalse(
        driver.findElements(By.xpath("//*[contains(normalize-space(.),\"Tạo dịch vụ thành công!\")]")).isEmpty());
}
```

Các bước thực hiện	Kết quả mong muốn	Kết quả
1. Mở trang quản lý dịch vụ 2. Ấn thêm mới 3. Nhập thông tin dịch vụ 4. Ấn Lưu 5. Kiểm tra thông báo	Thêm bản ghi thành công	Đạt

Test case ID: VS-09
Test case: Cập nhật dịch vụ


```

@Test(description = "VS-09 Chinh sửa dịch vụ")
Run | Debug
public void case_VS_009() {
    loginAdmin();
    openVasPage();
    wait.until(ExpectedConditions.elementToBeClickable(By.cssSelector(".border-t:nth-child(1) .text-blue-600")))
    // .click();
    wait.until(ExpectedConditions.elementToBeClickable(By.cssSelector("[data-testid='admin-vas-btn-edit-']")))
    // .click();
    wait.until(ExpectedConditions.elementToBeClickable(By.cssSelector("div:nth-child(3) > .px-3"))).click();
    wait.until(ExpectedConditions.visibilityOfElementLocated(By.cssSelector("div:nth-child(3) > .px-3"))).clear();
    wait.until(ExpectedConditions.visibilityOfElementLocated(By.cssSelector("div:nth-child(3) > .px-3")))
    .sendKeys("10000");
    wait.until(ExpectedConditions.elementToBeClickable(By.cssSelector(".disabled\\3Aopacity-50:nth-child(2)")))
    .click();
    Assert.assertFalse(
        driver.findElements(By.xpath("//*[contains(normalize-space(.),\"Cập nhật dịch vụ thành công!\"]"))
        .isEmpty());
}

```

Các bước thực hiện	Kết quả mong muốn	Kết quả
1. Mở trang quản lý dịch vụ 2. Ấn Sửa 3. Nhập thông tin dịch vụ 4. Ấn Lưu 5. Kiểm tra thông báo	Cập nhật bản ghi thành công	Đạt

Test case ID: VS-10

Test case: Xóa dịch vụ thành công

```

@Test(description = "VS-10 Xóa dịch vụ thành công")
Run | Debug
public void case_VS_010() {
    loginAdmin();
    openVasPage();
    wait.until(ExpectedConditions.elementToBeClickable(By.cssSelector(".border-t:nth-child(1) .text-red-600")))
    .click();
    wait.until(ExpectedConditions.elementToBeClickable(By.cssSelector("[data-testid='confirm-modal-confirm']")))
    .click();
    Assert.assertFalse(
        driver.findElements(By.xpath("//*[contains(normalize-space(.),\"Xóa dịch vụ thành công!\"]"))
        .isEmpty());
}

```

Các bước thực hiện	Kết quả mong muốn	Kết quả
1. Mở trang quản lý dịch vụ 2. Ấn Xóa 3. Ấn Xác nhận xóa 4. Kiểm tra thông báo	Xóa thành công	Đạt

3.2.2.9. Kiểm thử trang Quản lý đánh giá

Test case ID: RV-04

Test case: Nhập thông tin và tìm kiếm đánh giá

```

@Test(description = "RV-04 Nhập thông tin và tìm kiếm đánh giá")
Run | Debug
public void case_RV_004() {
    loginAdmin();
    openReviewsPage();
    wait.until(
        ExpectedConditions.elementToBeClickable(By.cssSelector("[data-testid='admin-reviews-search-input']")))
    .click();
    wait.until(ExpectedConditions
        .visibilityOfElementLocated(By.cssSelector("[data-testid='admin-reviews-search-input']")))
    .clear();
    wait.until(ExpectedConditions
        .visibilityOfElementLocated(By.cssSelector("[data-testid='admin-reviews-search-input']")))
    .sendKeys("BK202510220008");
    wait.until(
        ExpectedConditions.elementToBeClickable(By.cssSelector("[data-testid='admin-reviews-search-submit']")))
    .click();
}

```

Các bước thực hiện	Kết quả mong muốn	Kết quả
1. Mở trang quản lý đánh giá 2. Nhập thông tin tìm kiếm 3. Ấn tìm kiếm 4. Kiểm tra bảng dữ liệu	Tìm kiếm thành công, bảng dữ liệu hiển thị tương ứng với thông tin tìm kiếm	Đạt

Test case ID: RV-06

Test case: Xóa đánh giá

```
@Test(description = "RV-06 Xóa đánh giá")
Run | Debug
public void case_RV_006() {
    loginAdmin();
    openReviewsPage();
    wait.until(ExpectedConditions.elementToBeClickable(By.cssSelector(".lucide-trash2"))).click();
    wait.until(ExpectedConditions.elementToBeClickable(By.cssSelector("[data-testid='confirm-modal-confirm']"))).click();
    Assert.assertFalse(driver
        .findElements(By.xpath("//*[contains(normalize-space(.),\"Xóa đánh giá thành công!\")]")).isEmpty());
}
```

Các bước thực hiện	Kết quả mong muốn	Kết quả
1. Mở trang quản lý đánh giá 2. Xóa đánh giá 3. Xác nhận xóa 4. Kiểm tra thông báo	Xóa thành công	Đạt

3.2.2.10. Kiểm thử trang Quản lý thông báo

Test case ID: NT-06

Test case: Đánh dấu đã đọc tất cả thông báo

```
@Test(description = "NT-06 Đánh dấu tất cả là đã đọc thông báo")
Run | Debug
public void case_NT_006() {
    loginAdmin();
    openNotificationsPage();
    wait.until(ExpectedConditions.elementToBeClickable(By.cssSelector(".bg-green-600"))).click();
    Assert.assertFalse(driver
        .findElements(By.xpath("//*[contains(normalize-space(.),\"Đã đánh dấu tất cả thông báo là đã đọc!\")]")).isEmpty());
}
```

Các bước thực hiện	Kết quả mong muốn	Kết quả
1. Mở trang thông báo 2. Ấn nút Đánh dấu đã đọc 3. Kiểm tra thông báo	Đánh dấu đã đọc thành công	Đạt

Test case ID: NT-07

Test case: Tải lại thông báo

```
@Test(description = "NT-07 Tải lại trang thông báo")
Run | Debug
public void case_NT_007() {
    loginAdmin();
    openNotificationsPage();
    By refreshBtn = By.cssSelector("[data-testid='admin-notifications-btn-refresh']");

    wait.until(ExpectedConditions.elementToBeClickable(refreshBtn)).click();
    Assert.assertFalse(
        driver.findElements(By.xpath("//*[contains(normalize-space(.),\"Đã tải lại thông báo!\")]")).isEmpty());
}
```

Các bước thực hiện	Kết quả mong muốn	Kết quả
1. Mở trang thông báo 2. Ấn nút Tải lại 3. Kiểm tra thông báo	Dữ liệu được tải lại thành công	Đạt

3.2.2.11. Kiểm thử trang Quản lý đặt vé

Test case ID: AB-08

Test case: Ấn xem chi tiết thông tin vé hiển thị

```
@Test(description = "AB-08 Ấn xem chi tiết thông tin vé hiển thị")
Run | Debug
public void case_AB_008() {
    loginAdmin();
    openBookingsPage();
    wait.until(ExpectedConditions.elementToBeClickable(By.cssSelector(".hover\\3A bg-gray-50:nth-child(1) path"))).click();
    Assert.assertTrue(wait.until(ExpectedConditions.presenceOfElementLocated(By.cssSelector(".text-gray-800"))).getText().contains("Chi tiết đặt vé"));
}
```

Các bước thực hiện	Kết quả mong muốn	Kết quả
1. Mở trang quản lý đặt vé 2. Ấn xem chi tiết vé 3. Kiểm tra thông tin chi tiết vé	Thông tin chi tiết vé hiển thị thành công	Đạt

Test case ID: AB-09

Test case: Xác nhận thanh toán cho vé

```
@Test(description = "AB-09 Xác nhận thanh toán cho vé")
Run | Debug
public void case_AB_009() {
    loginAdmin();
    openBookingsPage();
    wait.until(
        ExpectedConditions.elementToBeClickable(By.cssSelector("[data-testid='admin-bookings-search-input']")))
        .click();
    wait.until(ExpectedConditions
        .visibilityOfElementLocated(By.cssSelector("[data-testid='admin-bookings-search-input']")))
        .clear();
    wait.until(ExpectedConditions
        .visibilityOfElementLocated(By.cssSelector("[data-testid='admin-bookings-search-input']")))
        .sendKeys("BK202510220008");
    wait.until(
        ExpectedConditions.elementToBeClickable(By.cssSelector("[data-testid='admin-bookings-search-submit']")))
        .click();
    wait.until(ExpectedConditions.elementToBeClickable(By.cssSelector(".lucide-check-circle"))).click();
    wait.until(ExpectedConditions.elementToBeClickable(By.cssSelector(".bg-yellow-600"))).click();
    Assert.assertFalse(
        driver.findElements(By.xpath("//*[contains(normalize-space(.),\"Xác nhận thanh toán thành công!\")]")
        .isEmpty());
}
```

Các bước thực hiện	Kết quả mong muốn	Kết quả
1. Mở trang quản lý đặt vé 2. Ấn xác nhận thanh toán cho vé 3. Kiểm tra thông báo sau khi xác nhận	Xác nhận thanh toán thành công	Đạt

Test case ID: AB-11

Test case: Hủy vé thành công

```

@Test(description = "AB-11 Hủy vé thành công")
Run / Debug
public void case_AB_011() {
    loginAdmin();
    openBookingsPage();
    wait.until(
        ExpectedConditions.elementToBeClickable(By.cssSelector("[data-testid='admin-bookings-search-input']")))
        .click();
    wait.until(ExpectedConditions
        .visibilityOfElementLocated(By.cssSelector("[data-testid='admin-bookings-search-input']")))
        .clear();
    wait.until(ExpectedConditions
        .visibilityOfElementLocated(By.cssSelector("[data-testid='admin-bookings-search-input']")))
        .sendKeys("BK202604140004");
    wait.until(
        ExpectedConditions.elementToBeClickable(By.cssSelector("[data-testid='admin-bookings-search-submit']")))
        .click();
    wait.until(ExpectedConditions.elementToBeClickable(By.cssSelector(".lucide-x"))).click();
    wait.until(ExpectedConditions.elementToBeClickable(By.cssSelector("[data-testid='confirm-modal-confirm']")))
        .click();
    Assert.assertFalse(
        driver.findElements(By.xpath("//*[contains(normalize-space(.),\"Hủy vé thành công!\")]")).isEmpty());
}

```

Các bước thực hiện	Kết quả mong muốn	Kết quả
1. Mở trang quản lý đặt vé 2. Chọn vé 3. Ấn hủy vé 4. Kiểm tra thông báo hủy vé	Hủy vé thành công	Đạt

KẾT LUẬN

Sau thời gian thực hiện đồ án, dưới sự hướng dẫn tận tình của thầy Hoàng Văn Thông, kết quả mà em thu được cụ thể như sau:

Kết quả đạt được:

- củng cố và trau dồi kiến thức tổng quan về quy trình đảm bảo chất lượng và kiểm thử phần mềm.
- Hiểu rõ và nắm bắt được cách hoạt động của công cụ kiểm thử Selenium bao gồm Selenium IDE và Selenium WebDriver.
- Nắm bắt được kỹ năng phân chia thời gian và tổ chức công việc hợp lý trong quá trình thực hiện dự án.
- Biết cách viết Test Case, xây dựng kế hoạch kiểm thử, thực thi kiểm thử tự động và thống kê báo cáo lỗi một cách khoa học.
- Ứng dụng lý thuyết vào thực tiễn thông qua việc xây dựng các kịch bản kiểm thử tự động bằng Selenium cho hệ thống Booking Hub.

Hạn chế:

- Chưa nắm vững các kỹ năng lập trình nâng cao trong việc xây dựng các kịch bản kiểm thử tự động phức tạp.
- Kịch bản kiểm thử tự động chưa bao quát được hết toàn bộ các chức năng và các trường hợp ngoại lệ của website.
- Chưa ứng dụng được Selenium trong kiểm thử nâng cao như: Kiểm thử hiệu năng (Performance Testing), kiểm thử tải (Load Testing) hay kiểm thử đa trình duyệt.
- Hệ thống mới chỉ dừng lại ở việc kiểm thử bằng Selenium IDE và Selenium WebDriver mà chưa triển khai được Selenium Grid để kiểm thử phân tán.

DANH MỤC TÀI LIỆU THAM KHẢO